



**Tomás José
Cardoso Matos**

**Proteção de Credenciais FIDO2 Utilizando Intel
SGX**

Securing FIDO2 Credentials Using Intel SGX



Proteção de Credenciais FIDO2 Utilizando Intel SGX

Securing FIDO2 Credentials Using Intel SGX

Dissertação apresentada à Universidade de Aveiro para cumprimento dos requisitos necessários à obtenção do grau de Mestre em Cibersegurança, realizada sob a orientação científica do Doutor André Ventura da Cruz Marnôto Zúquete, Professor Associado do Departamento de Eletrónica, Telecomunicações e Informática da Universidade de Aveiro, do Doutor Tomás Oliveira e Silva, Professor Associado convidado do Departamento de Eletrónica, Telecomunicações e Informática da Universidade de Aveiro.

o júri / the jury

presidente / president

Prof. João Paulo Silva Barraca

Professor Associado da Universidade de Aveiro

vogais / examiners committee

Prof. Manuel Eduardo Correia

Professor Associado com agregação do DCE-FEUP

Prof. André Zúquete

Professor Associado da Universidade de Aveiro

**agradecimentos /
acknowledgements**

Agradeço toda a ajuda a todos os meus colegas e companheiros.

palavras-chave

FIDO2, SGX, Phishing, Comprometimento de Dados, Gestores de Credenciais.

resumo

À medida que a infraestrutura digital se expande, novos métodos de autenticação têm ganho popularidade tanto nas empresas como na comunidade de investigação, inovando e aumentando a segurança dos processos de autenticação. O FIDO2 é uma norma que visa substituir as senhas através de criptografia assimétrica com mecanismos de desafio-resposta, mantendo as chaves privadas armazenadas em ambientes de computação seguros. Como qualquer tecnologia, apresenta limitações, como o custo dos dispositivos físicos pessoais e os mecanismos de recuperação, que dificultam a sua adoção. Devido a estes problemas, este trabalho propõe um gestor de credenciais centralizado que armazena este tipo de credenciais e fornece autenticação resistente a phishing sem necessitar de investimento em hardware, com mecanismos de replicação para tolerância a falhas. Para lidar com violações de dados na aplicação centralizada, tiramos partido das características do Intel SGX, que fornece um ambiente de execução fidedigno e mecanismos de atestação para garantir que os utilizadores obtêm o comportamento esperado ao interagir com as suas credenciais.

keywords

FIDO2, SGX, Phishing, Data Breaches, Credential Managers.

abstract

As digital infrastructure expands, new authentication methods have become popular in both enterprises and the research community, innovating and increasing the security of authentication processes. FIDO2 is a standard that aims to replace passwords using asymmetric cryptography through challenge-response mechanisms, keeping private keys stored in secure computing environments. As with all technologies, it comes with drawbacks, such as the cost of personal physical devices and recovery mechanisms, that make adoption harder. Due to these problems, this work proposes a centralized credential manager that stores these types of credentials and provides phishing-resilient authentication without requiring hardware investment and with replication mechanisms for fault tolerance. To tackle data breaches in the centralized application, we leverage Intel SGX capabilities, which provide a secure execution environment and attestation mechanisms to ensure that users get the expected behavior when interacting with their credentials.

**acknowledgement of use of
AI tools**

**Recognition of the use of generative Artificial Intelligence
technologies and tools, software and other support tools.**

I acknowledge the use of Claude (Anthropic, <https://claude.ai/>) for writing and productivity, and for assistance with source code review. All design decisions, implementation work, and final editorial choices are my own.

Index

Index	i
List of Tables	iii
List of Figures	iv
Glossary	v
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement and Objectives	2
1.3 Proposed Approach	3
1.4 Contributions	4
1.5 Document Structure	5
2 State of the Art	6
2.1 Introduction	6
2.2 Trusted Execution Environments	6
2.2.1 TEE Architectures	6
2.2.2 SGX in Detail	7
2.2.3 SGX Security Analysis	8
2.2.4 SGX Applications	9
2.3 Authentication	11
2.3.1 Traditional Authentication Methods	11
2.3.2 FIDO Alliance Standards	12
2.3.3 CTAP Security Analysis	17
2.3.4 Attestation and Certificates	19
2.3.5 Credential Storage Models	20
2.3.6 Cryptographic Operations	21
2.4 Credential Management Systems	22
2.4.1 Password Managers	22
2.4.2 Passkeys and Synced Authenticators	23
2.4.3 Enterprise Credential Management	24
2.4.4 Credential Portability	25
2.5 Related Work	27
2.5.1 Linux UHID for Device Emulation	27
2.5.2 SGX-based Credential/Password Managers	28
2.5.3 FIDO2 with TEEs	28
2.5.4 Remote Attestation in Practice	29
2.6 Comparative Analysis	30
2.6.1 Security and Trust	31
2.6.2 Usability and Portability	31
2.6.3 Compatibility	32
2.6.4 Privacy and Performance	32
2.6.5 Key findings and Motivation	32

3	Proposed solution	34
3.1	Proposed architecture	34
3.1.1	User Host	35
3.1.2	SGX Host	36
3.2	Threat model and trust assumptions	36
3.2.1	Trusted components	36
3.2.2	Untrusted components	37
3.2.3	Adversaries considered	37
3.2.4	Known limitations and out-of-scope threats	38
3.3	Credential creation	39
3.3.1	Key derivation	39
3.3.2	Persistent record	39
3.4	User Authentication	40
3.4.1	Authentication with a Credential ID	41
3.4.2	Resident credentials	42
3.5	Credential migration	42
3.5.1	What is transferred	42
3.5.2	Protocol	43
3.5.3	Mutual attestation and key freshness	47
3.5.4	Migration of imported credentials	48
3.5.5	Trust root for quote verification	48
3.6	Security properties	49
3.6.1	Confidentiality of credential material at rest	49
3.6.2	Unlinkability of stored records	49
3.6.3	Integrity of stored records	50
3.6.4	End-to-end channel confidentiality	50
3.6.5	Mutual attestation in migration	50
3.6.6	Forward secrecy of migration transfers	51
3.6.7	Summary	51
4	Implementation	53
4.1	Authentication Emulator	53
4.1.1	UHID device	53
4.1.2	CTAP Interface	56
4.1.3	User Attribute Collection and UID derivation	60
4.2	Communication channel	64
4.2.1	Per-request handshake	64
4.2.2	Wire format	65
4.3	SGX Host	66
4.3.1	HTTP Service	66
4.3.2	Enclave	69
4.3.3	Storage	85
5	Evaluation	89
5.1	Test Bench	89
5.2	Communication with the SGX Host	89
5.3	Test Client	90
5.3.1	Timing Methodology	90
5.4	Results	91
5.4.1	First-Packet Measurement	92
5.4.2	Last-Keepalive Measurement	92
6	Conclusion	94
6.1	Conclusion	94
6.2	Future Work	95
	References	97

List of Tables

2.1	Cryptographic Operations in FIDO2	21
2.2	Comparison of credential management approaches	30
3.1	Mapping of security properties to threats from the threat model.	51
5.1	Per-operation timing (ms) for the hardware FEITIAN device and the SGX-backed virtual authenticator, with post-touch defined as first response packet to end of response (definition (b)). Real-device measurements exclude the user-presence wait. Excludes device-side cryptographic work, which is not observable from the host. Single execution per cell.	91
5.2	Per-operation timing (ms) for the hardware FEITIAN device and the SGX-backed virtual authenticator, with post-touch defined as last keepalive to end of response (definition (a)). Real-device measurements exclude the user-presence wait. Includes part of the user-presence window, as discussed in Section 5.3.1. Single execution per cell.	92

List of Figures

2.1	PIN authentication	18
3.1	Architecture of the proposed SGX-based FIDO2 credential manager. The User Host (left) runs the client application — typically a browser — and the Authenticator Emulator, which speaks CTAP to the client. The SGX Host (right) exposes an HTTP service that forwards requests to the SGX enclave, the only trusted component beyond SGX hardware itself. Credential records are persisted in the host’s database, sealed by the enclave.	35
3.2	Migration, phase 1. The user requests migration through the Authenticator Emulator, which asks the destination SGX host for a quote. The destination’s enclave produces a quote attesting its stable IK-derived public key P_{dst} via the report data field.	44
3.3	Migration, phase 2. The source enclave verifies the destination’s quote, generates a fresh ephemeral keypair (e_{priv}, e_{pub}) , produces its own quote binding e_{pub} into report data, and constructs an ECIES-style envelope. It then enumerates the user’s credentials, decrypts each under its local IK (upgrading derived credentials to migrated form), encrypts the serialized records under the fresh payload key K_{pay} , and returns the bundle to the Emulator.	45
3.4	Migration, phase 3. The destination enclave verifies the source’s quote and crucially checks that the ephemeral public key e_{pub} in the transfer envelope matches the one attested in the source’s quote. After this binding check, it completes the ECDH exchange, unwraps the payload key K_{pay} , decrypts the credential payload, and re-indexes each record under its own IK before insertion.	47
4.1	The Emulator’s attribute-collection panel during a registration request. The header shows the relying party and the user account; the five entry fields collect the user attributes; preset buttons populate common values for testing.	61
4.2	In an authentication with resident credentials, after the attribute-collection requests the user’s choice on which user to authenticate.	62
4.3	The migration panel from two services and enclaves located on the same host but in diferent ports. The two skipped credentials were credential already migrated and the two insert, credentials that were added after that previous migration.	63

Glossary

AAGUID	Authenticator Attestation Globally Unique Identifier	IK	Identity Key
AEAD	Authenticated Encryption with Associated Data	ISV	Independent Software Vendor
AES	Advanced Encryption Standard	ISVSVN	ISV Security Version Number
AES-GCM	AES in Galois/Counter Mode	IV	Initialisation Vector
AK	Attestation Key	JSON	JavaScript Object Notation
API	Application Programming Interface	KDF	Key Derivation Function
CA	Certificate Authority	MAC	Message Authentication Code
CBOR	Concise Binary Object Representation	PCCS	Provisioning Certification Caching Service
CID	Channel Identifier	PCE	Provisioning Certification Enclave
COSE	CBOR Object Signing and Encryption	PCK	Provisioning Certification Key
CPU	Central Processing Unit	PCS	Provisioning Certification Service
CRL	Certificate Revocation List	PEM	Privacy-Enhanced Mail (certificate encoding format)
CTAP	Client-to-Authenticator Protocol	PRK	Pseudo-Random Key
CTAP-HID	CTAP over Human Interface Device	PRF	Pseudo-Random Function
DCAP	Data Center Attestation Primitives	ProdID	Product Identifier
DER	Distinguished Encoding Rules	PSE	Platform Service Enclave
ECDH	Elliptic Curve Diffie-Hellman	QE	Quoting Enclave
ECDSA	Elliptic Curve Digital Signature Algorithm	RP	Relying Party
ECIES	Elliptic Curve Integrated Encryption Scheme	SDK	Software Development Kit
EDL	Enclave Definition Language	SGX	Software Guard Extensions
EID	Enclave Identifier	SHA	Secure Hash Algorithm
EPID	Enhanced Privacy Identifier	SQL	Structured Query Language
ES256	ECDSA with SHA-256 (COSE algorithm –7)	SVN	Security Version Number
FIDO	Fast IDentity Online	TCB	Trusted Computing Base
FIPS	Federal Information Processing Standards	TDX	Trust Domain Extensions
FLC	Flexible Launch Control	TEE	Trusted Execution Environment
FMSPC	Family-Model-Stepping-Platform-CustomSKU	TLS	Transport Layer Security
GUI	Graphical User Interface	TLV	Tag-Length-Value
HID	Human Interface Device	TPM	Trusted Platform Module
HKDF	HMAC-based Key Derivation Function	TTL	Time To Live
HMAC	Hash-based Message Authentication Code	U2F	Universal 2nd Factor
HSM	Hardware Security Module	UID	User Identifier
HTTP	Hypertext Transfer Protocol	UHID	Userspace Human Interface Device
HTTPS	HTTP Secure	URTS	Untrusted Runtime System
		USB	Universal Serial Bus
		WebAuthn	Web Authentication

Introduction

1.1 Context and Motivation

Authentication through password credentials has become fundamental to our digital lives, but it is facing serious challenges. Recent reports show worrying trends in how credentials are being compromised, making it clear that we need better ways to authenticate users.

Phishing attacks are getting worse at an alarming rate. The KnowBe4 Phishing Threat Trends Report (2024-2025)¹ and the Cybercrime Information Center's 2025 Phishing Landscape² show that phishing attacks jumped by 17.3% in just six months. What is particularly concerning is that 82.6% of these malicious messages now use AI-generated content to slip past traditional filters. Check Point's analysis³ adds to this picture: compromised credentials surged by 160% in early 2025, with over 14,000 employee credentials exposed in just one month. Attackers are getting better at exploiting human behavior through increasingly sophisticated AI-powered social engineering tactics.

The problems with traditional passwords are well documented. Research by the National Institute of Standards and Technology (NIST) highlights that users struggle to maintain unique passwords across multiple services [1], leading to widespread password reuse. When one service experiences a data breach, attackers leverage these credentials in credential stuffing attacks against other platforms. Verizon's Data Breach Investigations Report consistently shows that stolen credentials remain among the top attack vectors⁴. Even users who create strong, unique passwords for each service are not safe from phishing attacks that simply trick them into handing over their credentials.

Several solutions exist, but each comes with its own problems. Multi-Factor Authentication (MFA) adds an extra security step, making phishing much harder. But being honest: most users find it annoying to pull out their phone or wait for a code every time they want to log in. Studies on authentication usability [2], [3] show that friction in authentication processes leads to reduced

¹KnowBe4 Research, *Phishing Threat Trends Report 2024-2025*, available at: <https://www.knowbe4.com/resources/reports/phishing-by-industry-benchmarking-report>

²Cybercrime Information Center, *2025 Phishing Landscape Analysis*, available at: <https://cybercrimeinfocenter.org/phishing-landscape-2025>

³Check Point Research, *Cyber Security Report 2025: Compromised Credentials Analysis*, available at: <https://www.checkpoint.com/cyber-hub/threat-prevention/what-is-phishing/>

⁴Verizon, *2024 Data Breach Investigations Report*, available at: <https://www.verizon.com/about/news/breach-industry-wide-dbir-finds>

adoption rates. This friction leads many to avoid MFA altogether [4].

FIDO2 authentication⁵ is genuinely phishing-resistant because it uses cryptographic keys instead of passwords that people can be fooled into revealing. The catch? You need to buy a hardware security key, carry it with you, and worry about what happens if you lose it.

Password managers offer a practical middle ground. They store all your passwords in an encrypted vault, generate random passwords for each service, and only require you to remember one master password. Many include MFA to access the vault itself. This nicely solves the password reuse problem. But there is a fundamental trust issue: you are putting all your eggs in one basket and hoping the password manager provider has perfect security. Recent incidents involving password manager breaches demonstrate this risk⁶. If their system gets compromised, or if they are not as trustworthy as they claim, all your credentials could be exposed at once. FIDO2 also evolved to being supported by password managers, such as Windows Hello⁷, and currently the term passkeys refers to FIDO2 credentials that are managed by password managers instead of hardware tokens.

This thesis explores how we can address these challenges using Intel SGX. The goal is to combine the convenience of centralized password managers with something traditional solutions lack: verifiable proof that the system is actually doing what it claims. By integrating SGX's attestation capabilities [5] with modern FIDO2 authentication, we can build a credential management system that does not require users to simply trust that the provider is being honest and secure.

1.2 Problem Statement and Objectives

This work tackles several interconnected problems in modern authentication systems. First, centralized credential managers, whether cloud-based or self-hosted, require users to trust that the service provider is secure and honest. When these systems get breached, every stored credential is potentially compromised. There is currently no way for users to verify that their password manager is actually running the code it claims to be running.

Second, FIDO2 tokens offer excellent phishing protection [6], but come with practical limitations. Users have to purchase them, keep track of them, and have backup strategies in case they are lost or damaged [7], [8]. Beyond usability concerns, the CTAP protocol between clients, like browsers, and FIDO2 authenticators can also introduce vulnerabilities, particularly the Man-in-the-Middle (MITM) attack that exploits weaknesses in the Diffie-Hellman key exchange governing PIN negotiation, as well as the ability to tamper with critical messages, including authenticator metadata (fundamental to guide clients' behavior and security requisites) and the registration and authentication process themselves.

⁵FIDO Alliance, *FIDO2: WebAuthn & CTAP Specifications Overview*, available at: <https://fidoalliance.org/fido2/>

⁶Goodin, D., *Password Manager LastPass Warns of Breach Allowing Access to Customer Data*, Ars Technica, 2022, available at: <https://arstechnica.com/information-technology/2022/12/lastpass-says-hackers-have-obtained-vault-data-and-a-wealth-of-customer-info/>

⁷Microsoft, *Support for passkeys in Windows*, available at <https://learn.microsoft.com/en-us/windows/security/identity-protection/passkeys/>

Third, keeping all the credentials centralized poses concerns related to availability and load balancing, so including a migration strategy is therefore important considering the relevance of the use case, as it could compromise the user’s full account access.

Related to this topic of credential migration, some questions must be answered. How can a user trust that the credentials are propagating correctly? How do we prevent malicious servers from serving outdated or tampered credentials?

In short, the core objective of this thesis is to design and implement a credential management system that addresses these issues. Specifically, aiming to:

- Enable users to verify the correctness of credential management code through remote attestation, eliminating blind trust in service providers;
- Integrate FIDO2 authentication capabilities within SGX enclaves, combining the phishing resistance of hardware tokens with the convenience of software-based authentication;
- Support multiple server instances where users can confidently choose any server, knowing through attestation that it is running the correct, unmodified code;
- Demonstrate that SGX’s security properties, particularly memory encryption and isolated execution, can effectively mitigate relevant attack vectors in this use case;
- Reduce the barriers to strong authentication by removing the need for physical tokens while maintaining similar security guarantees;

In essence, building a credential manager that does not require users to trust the operator, while maintaining the usability advantages of traditional password managers and the security benefits of FIDO2.

1.3 Proposed Approach

The solution focuses on a specific component of FIDO2: the Client to Authenticator Protocol (CTAP). The idea is to build a software-based authenticator that browsers and other clients will treat as if it were a physical FIDO2 device. When a user attempts to authenticate to a website, the browser communicates with the emulated authenticator, which then connects to the SGX-based credential manager running on a remote server.

Our solution requires participation in this protocol to interact with all clients, not just browsers, meaning the security risks that already exist remain a concern. We cannot guaranty that messages exchanged between these two entities are free from tampering by an intermediary. However, we consider a MITM attack on this protocol highly unlikely in practice, as it presupposes a local host compromise [9]. Some research efforts address these issues [10], but solving them falls outside the scope of this work.

We mitigate some exposure by blocking and controlling certain dangerous CTAP commands, such as credential resetting instructions, as well as making our emulated authenticator resilient to the permission vulnerabilities described in [11].

The registration flow works as follows: the client-side component (our authenticator emulation) first selects which credential manager server to use and remotely attests it using SGX’s

attestation mechanisms. This allows the client to verify that the server is actually running the correct, unmodified code. Once that trust is established, the emulated authenticator interacts with the SGX server, where the user is prompted with input fields that are hashed to generate a unique identifier. With the registration data (the relying party identifier, the user identifier in that relying party and the generated unique identifier) the SGX server computes the credential id and the key pair, that are derived from the specific SGX capacities. The public key and the credential identifier are sent to the client, which forwards them to the relying party according to the FIDO2 specifications. To enable credential migration, it is necessary to store the necessary data to list all a specific user credentials.

During the authentication phase, a relying party will send a credential identifier that combined with the user's unique identifier will provide the necessary data so that SGX can derive the key pair and answer the challenge correctly.

To prevent data loss if a server fails, we rely on multiple servers working together. For handling credential backups and migrations between servers, we designed the system so the user manages this process rather than having SGX servers communicating directly with each other. Users, using the emulated authenticator, can synchronize their data by simply fetching it from one server and sending it to all the others, with the trust assumption that the server receiving it is running honest code.

1.4 Contributions

The main contributions of this work are the following:

- A virtual FIDO2/WebAuthn authenticator implemented entirely in software, with its credential material protected inside an Intel SGX enclave and exposed to the operating system through the Linux UHID interface.
- A per-request encrypted channel between the Authenticator Emulator and the enclave, established through ECDH against the enclave's remotely attested public key, with no transport-layer trust required.
- A credential migration protocol that transfers credentials directly between two SGX hosts under mutual remote attestation, without any cloud or third-party custodian in the trust chain.
- An evaluation of the resulting system against a hardware FIDO2 token used as a baseline, measured end-to-end through the same kernel and CTAP-HID interface.

Part of the work here described was presented at The First International Conference on Security and Cybersecurity in the AI and Digital Context (CYBERSEC 2026) with the title "Securing FIDO2 Credentials Using Intel SGX", co-authored with André Zúquete and Tomás Oliveira e Silva. The event took place in June 7-11, 2026, at Porto, Portugal.

1.5 Document Structure

Chapter 2: State of the Art. Reviews the technical background relevant to this work: trusted execution environments and Intel SGX, the FIDO2/WebAuthn authentication standards, and existing credential management systems, including passkeys and synced authenticators. Closes with a comparative analysis that motivates the design choices of the proposed solution.

Chapter 3: Proposed Solution. Presents the system architecture, threat model, and trust assumptions. Defines the cryptographic constructions used for credential creation, authentication, and migration between SGX hosts, and states the security properties the design aims to provide.

Chapter 4: Implementation. Describes the realisation of the design across three components: the Linux Authenticator Emulator that bridges CTAP to the enclave, the per-request communication channel that secures host-to-enclave traffic, and the SGX host comprising the HTTP service, the enclave itself, and the storage layer.

Chapter 5: Evaluation. Describes the test bench and timing methodology used to measure the system end-to-end, and reports the results obtained against a hardware FIDO2 token used as a reference.

Chapter 6: Conclusion. Summarises the contributions of the work and the position it occupies relative to existing FIDO2 authenticator designs. Also, identifies the gaps left open by the current implementation and the directions in which the system could be extended.

State of the Art

2.1 Introduction

This chapter examines the literature that is relevant to our research. We will cover three main topics: Intel SGX and how it is being used for security applications, FIDO2 authentication, and different approaches to credential management which have affected our thinking. Regarding SGX, we are curious to find out how other researchers and companies have applied its features, primarily remote attestation and isolated execution, to create secure systems. With FIDO2, we want to learn about how the protocols that rule it work and how can we deploy an authenticator without dedicated hardware. For credential managers, we will compare commercial products to research prototypes to understand what worked well and what problems still need solving.

2.2 Trusted Execution Environments

2.2.1 TEE Architectures

Trusted Execution Environments (TEEs) have become essential in modern security, though people often misunderstand what they actually do. The basic idea is simple: TEEs create isolated spaces where sensitive code and data can run protected from everything else on the system, for example: your OS, hypervisor, other apps, all of it [12]. Imagine a secure room inside your computer where code can execute safely. Even if an attacker takes over the entire rest of your system, they still cannot see inside that room or mess with what is happening there.

Hardware vendors have built TEEs in pretty different ways. Intel's SGX creates small protected areas called "enclaves" using memory encryption and processor-level isolation. Your sensitive code runs in these enclaves, completely shielded from everything outside. ARM went a different route with TrustZone, where they basically split the processor into two separate worlds. There is a "normal world" for everyday stuff and a "secure world" for anything sensitive. AMD's SEV (Secure Encrypted Virtualization) targets a different use case: it protects entire virtual machines by encrypting their memory, so even the hypervisor running underneath cannot snoop on what is inside. Then there is RISC-V Keystone, also newer and open source, allowing developers more freedom to control how their TEE works. But each of these approaches makes different decisions about what threats actually matter and where they draw the security border.

2.2.2 SGX in Detail

SGX is Intel's trusted execution environment technology, first introduced in 2015 with their 6th generation (Skylake) processors¹. As we mentioned earlier the room analogy, you can also think of it like a safe inside a bank; even if attackers break into the bank and control everything, they still cannot crack open the safe to see what is inside.

Getting more technical about how this actually works. SGX reserves a special region of memory called the Processor Reserved Memory (PRM) [5]. The CPU actively protects this region from everyone, the operating system, hypervisor, even System Management Mode (SMM) code, and DMA requests from peripherals. Nobody gets in except the enclave itself. Inside the PRM sits the Enclave Page Cache (EPC), which is divided into 4 KB pages that hold the actual enclave code and data. The operating system is still responsible for managing which pages get assigned to which enclaves, but the CPU keeps track of everything in the Enclave Page Cache Metadata (EPCM) to make sure each page belongs to exactly one enclave.

Here is where it gets interesting: the untrusted operating system actually loads the initial code and data into an enclave. During this loading phase, the OS asks the CPU to copy data from regular memory into EPC pages. This means the OS knows what the initial enclave state looks like. But here is the clever part: while the enclave is being loaded, the CPU calculates a cryptographic hash of everything going in. Once loading is done and the enclave gets marked as initialized, that hash becomes the enclave's "measurement", basically a fingerprint of exactly what code and data are inside [5]. After initialization, the loading mechanism gets disabled, so nothing can be added or changed.

This measurement hash is crucial for remote attestation. A remote party can go through an attestation process to verify they are actually talking to an enclave with a specific measurement hash running in a secure environment. It is proof that the code is what you expect it to be.

The way you interact with enclaves requires special CPU instructions. Your regular application code (running outside the enclave in what is called "untrusted" space) can call functions inside the enclave using "ecalls" (enclave calls). The enclave can also call back out using "ocalls" (out calls) when it needs something from the outside world. Enclave code always runs in ring 3 (user mode) and uses the same address translation set up by the OS, the enclave does not get its own page tables.

One tricky bit: when an interrupt or page fault happens while enclave code is running, the CPU cannot just handle it normally because that would leak enclave state. Instead, it performs an Asynchronous Enclave Exit (AEX). The CPU saves the enclave's state into a protected area inside the enclave itself, switches to non-enclave code, and fills all the registers with synthetic dummy values before handling the interrupt. This prevents the OS from seeing any real enclave data [5].

The OS is also responsible for managing EPC pages, it can even evict them to regular DRAM if it runs out of EPC space. But SGX protects evicted pages using encryption to guarantee their confidentiality, integrity, and freshness while they are sitting in untrusted memory.

¹Intel Corporation: Intel® Software Guard Extensions Programming Reference, 2016. Document Number: 329298-002US. <https://www.intel.com/content/dam/develop/external/us/en/documents/329298-002-629101.pdf>

It is also important to denote how does the encryption works, how is the key management and how does the EEM (Enclave Execution Mode) encrypt the EPC as well as what makes it all secure.

The real magic happens in the Memory Encryption Engine (MEE), which is built directly into the CPU. The MEE automatically encrypts every EPC page before it gets written to RAM and decrypts it when it is read back in. This means that even if someone physically accesses your RAM chips or uses a hardware debugger, all they will see is encrypted data.

But encryption is only useful if the keys are secure. SGX uses a hierarchical key structure that starts with keys burned into the processor during manufacturing. At the root is the Root Provisioning Key, which is fused into the CPU and never leaves it, not even Intel knows this key after the processor is made. There is also the Root Sealing Key, another fused secret that is unique to each processor [5].

From these root keys, SGX derives other keys for specific purposes. The most important ones are the sealing keys and attestation keys. Sealing keys let an enclave encrypt data so it can be stored outside the enclave (say, on disk) and later decrypted by the same enclave or by other enclaves with the same measurement hash. This is how enclaves can persist data across reboots. The attestation keys are used during remote attestation to prove to a remote party that they are talking to a legitimate enclave running on real SGX hardware.

Here is what makes this secure: all these keys are derived inside the CPU using the enclave's measurement hash and other metadata. The actual key derivation happens in hardware, and the derived keys never leave the CPU in plaintext. When an enclave wants to seal data, it asks the CPU to derive the sealing key, the CPU encrypts the data, and then the key is immediately erased from registers. The encrypted data can safely leave the processor, but only an enclave with the right measurement (and running on the same CPU) can ask the processor to derive the same key again to decrypt it.

The MEE also handles integrity protection, not just confidentiality. It uses a Message Authentication Code (MAC) tree structure to detect if any EPC page has been tampered with or rolled back to an older version. Every time a page is written, the MEE updates the MAC tree. When reading a page, it verifies the MAC to ensure nobody has modified the encrypted data in RAM. This protects against replay attacks where an attacker might try to restore old encrypted data [5].

2.2.3 SGX Security Analysis

Given the need for secure computation throughout this thesis, a security assessment of Intel SGX and TEE implementations is crucial. We focus on documented vulnerabilities and attack vectors, along with mitigation strategies.

Although Intel SGX provides hardware-based isolation, there are numerous identified attack categories within its real-world deployment that demonstrate limitations in the technology [13]. Speculative execution attacks such as Foreshadow (L1 Terminal Fault) demonstrated that attacks were able to retrieve enclave data based on how modern CPUs perform speculative operations [14]. Correspondingly, MDS attacks targeted internal CPU buffers and leakage of information coming from enclaves [15]. Intel made some fixes, like adding microcode patches

and software updates, but these changes often come with performance trade-offs that degrade enclave efficiency^{2,3}.

Physical attacks are another issue. Using voltage manipulation, Plundervolt was able to induce errors and thus compromise both data integrity and confidentiality in SGX calculations [16]. Intel fixed this by restricting access to voltage control, but this attack revealed how performance tuning made for hardware features is a security vulnerability.

The recent TEE.fail research goes further in this direction with its work by exposing memory bus interposition attacks on DDR5 systems [17], demonstrating that access to memory channels physically can bypass the entire set of defenses set up by the enclaves around these devices.

The SGX SDK has been a source of potential vulnerabilities, with the problem of improper initialization [18] for example, as it was an important attack vector in many cases, privilege escalation [19] and information leaks [20]. These software-level issues are extremely problematic, as they can seriously affect any application built on top of any existing SDK versions which have vulnerabilities.

2.2.4 SGX Applications

To better understand how SGX fits into our password manager design, it helps to look at how others have actually used this technology. Zheng et al. [21] did a comprehensive review of SGX applications that shows some interesting patterns in what works and what does not.

The most straightforward use of SGX has been for encryption. Systems like Iron [22] put cryptographic operations inside enclaves to protect keys and sensitive data. This makes sense: if you are going to trust SGX, protecting your encryption keys is a natural starting point. Which is just what we want for a password manager.

Software partitioning is yet another approach where developers divide their apps into trusted and untrusted segments. To make the whole thing more compact, one idea would be to keep the enclave small by only putting security-critical code into it. This reduces the attack surface though figuring out where to line up the trusted and untrusted code is not typically easy. It means for us that we need to decide what to include in the enclave and what can be left outside.

People have built some creative frameworks with SGX. EnclaveDB [23] protects database queries even from database administrators who might have malicious intent. BASTION-SGX [24] secures Bluetooth connections. There is even work on protecting video games from cheaters by putting game logic in enclaves. These examples show that SGX can integrate many existing systems.

What is particularly relevant for our work is how SGX has been used in cloud environments. Projects such as SecureCloud [25] and SCONE [26] demonstrate that you can run SGX-protected services on infrastructure you do not control. They've solved tricky problems like migrating virtual machines that contain enclaves and scaling services up and down while maintaining

²<https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/11-terminal-fault.html>

³<https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/microarchitectural-data-sampling.html>

security guarantees. This is crucial because most people are not going to run their own password manager server at home.

What we take from all this is straightforward: SGX has been successfully used to protect exactly the kinds of things a password manager needs to protect. Cryptographic keys, integration with existing authentication systems, and operation on untrusted cloud infrastructure with verifiable security guarantees have all been addressed in practice. The technology is not perfect, as we discussed with the vulnerabilities, but there is a solid foundation of real applications that have made it work in practice.

Zheng et al. [21] in their paper also referred to some other application categories like Network Security and Privacy with applications like SGX-Tor [27] to leverage secure and anonymous routing, LightBox [28] for network middleware that prevents metadata leakage, and various Network Function Virtualization (NFV) solutions [29] that process packets inside enclaves.

Beyond cloud computing, SGX has been applied to Big Data and Analytics. Projects like VC3 [30] run MapReduce computations on untrusted infrastructure demonstrating that SGX can manage high-throughput data processing with privacy guarantees.

An interesting category is Legacy Application Protection. Haven [31] and Graphene-SGX [32] let you run existing Windows and Linux applications inside enclaves without modifying their code. They also take a Library OS approach to intercept system calls and manage them securely. This matters because rewriting every application to use SGX is not realistic.

For developers, there are tools that make working with SGX easier. Glamdring [33] automatically figures out which parts of your C code should go in the enclave. AsyncShock [34] helps find synchronization bugs that could compromise enclave security. Moat [35] verifies that your enclave program does not accidentally leak secrets. These tools address the fact that writing secure enclave code is hard and error-prone.

SGX has also found its way into Digital Rights and IP Protection. SGXCrypter [36] protects executables from reverse engineering, while game developers use enclaves for anti-cheat systems [37]. Considering that potential cheaters cannot see how the code works, they cannot easily bypass the protections.

Finally, some interesting use cases worth mentioning. PRESAGE [38] does privacy-preserving genetic testing, where even the testing service cannot see your DNA data. Others have used SGX for Byzantine fault tolerance protocols [39]. These niche applications show that SGX's security model can adapt to very different problem domains.

Zheng et al. [21] made this study in 2021, some newer use case categories have grown significantly, particularly as hardware limitations were overcome and production deployments became commonplace.

The most relevant advancements have been Blockchain and Confidential Smart Contracts. Unlike the traditional blockchain system in which all sensitive data is accessible, SGX makes it so that private computations which the blockchain can validate without needing the actual data [40]. Projects like Secret Network⁴ implemented secure smart contracts with confidential storage of transaction data encrypted while in use. But this same project also had a major security incident in 2022, when xAPIC, MMIO and similar vulnerabilities allowed attackers to

⁴<https://scrt.network/>

possibly extract the consensus seed [41]. This highlighted the potential and pitfalls of turning to SGX for blockchain privacy⁵.

Privacy-Preserving Machine Learning matured significantly during this period. The key breakthrough was solving SGX's memory limitations. Microsoft Azure's DCsv3-series VMs, launched in 2022, increased the Enclave Page Cache from 128MB to 256GB - a 1500x increase - and expanded CPU cores to 48 [42]. This made real AI workloads practical. Intel released frameworks for running TensorFlow inside SGX enclaves with full remote attestation support [43]. By November 2023, Microsoft processed \$25 billion in annual credit card transactions through SGX-protected systems, which leads to Level 1 PCI-DSS compliance [44].

Enterprise Confidential Computing moved from experimental to mainstream. Cloud providers now offer SGX as standard infrastructure rather than experimental features. Azure's confidential computing services support not just virtual machines but also Kubernetes clusters with SGX-enabled worker nodes for confidential containers [45]. Companies can now "lift and shift" existing workloads into SGX-protected environments without major code modifications, using library OS solutions like Gramine⁶.

However, Intel did make a strategic move in 2021 and it has to be noted that they deprecated SGX from consumer processors (11th and 12th gen Intel Core) and doubled down on Xeon⁷ for cloud and enterprise use. This reflects where SGX truly discovered its market: Not in consumer devices, but in data centers featuring cloud services that require hardware-backed confidential computing guarantees.

2.3 Authentication

With the SGX overview complete, the next step crucial to this thesis development is the authentication part. Here we will refer to traditional authentication methods that have some concerns as mentioned earlier, the FIDO Alliance standards, a review of the CTAP protocol, and types of authenticators that can be part of the FIDO authentication pipeline.

2.3.1 Traditional Authentication Methods

Some regular authentication methods are still highly relevant today and widely used by the majority of the population. But what exactly are these traditional methods?

The king of authentication is still password-based authentication. Despite decades of security research pointing out its flaws, passwords remain the default way most people prove their identity online. The concept is simple: while in registration phase users create a secret string associated with his username or user id and whenever they want access they account it is only necessary to prove the knowledge of that secret by typing it. The problem is that humans are terrible at creating and remembering strong passwords. We reuse them across sites, pick predictable patterns like "Password123", or write them on sticky notes attached to our monitors.

⁵Hyperledger Labs Fabric Private Chaincode: <https://hyperledger.github.io/fabric-rfcs/text/0000-fabric-private-chaincode-1.0.html>

⁶<https://github.com/gramineproject/gramine>

⁷<https://www.intel.com/content/www/us/en/architecture-and-technology/software-guard-extensions-processors.html>

Other authentication form that sees through simple passwords is the Multi-Factor Authentication (MFA), which tries to patch password security by requiring another forms of authentication in addition to the password. This usually includes three categories: something you know (password, PIN), something you have (phone, security token), or something you are (fingerprint, face). Common implementations include SMS codes sent to your phone, time-based one-time passwords (TOTP) from apps like Google Authenticator, or email verification links. MFA significantly improves security over passwords alone, but it is not foolproof — SMS codes can be intercepted via SIM swapping attacks and TOTP codes might be phished if the user is not careful.

Other classic techniques that are seldom used today include security questions (“What was your first pet’s name?”), which are often easier to guess and more common to research than passwords, and PIN codes, which are basically short numeric passwords, usually for physical devices or payment cards.

The fundamental limitation of all these traditional methods is that they are based on shared secrets. When you authenticate in a website, what you are doing is sending your password to that site’s server. If that server gets compromised - and major breaches happen regularly - attackers gain access to millions of passwords at once. Even when properly hashed and salted, password databases become targets for offline cracking attacks. And because users reuse passwords, one breach can cascade into many compromised accounts across different services. Beyond server-side breaches, users are commonly tricked into submitting their credentials on replicas of legitimate websites, the phishing attacks that have become an increasingly prevalent threat [1].

This is the core problem that modern authentication methods like FIDO2 aim to solve by moving away from shared secrets entirely.

2.3.2 FIDO Alliance Standards

To address the problems related to traditional authentication methods, the FIDO (Fast IDentity Online) Alliance developed a set of standards that provide an innovative and phishing-resistant solution⁸. The alliance aims to reduce the world’s reliance on passwords through open, scalable, and interoperable authentication standards.

The evolution of the standards of FIDO has been a matter for various generations. FIDO Universal 2nd Factor (U2F), introduced in 2014, was the first standard adopted. It gave a simple way of introducing hardware-based, second-factor authentication via USB security keys. However, as the name infers U2F was designed as a second factor, so it is still required a password as the first factor ⁹.

Then launched in 2018 appeared FIDO2, which combines two protocols: WebAuthn (Web Authentication) and CTAP (Client to Authenticator Protocol). WebAuthn is protocol that ensures the authentication and registry from a random relying party to a Client (possible a browser),

⁸The FIDO Alliance was founded in 2012 by PayPal, Lenovo, Nok Nok Labs, Validity Sensors, Infineon, and Agnitio. See: <https://fidoalliance.org/overview/>

⁹FIDO Alliance Specifications: <https://fidoalliance.org/specifications/>

specifying message formats and the communication between those entities¹⁰, while CTAP defines how authenticators (like security keys) communicate with the client¹¹. Unlike U2F, FIDO2 enables true passwordless authentication so users can authenticate using only biometrics or a PIN with their authenticator and not sending them to a relying party.

More recently, the industry has converged around passkeys, which are FIDO2 credentials that sync across devices within the same ecosystem¹². This handles one of the pivotal adoption barriers: device loss or damage. Traditional FIDO security keys store credentials locally, so losing the key means you lose your access to accounts. Passkeys solve this problem by securely syncing credentials through encrypted cloud storage across devices (which enhances usability and guarantees security).

Although we can recognize these advantages, there are some practical challenges that pose negative aspects for end users. The most notable is that physical security keys typically cost €20-€80. This can be prohibitive for users who do not perceive their accounts as high-value targets and do not have the financial means to spend money on that. There is also a user knowledge gap, where most people do not understand what security keys are or how to use them. As well as the user-side challenges, there is also an industry gap, as most web applications that we use nowadays do not yet support FIDO2 authentication.

For these reasons, FIDO authentication is often deployed as part of a multi-factor authentication strategy rather than the sole authentication method. Services typically allow users to register multiple authenticators as backups, and many still offer password-based fallback options. However, the industry trend is moving toward passwordless-first designs, with passwords becoming the backup rather than the primary authentication method¹³.

2.3.2.1 CTAP Protocol Deep Dive

As previously mentioned, FIDO2 authentication is based on two existing protocols involved as a pair: WebAuthn and CTAP. WebAuthn defines a way for the relying party to communicate to a client using standard message formats and cryptographic operations; this thesis concentrates on CTAP and considers WebAuthn to operate according to supposed. CTAP (Client to Authenticator Protocol) defines the communication layer between the client platform (such as a browser or operating system) and the authenticator device itself (such as a USB security key, smartphone, or platform authenticator).

Although WebAuthn does not have much impact in this work, as it will be operating as expected, it is essential to understanding its mechanics in order to grasp the complete authentication flow. The protocol defines several message types that are exchanged between the two communicating entities. During the registration phase, the relying party (web server) sends a challenge along with user and relying party information, to which the client after communicating

¹⁰W3C: Web Authentication - An API for accessing Public Key Credentials Level 2, 2021. <https://www.w3.org/TR/webauthn-2/>

¹¹FIDO Alliance: Client to Authenticator Protocol (CTAP), 2021. <https://fidoalliance.org/specs/fido-v2.1-ps-20210615/fido-client-to-authenticator-protocol-v2.1-ps-errata-20220621.html>

¹²FIDO Alliance Passkeys: <https://fidoalliance.org/passkeys/>

¹³Google Security Blog: "The beginning of the end of the password," 2023. <https://security.googleblog.com/2023/05/so-long-passwords-thanks-for-all-phish.html>

with the authenticator responds with a newly created public key credential. In the authentication phase, the server sends an authentication challenge, and the client returns a signed assertion proving possession of the corresponding private key. The client receives these structured requests from the relying party and translates them into commands that the authenticator can execute via CTAP, making WebAuthn and CTAP the bridge between web-based services and hardware authenticators.

With a small overview of the WebAuthn protocol, let's deep dive into CTAP. When a web application initiates a FIDO2 registration or authentication, the browser invokes the WebAuthn API through `navigator.credentials.create()` (for registration) or `navigator.credentials.get()` (for authentication). This triggers the client platform to begin discovering authenticators capable of participating in the operation, so initially there is a phase of authenticator enumeration. In this step, the client inspects whether USB security keys, NFC readers, BLE-based authenticators, or platform-integrated secure modules are available, and determines which expose FIDO2-compatible interfaces. For example, for HID-based devices, the browser filters candidates using the HID usage page `0xF1D0`, a value assigned specifically to FIDO authenticators. Devices may also signal support via capability flags (e.g., `ID_FIDO_TOKEN = 1`) that indicate compatibility with CTAP, FIDO2, or legacy U2F.

After identifying eligible authenticators, the client establishes communication solely through the transport-specific CTAP interface defined by FIDO. This isolation is essential to maintain security: any state accessible through CTAP, such as credential metadata, signature counters, or PIN configuration, must not be observable or modifiable through other device interfaces. Likewise, non-discoverable credentials created via CTAP must remain valid only when accessed through FIDO transports, preventing unintended leakage or misuse [46].

FIDO2 supports several transports, including USB HID, NFC, BLE, Lightning (Apple's tunneled HID implementation), and hybrid channels, each with strict binding rules. For example, NFC authenticators require explicit applet selection before CTAP commands are processed and must not automatically activate their FIDO applet on power-up. USB authenticators rely on the `FIDO_USAGE_PAGE` and `FIDO_USAGE_CTAPHID` descriptors, while BLE authenticators expose a dedicated FIDO GATT service. NFC authenticators must deselect their FIDO applet after use; USB HID authenticators typically rely on timeout-based session termination to close CTAP channels.

After enumeration, the browser initiates the CTAP handshake. It broadcasts a `INIT` command to all detected FIDO-compatible devices. Each authenticator replies with an initialization packet containing a fresh nonce echo and a newly allocated Channel Identifier (CID).

CTAP messages exchanged over transports like USB HID are serialized using CBOR (Concise Binary Object Representation ¹⁴) and transmitted in fixed-size packets (typically 64 bytes for HID reports). Each CTAPHID message begins with an initialization frame that contains the total expected message length, allowing the client to reconstruct larger messages that exceed a single packet. Subsequent continuation packets follow until the entire payload is received and reassembled.

¹⁴CBOR is a binary data serialization format defined in RFC 7049, designed to be more compact than JSON while remaining self-describing.

After initialization, the browser/client probes all connected authenticators by sending an `authenticatorGetInfo` command (CTAP command 0x04). This request retrieves metadata such as supported CTAP versions, available extensions, credential storage capabilities, and PIN configuration. Importantly, receiving this probe does not mean the authenticator has been selected. It is merely part of the discovery phase. Only the authenticator ultimately chosen will receive the actual command, either `authenticatorMakeCredential` (0x01) for registration or `authenticatorGetAssertion` (0x02) for authentication.

In case of multiple authenticators being connected, the client behaves according to the metadata announced earlier in the `authenticatorGetInfo` command, more specifically the FIDO2 supported versions and the capabilities. Supposing that all the connected authenticators support the PIN authentication, but do not the built-in (like biometrics), they will receive a `authenticatorMakeCredential` request from the Client containing dummy metadata (the relying party is often set to “.dummy” in chromium browsers and “make.me.blink” for firefox), forcing the authenticators to responde with an error status.

For authenticators supporting FIDO2.1 and FIDO2.2 version this request is excluded, but as in the FIDO2.0 version, the authenticator selection is purely bounded with the user presence/user verification. So the authenticator that responds first proceeds with the communication, allowing the user to implicitly choose which authenticator is intended to use for registration or authentication.

In our work, we can leverage this faster condition to always gain the device selection with our emulated software FIDO device.

Normally communication between the client and authenticator uses hexadecimal commands to specify operations [46]. Two commands form the core of CTAP: `authenticatorMakeCredential` (0x01) and `authenticatorGetAssertion` (0x02). `MakeCredential` handles registration. It instructs the authenticator to create a new key pair and save the credential. `GetAssertion` handles authentication. It asks the authenticator to sign a challenge, proving it owns a registered credential. If a service has multiple credentials stored, `authenticatorGetNextAssertion` (0x08) retrieves the rest.

The protocol provides commands for device management too. Clients use `authenticatorGetInfo` (0x04) to discover what an authenticator can do. PIN management happens via `authenticatorClientPIN` (0x06), covering everything from initial setup to token retrieval. Biometric devices use `authenticatorBioEnrollment` (0x09) for fingerprint registration. The `authenticatorCredentialManagement` (0x0A) command lets users browse and remove stored credentials. Other commands include device reset (0x07), user presence checks (0x0B), blob storage (0x0C), and configuration adjustments (0x0D) [46].

2.3.2.2 Authenticator Metadata

One of the most important messages in CTAP, and arguably the most critical from a security perspective, is the `authenticatorGetInfo` request sent from the client to the authenticator. This message prompts the authenticator to reveal key information about itself. While necessary for interoperability, it also exposes details that could help an attacker evaluate whether the device is weak or vulnerable to specific attacks.

Down bellow is shown a captured message of a response that a Feitian Technologies device sent to the client.

```
{
  1:  ["U2F_V2", "FIDO_2_0", "FIDO_2_1_PRE"],
  2:  ["credProtect", "hmac-secret"],
  3:  aaguid,
  4:  {
    "rk": True,
    "up": True,
    "plat": False,
    "clientPin": True,
    "credentialMgmtPreview": False
  },
  5:  1024,
  6:  [1],
  7:  6,
  8:  96,
  9:  ["ble", "nfc", "usb"],
  10: [{ "alg": -7, "type": "public-key" }]
}
```

In this response we can see several pieces of information, such as the supported CTAP versions, available extensions, and the AAGUID that identifies the authenticator model. However, the most security-relevant details appear in the fourth field (0x04), the options block. From the values shown in this structure, we can infer several important characteristics about the device:

- It stores sensitive keys internally (rk = True), meaning it supports resident keys.
- It requires user interaction (up = True), ensuring that operations cannot be performed silently.
- It behaves as a roaming authenticator, not as a platform authenticator like Windows Hello (plat = False).
- It is capable of PIN-based user verification, and a PIN is configured (clientPin = True).
- It exposes credential-management capabilities, which may allow enumeration or manipulation of stored credentials (credentialMgmtPreview = False indicates the preview version is not used, but the feature exists).

More importantly, we can also see what is missing. Certain flags that strengthen security are not present. For example:

- The uv flag, which indicates support for built-in user verification (such as a fingerprint sensor), is also missing. This tells us the device relies on external PIN verification instead of on-device biometrics.

- The `alwaysUv` flag, which indicates that user verification (PIN or biometrics) is always required, does not appear. Since it is neither set to `True` nor explicitly set to `False`, the authenticator does not enforce mandatory user verification (big issue allowing exploits with malicious clients or MiTM attacks).
- The enterprise attestation flag (`ep = True`), which some enterprise-focused authenticators support, is not included. Its absence suggests the device does not implement enterprise attestation.
- The `setMinPINLength` capability is also absent. Authenticators that support this feature can process the `setMinPINLength` subcommand, allowing the relying party (through the client) to enforce stronger PIN policies increasing the minimum PIN length. Because this flag is missing, the device cannot enforce stronger PIN requirements even if the ecosystem requests them.

2.3.3 CTAP Security Analysis

Considering these specifications, we can proceed to another relevant and critical message that clients commonly use during registration or authentication: `authenticatorClientPIN`. Although PIN usage is not strictly required within many authenticators, most clients enforce it to strengthen security, of course, depending on each authenticator metadata. When a client sends an `authenticatorClientPIN` request, it includes the CBOR-encoded `pinUvAuthProtocol` (typically 1 or 2) along with a subcommand that specifies the desired operation. These subcommands cover a range of functionalities, including retrieving the number of remaining PIN retries, performing key agreement for the Diffie-Hellman handshake, setting a new PIN, changing an existing PIN and obtaining the PIN token (which acts as an authentication cookie for subsequent protected operations), among others that were not observed during analysis of Chromium's and Firefox behavior.

When subcommand 2 is received, the authenticator initiates a key agreement process by generating a new Diffie-Hellman private and public key pair and returning the public key to the client. The client, upon sending this command, also generates its own Diffie-Hellman key pair. Both parties can now independently derive the shared secret using their respective private keys and the other party's public key (assuming no man-in-the-middle attack is present).

The next step is subcommand 5, which corresponds to a `getPINToken` call. In this request, the CBOR message includes the client's Diffie-Hellman public key and an encrypted blob containing the hash of the user's PIN, encrypted with the shared secret. Using this information, the authenticator derives the same shared secret, decrypts the blob, and compares the PIN hash with the previously stored one. If verification succeeds, the authenticator generates a random 32-byte value (the PIN token) and stores it internally. To complete the operation, the authenticator encrypts the PIN token with the shared secret and sends it to the client, who can then decrypt and retrieve it as demonstrated in Figure 2.1.

With these steps completed, the browser can proceed to the actual operation of creating a credential (registration) or verifying an existing one. Typically, browsers obtain a fresh PIN token each time the user initiates either of these operations.

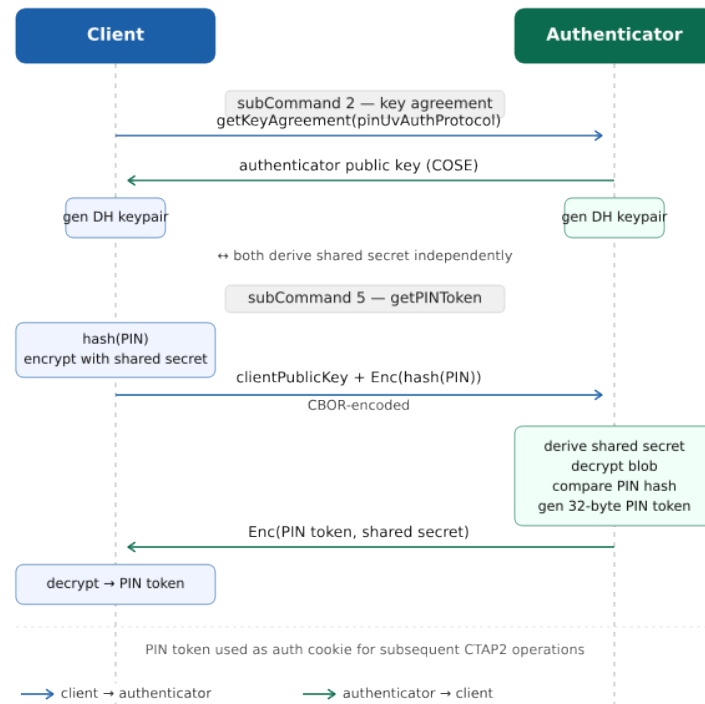


Figure 2.1: PIN authentication

One of core problem resides in the authenticator side, where are considered vulnerable devices/authenticators that do not require user verification at each message sent from the client, such as PIN verification or biometric authentication. The most critical calls are the Reset and GetAssertion (authenticate) operations.

The reset operations can be catastrophic for end-users who rely on their authenticator as their sole authentication factor, potentially resulting in complete loss of access to their accounts. The transport layer plays a critical role in the severity of this vulnerability. The paper [11] demonstrated a serious exploitation scenario when using NFC as the transport mechanism. While the specification includes a safeguard requiring that reset can only be invoked within the first 10 seconds after powering up the authenticator, this protection proves insufficient for NFC devices.

In NFC scenarios, each tap effectively constitutes a new power-up event, making the 10-second window continuously available despite the user presence requirement.

To mitigate this risk, many modern authenticators now block the reset command entirely when accessed via NFC. However, attackers can still exploit this vulnerability easily if they gain physical access to the authenticator through other transports, such as USB, without requiring a man-in-the-middle attack.

Also, the GetAssertion where an attacker can obtain a valid challenge signature without requiring any user verification.

When this permission weakness is combined with a man-in-the-middle attack, the situation can escalate to a full compromise. A legitimate user may unknowingly interact with their authenticator while the attacker intercepts and forwards the authentication request. In such a scenario, the attacker can obtain a valid assertion and gain access to the user's account.

To conclude, the most critical weakness in the CTAP protocol concerns the possibility of man-in-the-middle (MitM) attacks, which, if successfully exploited, can allow an attacker to bypass client-side authentication checks. The root of this issue lies in the Diffie–Hellman key exchange used during PIN-based user verification. Although this problem has already been discussed in several works [6] [10], and some proposals explore stronger alternatives such as sPACA or the use of Trusted Execution Environments (TEEs), many authenticators from major vendors like Yubico and Feitian still rely on the vulnerable construction.

As a result, a large portion of FIDO2 hardware in active use today remains susceptible to such attacks.

Fortunately, FIDO2 with CTAP-based PIN verification is not yet the primary authentication method for most users. Instead, it is commonly used as a second factor (U2F), which mitigates the practical impact to some extent. Nevertheless, the vulnerability remains relevant, especially as FIDO2 adoption grows and more services begin to rely on it for passwordless authentication.

Considering the MitM possibility there are several attacks possible such as Session Hijacking, Bypass Client side Verification, Credentials leakage and the Rebinding Attack [9].

2.3.4 Attestation and Certificates

An important functionality that was not referred is the Attestation, imagine that you possess a FIDO2 authenticator and are willing to register in a given website, how does the website know you are using a real, certificated security key and not a possible compromised malicious software pretending to be a legitimate one?

The attestation process is basically the verification of a digital certificate chain that proves the authenticator's identity and authenticity. The certificate contains information about the token's manufacturer, his model and a cryptographic proof that the device is genuine. In the registration process, if the relying party requires attestation, the authenticator signs the new public key along with other registration data with the attestation private key (stored internally in the device during manufacturing). This signature with the attestation certificate are the only requirements to achieve attestation. When the client receives it, it just needs to forward to the relying party as part of the registration response. The verification process happens in the relying party side by checking the signature against the certificate and also validate whether the chain includes a known trusted root CA, typically corresponding to the authenticator manufacturer or as an entity listed in the FIDO Metadata Service (MDS) ¹⁵.

This technique favors the relying party, not the end user (authenticator owner). The way that organizations leverage a security boost is by having the possibility of enforcing security policies based on the authenticator characteristics¹⁶. For example, allow only authenticators present in an accepted list (containing like Yubico or Feitian), rejecting software-based or uncertified devices or imposing specific standards to the device usage.

Although the aforementioned advantages, this comes with a privacy trade-off: in a scenario where a user registers in multiple services using the same attestation certificate, those services

¹⁵W3C: Web Authentication - An API for accessing Public Key Credentials Level 2, Section 6.5: Attestation. <https://www.w3.org/TR/webauthn-2/#sectn-attestation>

¹⁶FIDO Alliance: FIDO Metadata Service. <https://fidoalliance.org/metadata/>

can correlate the registrations to the same authenticator batch, reducing the user's anonymity set [6].

Given a security perspective, we can ensure that credentials were generated by authenticators defined on security policy accepted lists and that it complies with certain security standards (ex: secure key storage, resistance to physical attacks). It is important to note that this technique is only leveraged in the registration workflow. In the authentication no such guarantee is provided, as the authenticator's trustworthiness is assumed to have been established at registration time.

In the context of this work, we introduced earlier the SGX attestation key. The logic is to provide the relying party the attestation data generated inside genuine Intel SGX enclave running verified code, rather than in potentially compromised software or untrusted hardware.¹⁷ So, as traditional FIDO2 authenticators prove that they are certified, the SGX-based attestation proves that the code that generated the registration data was executed in a secure enclave with hardware-enforced isolation. [5]

2.3.5 Credential Storage Models

There are two different ways authenticators manage to use their credentials in the FIDO2 flow. Each one provides different trade-offs when referring to storage requirements, privacy, and usability.

The less complex and most seen is the resident key model (indicated by: `rk=True` in the authenticator capabilities). This refers to when the authenticator stores the complete credential, including the private key, relying party identifier, and user information directly in its internal storage. This model enables what FIDO2 calls "discoverable credentials". So for example, when a user visits a website does not even need to insert his username, the authenticator searches its internal storage for credentials that match the relying party's ID. Therefore, storage is limited ranging from 25 to 100 credentials depending on the device¹⁸. Once the limit is reached it is necessary to delete older ones to continue registering. This is problematic if users access hundreds of services.

In order to address this scalability issue, FIDO2 also supports non-resident credentials. In this model the authenticator, instead of storing each credential, generates a credential and encrypts it using a device-specific wrapping key. In registry, this blob is sent to the relying party which stores it associated with user data. During the authentication process it is necessary that the relying party sends back that same blob along with the authentication request. With the wrapping key, the authenticator can retrieve the respective private key and use it to sign the received challenge and create the respective authentication response [46].

This way, the authenticator does not require any storage as everything resides externally in an encrypted form. Comparing both alternatives this one sacrifices user experience. Since servers have to store the blob associated with user data, the user has to provide some identification to fetch to the authenticator the specific blob that it is able to decipher.

¹⁷Intel: Intel SGX Remote Attestation. <https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/attestation-services.html>

¹⁸YubiKey 5 Series with older firmware (pre-5.7) supports approximately 25 resident credentials, while newer firmware versions (5.7+) support up to 100. See: Yubico Technical Manual: <https://docs.yubico.com/hardware/yubikey/yk-tech-manual/yk5-apps.html>

Of course, the security around non-resident credentials lays around the wrapping key. If it is compromised, all credentials created by that device are vulnerable [10].

2.3.6 Cryptographic Operations

FIDO2 authentication relies on multiple cryptographic operations throughout the credential lifecycle. Table 2.1 provides an overview of the fundamental cryptographic operations used in FIDO2, their purposes, and the algorithms involved.

Operation	Algorithm	Purpose & Description	Reference
Key Generation (MakeCredential)	ES256 (ECDSA P-256), EdDSA (Ed25519), RS256, PS256	Generate fresh asymmetric key pair during registration. Private key stays in authenticator, public key sent to relying party. ES256 is mandatory.	[47]
Challenge Signing (GetAssertion)	Same as key generation algorithm	Sign authentication challenge + RP ID hash + flags + counter. Proves possession of private key without revealing it.	[47]
Diffie-Hellman Key Exchange (PIN verification)	ECDH with P-256	Establish shared secret between client and authenticator. Used to encrypt PIN hash and PIN tokens during authenticatorClientPIN. Vulnerable to MitM attacks.	[46]
PIN Encryption	AES-256-CBC	Encrypt PIN hash and PIN tokens using shared secret from DH exchange. Protects PIN during transmission.	[46]
Attestation Signing (Registration)	Attestation key algorithm (usually ECDSA P-256 or RSA)	Sign the new credential's public key + registration data using device's attestation private key. Creates certificate chain to manufacturer.	[47]
Hash Functions	SHA-256 (primary)	Hash RP ID, compute report data, derive shared secrets. SHA-384/512 for higher security algorithms.	[47]
Signature Counter	Monotonic counter	Increments with each assertion. Used for clone detection. Counter included in signed data.	[47]
Credential ID Encryption (Non-resident keys)	AES with device wrapping key	Encrypt private key + metadata into credential ID blob. Allows unlimited credentials without storage.	[46]

Table 2.1: Cryptographic Operations in FIDO2

FIDO2 supports multiple signature algorithms identified by COSE (CBOR Object Signing and Encryption) identifiers:¹⁹

- **ES256 (COSE alg: -7):** ECDSA with P-256 curve and SHA-256. Mandatory for all FIDO2 authenticators. Produces 64-byte signatures.
- **EdDSA (COSE alg: -8):** Edwards-curve DSA (typically Ed25519). Deterministic signatures, resistant to side-channel attacks.
- **RS256 (COSE alg: -257):** RSA-based signatures with SHA-256. Larger signatures (256 bytes), legacy compatibility.

¹⁹IANA: COSE Algorithms Registry. <https://www.iana.org/assignments/cose/cose.xhtml#algorithms>

- **PS256 (COSE alg: -37):** RSA with probabilistic padding. More secure than RS256.

2.4 Credential Management Systems

Considering the FIDO2 overview complete, it is time to analyse credential management systems that may be relevant in order to achieve a better solution when developing the FIDO2 and SGX credential system.

2.4.1 Password Managers

Password managers emerged as a practical way of managing the ever-increasing number of online accounts people need to manage. Instead of remembering dozens of passwords or reusing the same password everywhere, people store their complete credentials in one encrypted vault protected by a single master password.

Local password managers were the first generation of these tools. There are applications such as KeePass which keep encrypted password databases directly located on the user's device. This is a good way to gain strong security because the credentials never leave the machine, but also has severe usability issues. Unless you manually copy the database file around, users cannot access their passwords from other devices. Syncing the database across different devices becomes an inconvenience, sometimes needing USB drives, cloud storage services, and manual processes. Without backups in place, all the passwords can get lost when the database file is corrupted or lost. These restrictions prevented most users who use computers with their mobile devices or tablets, all working day long, from using the local password manager.

Another type of managers are the Cloud-based password managers that solved the syncing problem since they store encrypted vaults on remote servers.

Managers like LastPass, 1Password, Bitwarden automatically sync passwords with all the user devices. For example, you can save a credential for your mobile device, and it will pop up on your laptop seconds later. This convenience ensured mass adoption but created new trust needs.

To better explain how these work, the majority of them profess to be under zero-knowledge architecture, which means they have no access to user passwords. Encryption and decryption only occur locally on the user's device, and only encrypted data is delivered to the servers. The master password never leaves the device. But users have to trust that the company actually does it the right way, and does not maintain its own backdoors. This trust has been tested by several high-profile breaches. LastPass was hit with a big breach in 2022, where the attackers accessed encrypted vault data, exposing customers with weak master passwords. Even having the encryption intact, the attack underscored how capturing everyone's credentials in one repository would lure a hacker.

In addition to breach fears, cloud password managers require an assumption of trust in the service provider's infrastructure, encryption implementation, and business practices. But if the company shuts down or gets acquired and the model changes for its security, then users all have to face tough migration choices with more than their credentials at stake.

We also consider managers integrated with browsers that are the middle ground between local and cloud solutions. Many browsers have built-in password managers that sync passwords through their account systems in modern browsers such as Chrome, Firefox, and Safari. These managers work together seamlessly with the browsing experience—they autofill credentials, auto-generate strong passwords, and notify users of reused or compromised passwords.

2.4.2 Passkeys and Synced Authenticators

In the context of this thesis, it is important to review the credential managers that are related to FIDO2 credentials, particularly how passkeys are implemented and managed by different platform providers and third-party services.

As previously mentioned, passkeys are FIDO2 credentials that sync across devices within the same ecosystem. However, the implementation details and security models, though, also do not look the same in the market among all the providers.

The major platform providers have created their own passkey architecture. Apple’s implementation utilizes iCloud Keychain to sync passkeys with all devices signed into the same Apple ID—iPhones, iPads, and Macs share the same credentials seamlessly. Google Password Manager also provides similar functionality on Android devices and Chrome browsers signed into a Google account. Microsoft introduced Windows Hello support for passkey, synced to Microsoft accounts.

The security model of these implementations centers on end-to-end encryption. Platform providers claim that encryption and decryption of passkey material occurs only on user devices, not on the provider’s servers. The cloud service stores encrypted blobs that it theoretically cannot decrypt. Each provider uses different key derivation mechanisms tied to user authentication factors—typically a combination of the account password, device-specific secrets, and in some cases hardware security modules.

However, this architecture introduces trust assumptions beyond those required for device-bound credentials. The account recovery mechanisms also present security considerations. The platform provider must be trusted by users to correctly implement the end-to-end encryption, maintain no backdoors, and have robust key derivation procedures. The account recovery mechanisms also bring security considerations; if a user can regain access to synced passkeys after losing all devices, an attacker who compromises the account recovery process could potentially gain that access. This is a fundamental tension: more flexible recovery methods have better usability, but security guarantees may be weakened.

Third-party password managers like Bitwarden²⁰ added passkey support. Storing them in their encrypted vaults combined with regular passwords.

Here the trust model differs slightly—users need to trust both the password manager’s encryption implementation and their cloud infrastructure; they acquire the benefit of a single credential store across all platforms, not just a single ecosystem.

The syncing of cryptographic credentials also widens the attack surface. Unlike device-specific access credentials, a hardware security key includes credentials on one device, but

²⁰<https://bitwarden.com/blog/security-vendors-join-forces-to-make-passkeys-more-portable-for-everyone/>

synced passkeys are present on every device the user signs into, along with encrypted copies in the cloud. A vulnerability in the sync function and the stored cloud could be compromised as part of the credentials.

The FIDO Alliance formalized synced passkeys with the multi-device FIDO credentials specification, which provides guidelines for secure synchronization while sustaining basic security characteristics of FIDO2. Synced credentials should also be phishing resistant, have attestation capabilities, and safeguard against credential stuffing attacks.

For the purpose of this thesis, it is important to understand the mechanism and its limitations of the passkey manager currently in place. Existing solutions depend above all on software-based encryption and trust in cloud providers; alternative solutions using trusted execution environments may provide different security guarantees, which will be addressed in the related work section.

2.4.3 Enterprise Credential Management

As consumer-focused credential managers concentrate on convenience and cross-device synchronization, enterprise credential management systems tend to emphasize centralized control, compliance, auditability, and connection to existing systems (infrastructure).

The first and foremost, infrastructure to administrate user identities, authentication methods and access policies over a whole organization is IAM (Enterprise Identity and Access Management). Current IAM programs can use existing IAM systems for the management of FIDO2 authentication but with the support adaptations as organizations transition to passwordless and phishing-resistant authentication methods.

Microsoft Entra ID (formerly Azure Active Directory) allows both device-bound passkeys securely stored on FIDO2 security keys and synced via Microsoft Authenticator²¹. Organizations can also configure identity policies for authentication resistant to phishing attacks, need attestation for passkeys bound to the device, and differences between security types across different user groups²². Microsoft launched FIDO2 provisioning APIs, which allow organizations to pre-configure security keys on behalf of their users, counter-acting this headache of initial enrollment and onboarding²³.

Another types of systems is the Privileged Access Management (PAM). Platforms like CyberArk and BeyondTrust aim to controll access to critical systems using privileged accounts. Those implement password vaults backed by Hardware Security Modules (HSMs) and include approval workflows with audit trails^{24 25}.

Unfortunately, traditional PAM solutions were designed around password-based authentication and are still evolving to support FIDO2 authentication models.

²¹Microsoft Entra ID documentation on passkeys (FIDO2): <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-authentication-passkeys-fido2>

²²Microsoft Learn: Enable passkeys for your organization. <https://learn.microsoft.com/en-us/entra/identity/authentication/how-to-enable-passkey-fido2>

²³Microsoft Community Hub: Public preview: Microsoft Entra ID FIDO2 provisioning APIs, 2024. <https://techcommunity.microsoft.com/blog/microsoft-entra-blog/public-preview-microsoft-entra-id-fido2-provisioning-apis/4062699>

²⁴CyberArk: Privileged Access Management (PAM). <https://www.cyberark.com/products/privileged-access-manager/>

²⁵<https://www.beyondtrust.com/resources/glossary/privileged-access-management-pam>

Also it is important to refer the Enterprise Public Key Infrastructure (PKI) systems that offer certificate-based authentication. Here certificate lifecycle management platforms automate issuance, renewal and revocation of digital certificates, with HSMs meeting FIPS 140-2 standards protecting certificate authority private keys ²⁶. While some organizations deploy certificates on FIDO2 security keys as an alternative authentication method, direct integration of FIDO2 credential lifecycle management into PKI platforms remains limited.

One of the biggest challenges in the enterprise landscape is the fragmentation of credential management. Organizations generally use separate systems for passwords (PAM or password managers), certificates (PKI platforms), and FIDO2 credentials (relying on IAM provider capabilities). Such fragmentation results in operational complexity, inconsistent enforcement of policies across different types of credentials, and challenges in compliance reporting. Solutions like Versasec vSEC:CMS address this by combining FIDO2, PKI, and traditional credentials, enabling unified management for a centralized view ²⁷, but adoption is still limited.

2.4.4 Credential Portability

First of all, what is credential portability? It is the ability to transfer authentication credentials between different platforms, providers, and systems, and it has become a significant issue of concern in relation to passwordless authentication

This problem manifests differently when considering consumer and enterprise contexts. Password users should be able to change password managers using the CSV export/import of passwords. But there is one big problem with FIDO2 credentials: the private keys are made to be non-exportable, to prevent theft. This security feature mitigates credential theft, but at the cost of re-registering all of one's passkeys with every service one uses when one moves from the Apple ecosystem to Android, or from 1Password to Bitwarden, a huge usability problem.

In an enterprise context, the challenge to portability is even greater. Due to acquisitions, vendor performance issues, or strategic infrastructure choices, organizations may have to switch the IAM provider. Without credential portability, such migration requires re-enrollment of thousands of people with new FIDO2 credentials, an operationally costly process that opens the door to security windows during transition periods, leading to temporary use of less secure fallback authentication methods. Multi-vendor scenarios, prevalent in large companies, are confronted with comparable problems in building cohesive authentication experiences across IAM systems.

The Credential Exchange Protocol (CXP) published by the FIDO Alliance in December 2023 is the first industry standard to attempt to overcome this challenge ²⁸. CXP provides a common protocol and format for securely exporting and importing passkeys between different credential providers. The key features of the protocol are that it encrypts the credentials transmitted throughout the transit path so its use-case is secure, and there is the need for authentication on both systems: source and destination. 1Password, Bitwarden, Dashlane, NordPass, and

²⁶Futurex: Public key infrastructure (PKI) & Certificate Authority. <https://www.futurex.com/solutions/pki-certificate-authority>

²⁷Versasec: FIDO2 device-bound passkeys with Microsoft Entra ID, 2024. <https://versasec.com/blog/fido2-device-bound-passkeys-with-microsoft-entra-id/>

²⁸FIDO Alliance: Credential Exchange Protocol specification, 2023. <https://fidoalliance.org/specs/fido-cxp-v1.0-20231211/>

other key suppliers of password management worked on the development of this protocol and committed to their implementation ²⁹.

However, there are some problems related to CXP. Apple, Google, and Microsoft are just some of the platforms that have not publicly pledged to offer credential export from their own native password managers, likely as part of ecosystem lock-in plans. The protocol relies on both source and destination systems to implement compatible versions, creating a coordination problem. Another challenge is that attestation and credential metadata should be standardized throughout the transfer; some enterprise policies have established specific authenticator types and attestation levels that need to be preserved during the migration of credentials, and thus, this creates complexity.

There are alternative strategies for portability with trade-offs. Some companies enforce multiple registered user credentials, one for each system but with more management overhead. There are also issues with the consistency, revocation propagation, and authoritative sources of truth, the most obvious of which are those associated with the creation of any credential synchronization protocols (which store credentials simultaneously in multiple IAM systems). Hardware cryptographic solutions for multiple-protocol based cryptographic keys (FIDO2, PIV³⁰, OpenPGP) have some portability to switch over to different cryptographic protocols in different authentication systems but they do not solve the cloud-based passkey migration issue.

The tension between security and portability remains fundamental to credential management system designs. Highly portable credentials that can be exported without restriction to use them have a higher risk of unauthorized duplication or theft. Completely non-portable credentials that are not designed for export offer stronger protection in terms of security requirements although vendor lock-in and rigidity in running them is another problem. For this thesis purpose, understanding these portability challenges influences the design of an SGX-based credential management system.

Some literature addressing this issue is [48], in which credential migration between TEEs using mutual attestation protocols is investigated. Shepherd et al. suggest a suite of protocols for managing heterogeneous TEE credentials through migration, revocation, backups, and updates, with bi-directional trust guarantees. It uses a Trusted Service Manager (TSM) that matches GlobalPlatform TEE specifications and expects mutual remote attestation for both source and destination systems in a secure manner during credential transmission between them. This work demonstrates that credential portability challenges are not limited to consumer password managers and FIDO2 implementations, but also affect enterprise TEE deployments, where the same tension between security and portability persists. It is also referred that the protocols are not limited by a specific type of TEE. Although, our implementations will not be ruled the same way as the referred in the research, we will leverage the remote attestation functionality as well, so understanding this mechanisms will help on the design of the SGX-based credential management system.

²⁹FIDO Alliance: FIDO Alliance publishes new specifications to promote user choice and enhanced UX, 2023. <https://fidoalliance.org/fido-alliance-publishes-new-specifications-to-promote-user-choice-and-enhanced-ux/>

³⁰Personal Identity Verification (PIV) is a US federal standard (NIST SP 800-73) defining a smart card-based framework for authentication, digital signing, and encryption.

2.5 Related Work

2.5.1 Linux UHID for Device Emulation

In order to build a FIDO2 device, we utilized the Linux `/dev/uhid` character device interface in this work. UHID (User-space HID I/O driver) is a kernel subsystem that allows userspace applications to implement the HID transport drivers without introducing the necessity of kernel modules.

The UHID interface³¹ works via bidirectional communication between userspace and the kernel HID subsystem. The authenticator emulator opens `/dev/uhid` and communicates with the kernel HID subsystem through a message-passing protocol. The key messages are `UHID_CREATE2` for registering a new HID device to the kernel and sending the HID report descriptor accompanied with device metadata, `UHID_INPUT2` to send reports showing HID values from the virtual device to the kernel, and `UHID_DESTROY` to take out the device. The kernel responds with the following `UHID_START` events: recognising the device register for `UHID_OPEN`, the file opening of a HID device by application and opens `UHID_OUTPUT` which returns output documents like `UHID_GET_REPORT` and `UHID_SET_REPORT`, which accepts synchronous control channel requests.

For FIDO2 authenticator emulation, UHID is specifically suitable as it offers direct implementation of the CTAP HID transport protocol entirely in userspace. The CTAP HID protocol includes a USB HID interface, unique report descriptors and positioning for CTAP messages. UHID also makes it possible for a virtual FIDO2 authenticator to take over an operating system's access, posing as a physical USB security key for CTAP requests from developers and users via the standard `/dev/hidraw` interface. It allows the SGX enclave to support FIDO2 operations, while keeping it in tune with other WebAuthn implementations which expect standard CTAPHID transport

The implementation requires careful handling of the HID report descriptor to match FIDO2 specifications, proper parsing and assembly of CTAP frames (which can span multiple HID reports), and state management for channel allocation and command atomicity as defined in the CTAP protocol.

Despite its practical relevance, UHID-based FIDO2 device emulation has received little attention in the academic literature, with existing work being limited to open-source projects. The closest related work is `fido2-hid-bridge`³², which uses the Linux UHID facility to create a virtual HID device that bridges FIDO2 CTAP2.1 commands from USB-HID-only clients (such as Firefox and Chrome) to PC/SC-connected authenticators such as NFC tokens or smartcards. Similarly, `fido2ble`³³ bridges FIDO2 BLE devices to the HID bus via `/dev/uhid`, enabling their use in browsers that only support USB HID. More recently, `tpm-fido2`³⁴ employs `/dev/uhid` to expose a virtual FIDO2 HID device backed by the host system's TPM 2.0 and fingerprint reader. In a similar vein, `passless`³⁵ implements a software-only FIDO2 authenticator in Rust

³¹<https://docs.kernel.org/hid/uhid.html>

³²<https://github.com/BryanJacobs/fido2-hid-bridge>

³³<https://github.com/PoneBiometrics/fido2ble>

³⁴<https://github.com/mc256/tpm-fido2-thinkpad-linux>

³⁵<https://github.com/pando85/passless>

that emulates a CTAP2 HID device via UHID, storing resident credentials on the local filesystem with passkey support and user verification through desktop notifications. To the best of our knowledge, no prior work has employed UHID for a fully virtualized SGX-backed FIDO2 authenticator, making this a novel application of the interface.

2.5.2 SGX-based Credential/Password Managers

Many password managers in the marketplace have incorporated Intel SGX to improve security. Dashlane³⁶ announced SGX integration to protect master passwords and encryption keys within enclaves in order to prevent exposure even if the OS itself has been compromised. 1Password³⁷ likewise explored SGX to protect sensitive cryptographic tasks and the decryption of credentials. These implementations show the industry recognition of the TEE benefits for credential protection but remain proprietary with limited technical details published.

While some of the proposed solutions focus on secure password derivation functions and key material within enclaves [49], others provide a solution to secure I/O and side-channel attacks [50]. However, previous work has not specifically focused on the various difficulties posed by managing FIDO2 protocol specifications with SGX's limitations, in particular virtual HID device emulation, attestable credential management and credential migration.

2.5.3 FIDO2 with TEEs

Through all types of authenticators, logically, the platform-based implementations of FIDO2 with TEEs were the ones that received the most attention and adoption within enterprises. Mobile platforms use ARM TrustZone and use it to securely store private keys in the trusted Execution Environment (TEE) and perform cryptographic operations in isolation from the rich OS. Also, Microsoft Windows Hello utilizes TPM-backed (Trusted Platform Module) credential storage.

Previous academic research has explored the security characteristics of FIDO2 in the TEE context, as well as innovative approaches to credential management issues. FeIDo [49] provides a virtual FIDO2 token, which uses SGX enclaves to create a hybrid solution to the merging of FIDO2 security and interoperability with existing eID prevalence. FeIDo solves the credential recovery issue by generating FIDO2 credentials from constant, personally identifying attributes obtained from eID documents (e.g., name, date of birth, place of birth). Since such attributes are constant during document renewals, the resulting credentials will survive token loss or theft, overcoming a significant usability factor on FIDO2. Personal data and resulting FIDO2 credentials are processed inside attestable SGX enclaves. FeIDo also shows the ability to extract and disseminate verifiable anonymous credentials for privacy-preserving age verification and other attribute-based verification.

The way these researchers deployed their credential manager was by including a “Client middleware” component in the architecture, which handles interaction with eIDs and the cloud SGX enclave, and communicates with browsers through a custom web extension they implemented. In this approach, the CTAP protocol introduced earlier is not used; instead, it is

³⁶<https://www.dashlane.com/download/whitepaper-en.pdf>

³⁷<https://1password.com/blog/using-intels-sgx-to-keep-secrets-even-safer>

replaced by remote attestation, which arguably provides stronger security guarantees. However, this design choice introduces limitations. Since the system works only through browser-based clients, this is a major limitation since many applications that require authentication do not support browser extensions or operate outside the browser environment entirely.

Prior work has shown some alternative techniques for support for TEE-based FIDO2 authentication from platform-specific implementations to eID-based credential derivation. But our approach aims on a software authenticator that serves as proxy to the SGX enclave and maintain compatibility with the standard CTAP protocol allowing the operation across diverse application environments beyond web browsers. This gap motivates the exploration of alternative architectures that balance security, portability, and broad compatibility with existing FIDO2 infrastructure.

2.5.4 Remote Attestation in Practice

Remote attestation is important for building trust on TEE systems, enabling remote parties to confirm the status of code in a real, untainted enclave. Intel has two attestation mechanisms for SGX which are Intel Attestation Service (IAS) and Data Center Attestation Primitives (DCAP).

The IAS is an attestation service operated by Intel. In this mechanisms, an SGX enclave produces a quote, a data structure that contains measurements of the code and data of the SGX enclave, signed by a platform-specific key. This quote goes into the IAS and it confirms that the signature is valid and the platform is genuine and in accordance with Intel's attestation verification report. After that, the IAS returns an attestation verification report signed by Intel. The relying party can then authenticate Intel's signature and finally trust the enclave. This poses an internet requirement to communicate with the Intel server and reliability of Intel's infrastructure.

DCAP provides decentralized attestation without having to rely on Intel's backend for each check. Rather, critical certificates and collateral are supplied ahead of time, cached locally, or served on organization-managed workloads. DCAP uses the same underlying attestation primitives, but moving the verification logic off Intel's servers and onto a local attestation service. This has improved scalability, lower latency, offline capability, and no single point of failure as we saw in IAS case. However, it comes with certificate provisioning and management and a more difficult deployment.

In the context of credential management with a remote SGX enclave, mutual attestation is not possible as most expected clients do not include SGX in their systems. This creates a one-way trust model, where the client must verify the remote enclave's authenticity, but the enclave cannot cryptographically verify the client's execution environment.

This restricts the design of the credential management system. So while this remote enclave can prove that it is running genuine SGX code with attestation, it will still need to adopt other mechanisms to authenticate clients and protect against unauthorized access. Traditional authentication methods, such as passwords or biometric data, need to be employed at the client side. The enclave must be aware that client-side secrets can potentially be compromised by malware or OS-level attacks.

For credential migration scenarios, this asymmetry becomes particularly challenging. While

the destination enclave can be attested, the source of credentials (whether a legitimate client or a malicious actor) does not provide equivalent security guarantees. This necessitates a meticulously designed protocol design to guarantee that credential transfers are performed only under the right authorization. Some approaches can reduce these threats through challenge-response protocols or time-limited authorization tokens, however, none offer the secure assurance of mutual enclave attestation.

2.6 Comparative Analysis

This section presents a comparative analysis against existing credential management approaches: Hardware Keys, Platform Authenticators, Traditional Password Managers, Passkey Managers with Cross-Platform Synchronisation (CXP), and FeIDo [49]. The comparison focuses on properties relevant to first-factor authentication, namely security, usability, compatibility, trust, and performance, as presented in Table 2.2.

Table 2.2: Comparison of credential management approaches

Property	Hardware Keys	Platform Auth.	Traditional Password Mgrs	Passkey Mgrs (CXP)	FeIDo [49]	Proposed
Security Properties						
Credential Isolation	High (Sec. Elem.)	Med-High (TEE/TPM)	Low (Software)	Low (Software)	High (SGX)	High (SGX)
Phishing Resistance	✓	✓	×	✓	✓	✓
Protection from OS	✓	✓	×	×	✓	✓
Usability						
Portability	None	None	High	High	Medium	High
Multi-device Access	Physical transfer	×	✓ (sync)	✓ (export)	Limited (eID req.)	✓
Recovery Mechanism	None	Platform dependent	Cloud sync	Encrypted export	eID derivation	User-centric migration
Setup Complexity	Low	Low	Low	Medium	High	Medium
Compatibility						
CTAP Protocol	✓	✓	N/A	✓	×	✓
Works with Native Apps	✓	✓	✓	✓	×	✓
OS Independence	✓	×	✓	✓	Partial	Partial
Client Hardware Req.	✓ (token)	✓ (TPM/TEE)	×	×	Optional	Optional
Trust Model						
Trust Requirements	Token manufact.	Platform vendor	Service provider	Service prov. + recipient	SGX + Serv. prov. + eID issuer	SGX + Service prov.
Privacy	Medium (attestation)	Medium (OS telemetry)	Variable	Variable	High (anon.+SGX)	High
Performance						
Auth./Reg. Latency	Low	Low	Low	Low	Medium (enclave)	Medium (enclave)

2.6.1 Security and Trust

Related to security, hardware-backed solutions (hardware keys, platform authenticators, FeIDo, and our proposed approach) provide a stronger credential isolation and protection from OS compromise, while software-based solutions (traditional password and passkey managers) fail ensuring these types of guaranty. Within the SGX-based approaches, the proposed solution supports two credential models: derived credentials, where the private key is never stored but deterministically derived on demand from a hardware-bound identity key, offering the strongest isolation guaranty; and migrated credentials, which are imported from another SGX instance and stored within the enclave, maintaining hardware-backed protection while enabling cross-deployment portability.

Moving to the trust model, hardware keys trust only the token manufacturer (simplest trust model). Platform authenticators (Apple, Google, Microsoft) convert trust to platform's vendors, giving them much greater vendor lock-in and privacy issues, since they are now privy to all authentication activities. When it comes to software-based password management, users need the trust of the software, as security and privacy implications can be inconsistent between implementations and policies. FeIDo presents the most complex trust model because trust must be placed on Intel's SGX implementation, the software that hosts the enclave infrastructure, the enclave operations, and the eID issuer. Both FeIDo and the proposed approach benefit from a notable SGX property: the integrity of the enclave code can be cryptographically verified through remote attestation, meaning that any party can confirm that the software running inside the enclave has not been tampered with. This provides a verifiable basis for trust that is absent in software-only solutions. The proposed method mitigates FeIDo complexity by removing the dependency on the eID issuer but requiring trust in Intel SGX implementation (the service provider) and the authenticator emulator, which is responsible for collecting user attributes and mediating credential registration and authentication. The trust in the client browser, although not reflected in the Table 2.2, remains relevant across all solutions.

2.6.2 Usability and Portability

Hardware keys and platform authenticators, though highly secure, do not allow the portability of credentials. Traditional password managers and passkey managers with strong encryption compatibility and CXP support ensure high portability by cloud synchronization or encrypted export, but sacrifice hardware-backed security. FeIDo provides a middle ground via eID-based credential derivation to offer medium portability without cloud synchronization, but with an added concern of having to interact with an eID for each authentication and the high initial setup complexity. The proposed approach achieves high portability through user-initiated, attestation-verified migration protocols between SGX instances, allowing credentials to be securely exported and imported across different deployments on demand, without sacrificing hardware-backed security guaranties.

2.6.3 Compatibility

The compatibility analysis exposes a critical limitation in current TEE-based approaches. While hardware keys, platform authenticators, and passkey managers support a fully compliant CTAP protocol that works in tune with native applications, FeIDo rejects that standard and instead uses a browser extension. This architectural choice restricts FeIDo to web-based authentication scenarios, leaving out native applications, command-line tools, and OS-level authentication. Furthermore, both FeIDo and the proposed approach exhibit partial OS dependence due to SGX hardware requirements, and a heavier dependency in the proposed approach related with the UHID interface that is only supported in Linux systems, contrasting with the full OS independence of hardware keys and the other software-based solutions.

2.6.4 Privacy and Performance

Related to privacy, for hardware keys this is not a concern (a potential cross-site tracking through attestation certificates that declare the vendor is tackled by replicating the certification credentials in batches of devices) and platform authenticators may expose authentication patterns through OS-level telemetry and credential synchronization services (iCloud Keychain, Google Password Manager). Traditional password and passkey managers show variable privacy depending on service provider policies and cloud synchronization practices. FeIDo achieves the strongest through anonymous credential mechanisms that enable attribute verification without revealing identity. The proposed approach achieves high privacy, as the credential keys are protected within the SGX enclave and never exposed to the service layer. Regarding performance, hardware keys and software-based solutions have low latency, while both FeIDo and our approach incur a medium latency overhead due to the cost of network communication and enclave context switches and operation.

2.6.5 Key findings and Motivation

This comparative review reveals a missing link in alternative credential management methodologies: no approach could achieve an adequate combination between strong TEE-backed security and high portability while remaining fully compatible with CTAP protocols. Hardware keys provide great security and CTAP support without any credential portability. Although platform authenticators have TPM protection, they create full vendor lock-in with no portability. Passkey managers need to leverage CXP for portability, but operate entirely in software without hardware security guarantees. FeIDo is the closest to our research objectives, using SGX for credential protection and recovering via eID-based derivation although also fundamentally abandoning CTAP support in favor of browser addons, as with the complexity of eID usage and integration.

2.6.5.1 The Attestation Gap

Hardware authenticators support attestation mechanisms that allow relying parties to verify security characteristics, while software-based FIDO2 implementations cannot provide equivalent assurances. FeIDo demonstrates that SGX-based virtual authenticators can offer attestation

capabilities, but the abandonment of CTAP protocol creates a deployment barrier that limits practical adoption.

2.6.5.2 The Migration Gap

Overall, FeIDo remains the closest work to our solution, yet beyond the concerns already discussed, it presents a more fundamental limitation: credential migration is not addressed. This raises significant availability concerns, particularly at scale. Consider the worst case: a large number of users derive credentials tied to a specific SGX processor, and that processor fails or becomes unavailable. Since credentials are always derived from hardware-bound secret material, they cannot be recovered on different hardware, leaving all affected users locked out. This represents a critical bottleneck in any derivation-only approach.

Keeping this in mind, the proposed solution supports user-initiated migration, which means that credentials are not permanently tied to a single hardware instance. If a deployment becomes unavailable, either due to hardware failure or a denial-of-service attack, users can switch to a different SGX instance to which they have migrated their credentials before losing access. This decoupling between credentials and hardware is a deliberate design choice that FeIDo, by relying solely on derivation, cannot offer, but is crucial in order to scale the solution.

2.6.5.3 The Trust Asymmetry

Finally, the trust asymmetry in remote enclave architectures presents an unsolved challenge. As discussed earlier, credential management with remote SGX enclaves faces a limitation: while the enclave can attest to its security properties, typical client devices without SGX support cannot provide equivalent cryptographic assurances. This asymmetry complicates secure credential provisioning and migration protocols, as the enclave must rely on traditional authentication mechanisms (user attributes, passwords, cryptographic keys) to verify user authorization.

Proposed solution

In this chapter, we present the conceptual overview of our solution. This work tackles the several interconnected problems mentioned earlier on modern authentication systems. First, centralized credential managers, whether cloud-based or self-hosted, require users to trust that the service provider is secure and honest. By leveraging Intel SGX enclaves, this trust assumption is replaced by a verifiable guarantee: credentials are encrypted at rest and processed exclusively within a hardware-isolated execution environment, whose integrity can be remotely attested by any party.

Considering the impact of data breaches on credential managers, our storage model is designed to contain no identifiable material: stored data cannot be linked to FIDO2 private keys without the code that runs inside an SGX enclave. Additionally, we allow FIDO2 private keys to migrate among equivalent SGX enclaves to enable redundancy, while ensuring that the stored data is re-identified under fresh, locally-derived identifiers on each enclave.

This chapter is organized as follows. Section 3.1 describes the proposed architecture within a complete WebAuthn environment, presenting each component and its role. Section 3.2 establishes the threat model and trust assumptions. Section 3.3 describes how credentials are created and how they can be re-derived. Section 3.4 explains the authentication process. Section 3.5 details the credential migration flow. Finally, Section 3.6 summarizes the security properties of the design.

3.1 Proposed architecture

Figure 3.1 illustrates the proposed solution architecture, which consists of six main components: the WebAuthn-enabled RP, the Client (browser), the Authenticator Emulator, the SGX host service, the SGX Enclave and the SGX host storage, used for persisting credential records.

Of these, the WebAuthn-enabled RP and the client are standard WebAuthn entities; our contribution lies in the four components hosted across the User Host and the SGX Host, described below.

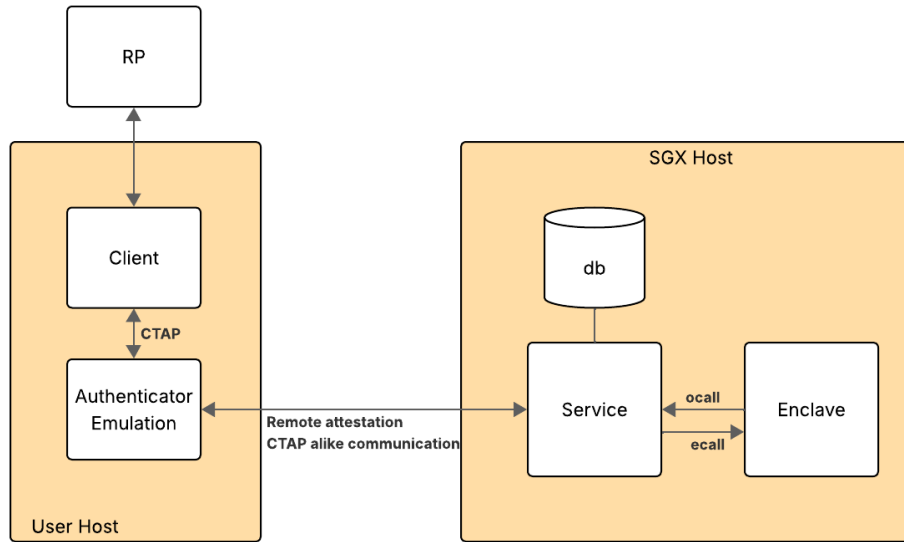


Figure 3.1: Architecture of the proposed SGX-based FIDO2 credential manager. The User Host (left) runs the client application — typically a browser — and the Authenticator Emulator, which speaks CTAP to the client. The SGX Host (right) exposes an HTTP service that forwards requests to the SGX enclave, the only trusted component beyond SGX hardware itself. Credential records are persisted in the host’s database, sealed by the enclave.

3.1.1 User Host

The User Host contains two crucial entities. The client is a browser or application that uses the FIDO2 protocol as an authentication mechanism. The Authenticator Emulator behaves as a hardware token, participating in the CTAP protocol communication described in Section 2.3.2.

3.1.1.1 Authentication Emulator

The Authenticator Emulator functions as a software Authenticator, exposing a CTAP interface to clients. It handles directly most interactions with clients, except those that require the collaboration of an SGX host: initialization, authenticator information retrieval, and most importantly, credential registration and assertion [46]. When the Emulator first contacts an SGX host — analogously to the CTAP-HID `INIT` step — it requests the enclave’s quote, validates it end-to-end (DCAP chain, expected `MRENCLAVE`, attested public key), and uses the resulting public key as the anchor for the per-request channel described in Section 3.4.

The PIN-related CTAP commands are therefore unnecessary in this design, as the authenticator declares the `uv` flag as `true` in the `authenticatorGetInfo` response, indicating that the authenticator is capable of built-in user verification. Although this typically implies biometric capabilities in conventional Authenticators, in our system it is performed through the user’s personal attributes, as described below.

For managing user credentials, the Authenticator Emulator gathers user-provided attributes (personal information, passphrase, etc.) and generates a user identifier (UID) through an SHA-256 hash of those attributes. Each UID aggregates a set of FIDO2 credentials belonging to a user. It is possible for a user to manage different UIDs (e.g., by providing a different passphrase for each set of FIDO2 credentials, something similar to having different hardware tokens for

different purposes).

3.1.2 SGX Host

The SGX Host is composed of three main components. The HTTP service is the entry point for communication with the Authenticator Emulator, exposing requests that mimic the CTAP protocol. It also manages the enclave’s lifecycle and mediates the calls that cross the trust boundary in both directions.

The enclave is the most important module in the system, as it is responsible for generating key pairs and validating that only the correct user is capable of making correct assertions, and implementing the migration mechanics. Beyond the user-dedicated Identity Key (IK) introduced in Section 3.3, the enclave also derives a stable enclave identity keypair from IK. Because IK is itself a deterministic function of the SGX sealing key and the enclave’s code measurement (M_{RENCLAVE}), this identity keypair is fixed for any given enclave codebase running on any given processor: it reconstructs to the same value on every launch, never appears outside the enclave in private form, and is the public anchor used by both the per-request communication channel (Sections 3.3 and 3.4) and the migration protocol (Section 3.5). The corresponding public key can be attested via an SGX quote, allowing any party to verify that a given public key belongs to the expected enclave on a genuine SGX platform.

Finally, the database is where the credentials are stored. All insert and lookup operations are mediated by the enclave, and the stored material is sealed under a secret key bound to the processor, so it can only be decrypted by an enclave running the same code on the same platform.

Of these three components, only the enclave is part of the trusted computing base; the HTTP service and the database operate in untrusted memory and are treated accordingly, as detailed in Section 3.2.

3.2 Threat model and trust assumptions

The security guarantees of our design depend on a precise specification of what is trusted and what is not. This section makes those assumptions explicit and identifies the adversaries the system is designed to resist.

3.2.1 Trusted components

The Trusted Computing Base (TCB) of the system is deliberately small. It comprises:

- **The SGX hardware** of the host machine, including its enforcement of enclave isolation, memory encryption, sealing, and attestation primitives.
- **The enclave code**, identified by its measurement (M_{RENCLAVE}). The integrity of this code is what remote attestation ultimately certifies, and any party interacting with the system can verify that they are communicating with the expected enclave version.
- **The Intel SGX attestation infrastructure**, transitively, since the validity of remote attestation depends on Intel’s provisioning certification chain.

Everything outside this set is treated as untrusted.

3.2.2 Untrusted components

We assume an adversary may fully control any of the following:

- **The operating system and hypervisor** of the SGX host, including privileged software with kernel-level access. SGX is designed precisely to defend against this class of adversary.
- **The HTTP service** that fronts the enclave. Although developed as part of this work, it runs outside the enclave and therefore must be assumed compromisable. The enclave does not rely on it for any security property: every input crossing the trust boundary is independently validated, and every persistent record is sealed before leaving the enclave.
- **The credential database.** An attacker with full read or write access to the database is part of our threat model. Confidentiality follows from sealing; integrity violations (e.g., record deletion or replay of stale records) are detectable but may cause availability loss, which we treat as out of scope for confidentiality claims.
- **The network** between the user host and the SGX host, and between peer SGX hosts during migration. We assume the underlying network may be fully controlled by an adversary capable of eavesdropping, modifying, replaying, and injecting messages. The system does not rely on transport-layer protections (e.g., TLS terminated by the HTTP service) for security; instead, all sensitive material is exchanged through an end-to-end encrypted channel established directly with the enclave, as described in Section 3.4 and Section 3.5. The handshake uses the enclave’s attested public key, so a network adversary cannot impersonate the enclave or recover session contents.

3.2.3 Adversaries considered

The design is intended to resist the following adversaries:

The honest-but-curious or fully malicious service provider. An operator hosting the SGX host (cloud provider, self-hosting administrator, or an attacker who has compromised either) who attempts to recover users’ FIDO2 private keys or to link stored records to user identities. The combination of in-enclave processing, sealing, and the deterministic-derivation design ensures that no useful credential material is reachable outside the enclave.

The opportunistic database attacker. An attacker who obtains a full copy of the credential database through a breach. Such an attacker observes only opaque, sealed records. Because records are encrypted under an enclave-bound key and indexed by values derived inside the enclave, the attacker cannot enumerate users, link records to a known user, or determine whether two records belong to the same user.

The network adversary. An attacker positioned between any two components of the system. Sensitive material is never exposed on the network: communication between the Authenticator Emulator and the enclave uses an end-to-end channel established via an ECIES

handshake against the enclave’s attested public key, with subsequent traffic protected by a request key known only to the Emulator and the enclave. The same construction protects credential transfer during migration. As a consequence, the network adversary is reduced to denial of service, which we treat as out of scope.

The dishonest peer enclave during migration. A party claiming to operate a destination enclave but in fact running unrelated code. Mutual remote attestation, with measurement (MRENCLAVE) verification on both sides, ensures that credentials are only released to an enclave running the same trusted code.

3.2.4 Known limitations and out-of-scope threats

We explicitly acknowledge the following threats as outside the scope of this work:

- **SGX micro-architectural side channels.** Attacks exploiting cache timing, speculative execution, page-fault patterns, and similar micro-architectural side channels against SGX enclaves are an active area of research, and several published attacks have been shown to extract enclave secrets in controlled settings. Our design inherits whatever residual exposure SGX has to these classes of attack; mitigations at the hardware or compiler level are orthogonal to our contribution and are not addressed here.
- **Weak user attributes.** The UID is derived from user-supplied attributes via SHA-256. If the input attribute set has low entropy — for instance, a short or guessable passphrase combined with publicly available personal information — an attacker who has both (i) database access and (ii) the ability to invoke the enclave can mount an online guessing attack against the UID by attempting authentications. We rely on the user to choose attributes with sufficient entropy, and note that the enclave-bound nature of the verification at least precludes offline brute-force attacks against the database alone.
- **Denial of service.** An attacker with control over the host operating system, the HTTP service, or the network can prevent the system from serving requests. Availability is not a security goal of this work; the design instead aims to ensure that loss of availability cannot be converted into loss of confidentiality or integrity. A more specific DoS vector is worth noting: an attacker may submit a large volume of registration requests for arbitrary relying parties, causing the enclave’s set of UID-RP mappings to grow without bound and degrading performance over time in the migration process. Rate-limiting by UID is ineffective here, since UIDs are derived from attacker-controlled inputs and are therefore unbounded in supply.

Defending against this attack requires accounting at a layer the enclave does not control — for example, request quotas enforced by the HTTP service — which is outside the trust boundary and therefore outside the scope of this work’s security guarantees.

- **Compromise of the user host before attribute entry.** A user host compromised with a keylogger or screen capture at the moment attributes are entered will leak those attributes to the attacker. This is a fundamental limitation of any system that relies on user-entered secrets, and is not specific to our design.

- **Physical attacks against the SGX processor.** Decapping, fault injection, and similar hardware attacks against the CPU package are outside our threat model and rely on Intel’s hardware countermeasures.

3.3 Credential creation

During credential registration, the enclave receives three inputs: the UID (a hash of the user’s attributes), the relying party identifier (RP), and the user identifier within that relying party (RP_user).

3.3.1 Key derivation

The enclave first derives a hardware-bound *Identity Key* (IK), reconstructed on every enclave launch from an SGX sealing key and bound to the enclave’s code measurement (MRENCLAVE). The IK therefore never leaves the enclave and is the same across runs of the same enclave on the same platform; it is the root from which all credential-specific material is derived.

The IK is then used to deterministically derive each credential’s Credential_ID and its key pair, via HMAC and HKDF operations over the supplied inputs:

$$\begin{aligned}\text{Credential_ID} &= \text{HMAC}_{\text{IK}}(\text{UID} \parallel \text{RP} \parallel \text{RP_user}) \\ \text{key pair} &= \text{HKDF}(\text{IK}, \text{"key_derive"} \parallel \text{Credential_ID})\end{aligned}$$

Since the key pair is derived from non-random data, no credential private material needs to be stored persistently. The corresponding public key is returned to the relying party as part of the WebAuthn registration response, and the private key is reconstructed on demand whenever an assertion is required.

3.3.2 Persistent record

Although the key pair itself does not require storage, a record of each registered credential must still be persisted, for two reasons. First, the migration process (Section 3.5) must be able to enumerate all credentials associated with a UID, which is only possible if registrations are recorded. Second, when the relying party does not supply a Credential_ID during authentication — as is the case for resident credentials — the enclave must be able to discover the user’s credentials from the UID alone.

Persistence must, however, preserve the unlinkability property stated in Section 3.2: a database compromise must reveal no information that links stored rows to users, relying parties, or one another. To this end, each row stored by the enclave is built from three derived values:

$$\begin{aligned}\text{mac_cred} &= \text{HMAC}_{\text{IK}}(\text{"storage_index"} \parallel \text{Credential_ID}) \\ \text{mac_uid} &= \text{HMAC}_{\text{IK}}(\text{"uid_index"} \parallel \text{UID}) \\ (K_{\text{row}}, \text{IV}) &= (\text{HMAC}_{\text{IK}}(\text{"cred_enc"} \parallel \text{mac_cred}), \text{HMAC}_{\text{IK}}(\text{"cred_iv"} \parallel \text{mac_cred})) \\ \text{blob} &= \text{AEAD}_{K_{\text{row}}, \text{IV}}(P, \text{mac_cred} \parallel \text{mac_uid})\end{aligned}$$

$$P = 0x01 \parallel \text{Credential_ID} \parallel \text{rp_len} \parallel \text{RP} \parallel \text{rpuid_len} \parallel \text{RP_user} \\ \parallel \text{rpusername_len} \parallel \text{RP_username}$$

where P is the plaintext record: a type marker indicating whether the credential was derived locally or imported from another enclave, followed by the `Credential_ID`, the `RP`, the `RP_user` (the WebAuthn-defined user handle within the relying party, returned to the `RP` on every assertion), and an `RP_username` field carrying a human-readable label that the Emulator displays when the user must choose between several resident credentials for the same `RP`. The distinct string labels ("`cred_enc`", "`cred_iv`") act as domain-separation tags, ensuring that K_{row} and the IV are cryptographically independent even though both are derived under the same key (IK) and the same context (`mac_cred`). The `Credential_ID` derivation in Section 3.3 does not include `RP_username`, since the WebAuthn relying party never sees that field and the same logical credential must remain reachable if the user later changes the displayed label. The UID never appears in the database; only its MAC does, serving as an opaque lookup key. The `Credential_ID` likewise never appears in the database directly, but is kept inside the encrypted plaintext: it is needed during migration, when the destination enclave must recompute `mac_cred` under its own IK to re-index the credential (see Section 3.5).

This construction provides three properties:

- **Confidentiality.** The plaintext record is encrypted under a key that only the enclave can derive, since the derivation depends on IK. Furthermore, K_{row} is unique per credential, so each row is protected by an independent key.
- **Unlinkability.** An attacker with full database access sees only opaque MAC values and AEAD ciphertexts. They cannot enumerate users, link two rows to the same UID, or recognize the credential of a known user, because reproducing `mac_cred` or `mac_uid` requires the IK. The domain-separation labels in each HMAC ensure that knowledge of one derived value (e.g., the public `Credential_ID`) does not yield any of the others without IK, since each is a distinct HMAC invocation under IK.
- **Integrity binding.** Because `mac_cred` and `mac_uid` are bound into the AEAD as additional authenticated data, a database-level attacker cannot move a ciphertext row to a different index pair without invalidating the authentication tag.

The same record layout is used for credentials derived locally and for those imported through migration; only the type marker in P differs. The handling of each case during authentication is described in the next section.

3.4 User Authentication

During authentication, the enclave is presented with the UID, the relying party identifier (RP), and the WebAuthn data to be signed: the `authData` structure and the `clientDataHash`. The relying party may also supply a `Credential_ID` indicating which specific credential it expects,

or it may omit it — in which case the enclave must discover all of the user’s credentials registered with that relying party. These two cases correspond to the two authentication flows described below.

In both cases, the enclave first recomputes the same lookup MACs used at registration:

$$\begin{aligned}\text{mac_cred} &= \text{HMAC}_{\text{IK}}(\text{"storage_index"} \parallel \text{Credential_ID}) \\ \text{mac_uid} &= \text{HMAC}_{\text{IK}}(\text{"uid_index"} \parallel \text{UID})\end{aligned}$$

Note that, `mac_cred` is only computed when a `Credential_ID` was supplied. Recall from Section 3.3.2 that `IK` is reconstructed from the SGX sealing key on each enclave launch, so any other enclave — or any other version of this enclave with a different `MRENCLAVE` — would produce different MAC values and therefore be unable to locate or decrypt the user’s records.

3.4.1 Authentication with a Credential ID

When the `Credential_ID` is supplied by the relying party, the enclave fetches the stored row indexed by `mac_cred` through an `ocall`, and decrypts it with the per-row key K_{row} and IV derived from `IK` and `mac_cred` as in Section 3.3.2. Decryption is authenticated: `mac_cred` and `mac_uid` are bound into the AEAD as additional authenticated data, so if the `UID` or `Credential_ID` presented in the request does not correspond to the row stored at that index, the authentication tag fails to verify and the operation is aborted. This is the primary integrity guarantee: the request must match the stored credential at all three of `mac_cred`, `mac_uid`, and the AEAD itself.

If decryption succeeds, the enclave further validates the contents of the recovered plaintext: the `Credential_ID` embedded in the record must match the one in the request, and so must the RP. These checks are redundant with respect to the AEAD — a successful decryption already implies that the row has not been tampered with.

The plaintext record carries a type marker indicating whether the credential was derived locally (`DERIVED`) or imported from another SGX host through migration (`MIGRATED`):

- **Derived credentials** carry no private key in the stored record. The enclave re-derives the key pair on demand from `IK` and the `Credential_ID`, using the same HKDF construction described in Section 3.3.
- **Migrated credentials** carry their private key in the (still-encrypted) record, because they were generated by another enclave’s `IK` and cannot be re-derived locally. The enclave reads the private key directly from the decrypted blob.

With the private key in hand, the enclave signs `authData` $\parallel$ `clientDataHash` using ECDSA over P-256, using the algorithm the authenticator declares as supported in Section 4.3.2.3, zeroes the private key from memory, and returns the signature to the Authenticator Emulator, which completes the WebAuthn flow.

3.4.2 Resident credentials

When no `Credential_ID` is supplied, the enclave enters the discoverable-credential flow. It queries the database for the count of records associated with the supplied `mac_uid`, and then iterates over each such row, decrypting it under its row key and validating the AEAD as in the previous case.

For each successfully decrypted record, the enclave checks whether the embedded RP matches the request RP. Records belonging to other relying parties are skipped. For each matching record, the enclave reconstructs the private key (re-derived or read from the blob, as above), signs `authData || clientDataHash`, and appends a tuple containing the signature, the `Credential_ID`, the user identifier within the relying party (`RP_user`), and the human-readable display label (`RP_username`) to the response.

The presence of `RP_user` and `RP_username` in the response reflects the two distinct consumers of this flow's output. The relying party needs `RP_user` to identify which user account each assertion corresponds to, since it did not specify a credential up front. The Authenticator Emulator needs `RP_username` to present the user with a recognizable chooser when more than one credential is returned: the enclave produces an assertion for every match, and the Emulator asks the user which one to forward to the relying party. The two fields serve different layers and travel together for that reason. If no records match the request RP, the enclave reports failure without revealing whether the UID itself was unknown or whether the user simply has no credentials for that relying party.

3.5 Credential migration

Binding credential derivation to the SGX sealing key has two advantages: it removes the need to persist private keys for locally-derived credentials, and it ensures that no entity outside the enclave — including the host operating system — can reconstruct them. The trade-off is hardware-specific: a credential created on one SGX platform cannot be re-derived on another, because each enclave's IK is unique to its processor.

Migration must therefore transfer enough material from a source enclave to a destination enclave for the destination to use the credentials, while preserving the same security properties at the destination as at the source: no exposure of secret material outside attested enclaves, and no linkability between stored records and any external observer.

3.5.1 What is transferred

Two designs are conceivable for the migration payload:

- Send the source enclave's IK together with each credential's metadata (`UID`, `RP`, `RP_user`, `RP_username`), and let the destination re-derive everything locally.
- Send each credential's metadata together with its private key, and let the destination re-encrypt and re-index everything under its own IK.

The first option would expose the source IK outside its creator enclave, breaking the central invariant of the design: IK is hardware-bound and never leaves the enclave. We therefore adopt the second option. The migration payload contains, for each credential, the type marker, Credential_ID, RP, RP_user, RP_username, and the credential's private key. For credentials stored in *derived* form (whose record carries no private key, since it can be re-derived from the source IK), the source enclave first regenerates the private key and appends it to the record, converting the credential into the *migrated* form before transmission. Credentials already in migrated form, those the source enclave itself had previously imported from elsewhere, are forwarded as they are stored.

3.5.2 Protocol

Migration is initiated by the user, who instructs the Authenticator Emulator to back up the credentials associated with a given UID from a source SGX host to a destination SGX host. The Emulator is the only entity that knows both endpoints and the UID involved, and acts as the trusted-by-the-user mediator throughout the exchange. The protocol itself follows ideas presented in [48], consists of three messages. Throughout, all Emulator-to-enclave traffic is itself protected by the per-request channel described in Section 3.4: the Emulator establishes an ephemeral-ECDH session with each enclave it talks to, and request and response bodies are encrypted under the request key. This protects credential migration against a network attacker observing between the Emulator and either SGX host. Inside this request-level encryption, the credential payload itself is protected by a second, end-to-end envelope between the two enclaves, which the Emulator cannot decrypt.

The protocol relies on the attested enclave identity keypairs introduced in Section 3.1: each enclave's stable, IK-derived public key, bound into its quote via the report data field, is what allows the other side to verify it is communicating with a genuine peer running the expected codebase.

1. **Destination quote.** (Figure 3.2) The destination enclave is asked to produce an SGX quote. The quote attests the destination's stable identity public key P_{dst} via the report data field. The Emulator receives the quote on its session with the destination, and forwards it on its session with the source, together with the UID of the user whose credentials are to be migrated. Since the P_{dst} present in the report is always constant, this step is not required in each migration process. An older, but valide, quote can still be used.

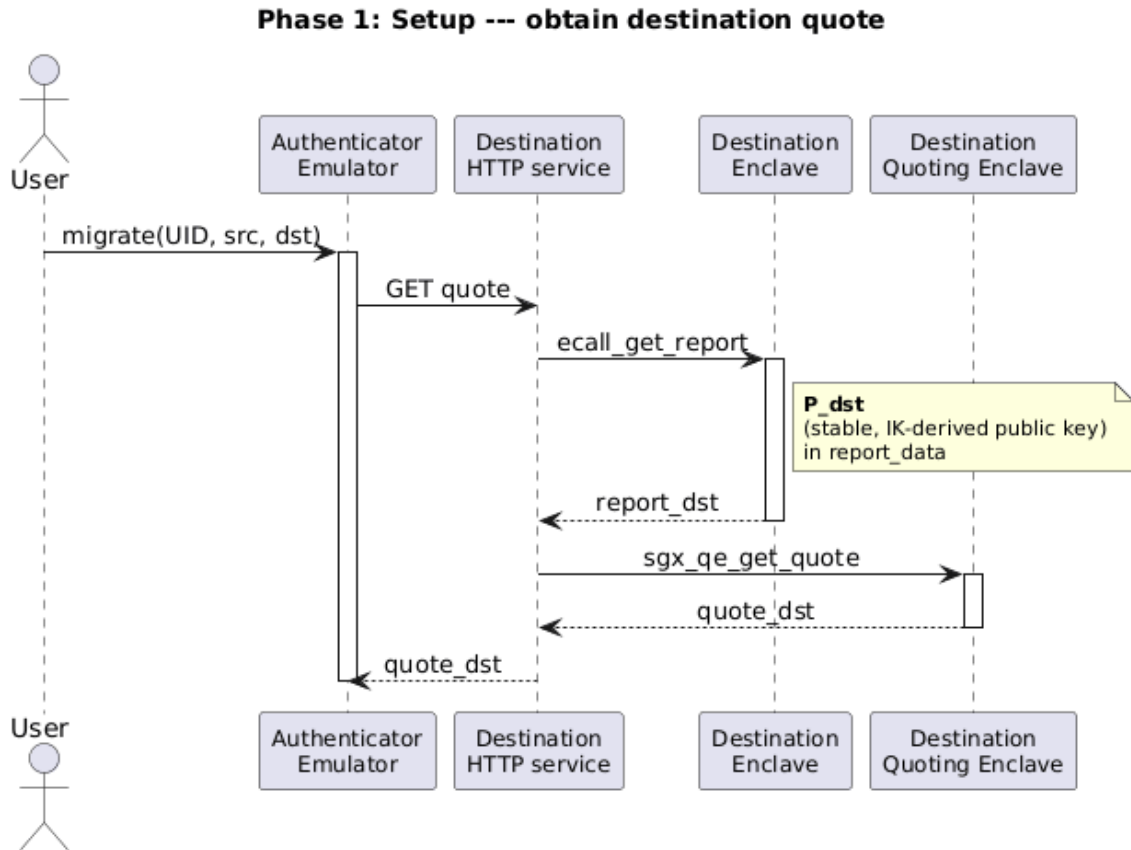


Figure 3.2: Migration, phase 1. The user requests migration through the Authenticator Emulator, which asks the destination SGX host for a quote. The destination's enclave produces a quote attesting its stable IK-derived public key P_{dst} via the report data field.

2. **Encrypted payload from the source.** (Figure 3.3) The source enclave verifies the destination's quote: it validates the DCAP certificate chain and checks that the destination's MRENCLAVE matches its own, ensuring that the peer is running an identical and trusted codebase. It then extracts P_{dst} from the report data and constructs an end-to-end transfer envelope:

- A fresh ephemeral P-256 keypair (e_{priv}, e_{pub}) is generated.
- The source then produces its own quote, binding e_{pub} into the report data field. Unlike the destination's quote — which attests a stable identity key — the source's quote attests a per-migration ephemeral key. The reason is discussed in Section 3.5.3.
- A shared secret is derived: $s = \text{ECDH}(e_{priv}, P_{dst})$.
- A wrapping key is derived: $K_{wrap} = \text{HMAC}(s, \text{"wrap_key"})$.

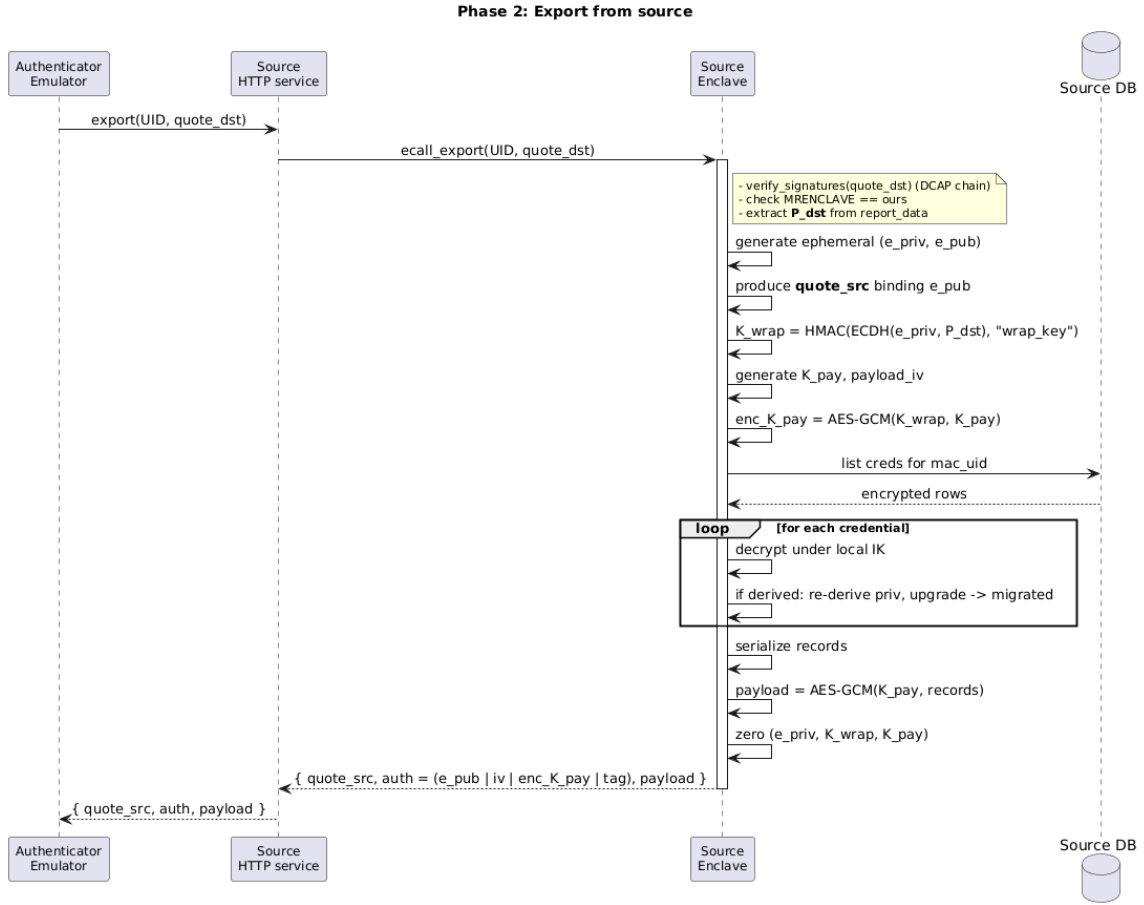


Figure 3.3: Migration, phase 2. The source enclave verifies the destination’s quote, generates a fresh ephemeral keypair $(e_{\text{priv}}, e_{\text{pub}})$, produces its own quote binding e_{pub} into report data, and constructs an ECIES-style envelope. It then enumerates the user’s credentials, decrypts each under its local IK (upgrading derived credentials to migrated form), encrypts the serialized records under the fresh payload key K_{pay} , and returns the bundle to the Emulator.

- A fresh symmetric payload key K_{pay} and a payload IV are generated. K_{pay} is then encrypted under K_{wrap} with AES-GCM.

The source enclave then enumerates all of the user’s credentials, decrypts each record under its local IK, upgrades any derived credentials to migrated form by appending their re-derived private key, and serializes the resulting records into a single transfer buffer. The buffer is encrypted under K_{pay} with AES-GCM. The source returns to the Emulator: its own quote, the transfer envelope (e_{pub} , the wrap IV, the encrypted K_{pay} , and the GCM tag), and the encrypted credential payload. All ephemeral private material (e_{priv} , K_{wrap} , K_{pay}) is zeroed inside the source enclave before the response leaves.

From the Emulator’s perspective, the source’s quote is a public artifact and is visible in cleartext within the Emulator-source session; the credential payload is opaque, since the Emulator does not hold IK_{dst} and cannot complete the ECDH that would yield K_{wrap} .

3. **Import at the destination.** (Figure 3.4) The Emulator forwards the source’s quote, the transfer envelope, and the encrypted payload to the destination, on its session with the destination. The destination first verifies the source’s quote — chain validation and

MRENCLAVE comparison — and then extracts $e_{\text{pub}}^{\text{quote}}$ from the source quote’s report data. It compares this against the e_{pub} supplied inside the transfer envelope and rejects the import if the two do not match. This check ensures that the ephemeral key used in the ECDH exchange was generated by the same attested source enclave whose quote is presented, and not substituted by an intermediary.

Once the binding is verified, the destination completes the ECIES-style exchange: it derives $s = \text{ECDH}(\text{IK}_{\text{dst-priv}}, e_{\text{pub}})$, recomputes K_{wrap} , unwraps K_{pay} , and decrypts the credential payload.

For each credential record in the decrypted payload, the destination computes, under its own IK:

$$\begin{aligned}\text{mac_uid}^{\text{dst}} &= \text{HMAC}_{\text{IK}_{\text{dst}}}(\text{"uid_index"} \parallel \text{UID}) \\ \text{mac_cred}^{\text{dst}} &= \text{HMAC}_{\text{IK}_{\text{dst}}}(\text{"storage_index"} \parallel \text{Credential_ID})\end{aligned}$$

and derives a fresh K_{row} and IV under its own IK as in Section 3.3.2. The credential record is then re-encrypted under the destination’s per-row keys, with the new MACs bound into the AEAD as additional authenticated data, and inserted into the destination’s database. Duplicate credentials already present at the destination — recognizable by colliding $\text{mac_cred}^{\text{dst}}$ values — are detected at insertion and skipped.

The result is that the same logical credential is indexed under entirely different mac_cred and mac_uid values at the source and at the destination, and is encrypted under entirely different per-row keys. An attacker who obtains both databases cannot tell that any pair of rows corresponds to the same credential, preserving the unlinkability property of Section 3.2 across migration boundaries.

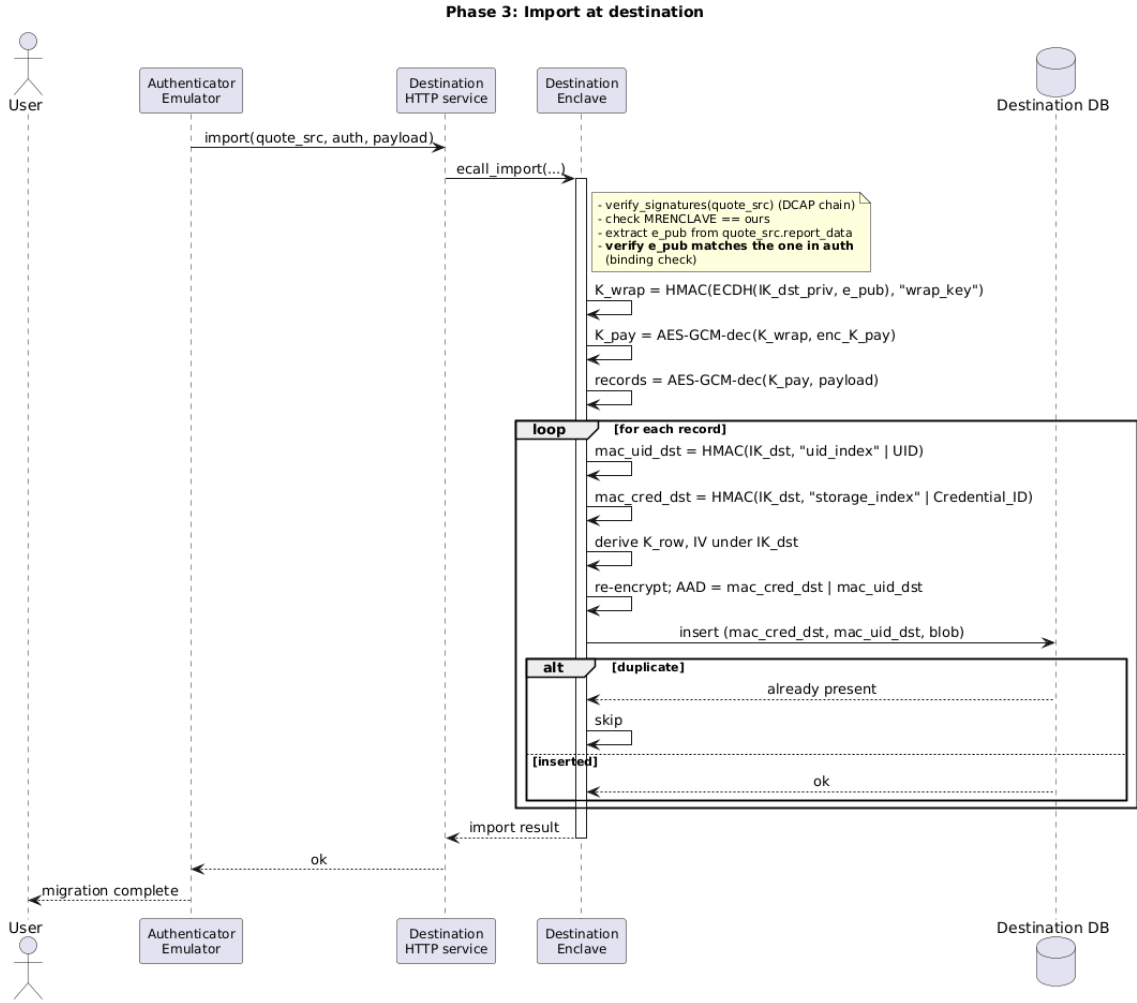


Figure 3.4: Migration, phase 3. The destination enclave verifies the source’s quote and crucially checks that the ephemeral public key e_{pub} in the transfer envelope matches the one attested in the source’s quote. After this binding check, it completes the ECDH exchange, unwraps the payload key K_{pay} , decrypts the credential payload, and re-indexes each record under its own IK before insertion.

3.5.3 Mutual attestation and key freshness

The migration protocol provides mutual remote attestation. Each side verifies the other’s quote, and crucially, the public key used in each side’s role in the ECDH exchange is bound into its own quote’s report data field:

- The destination’s quote attests its stable IK-derived public key P_{dst} , since that is the key the source will use to encrypt material destined for it.
- The source’s quote, generated per migration, attests the ephemeral public key e_{pub} that the destination will use in its own ECDH computation. The ephemeral keypair is generated immediately before the quote and zeroed inside the source enclave once the response leaves.

The asymmetry — stable key on the destination, ephemeral on the source — reflects each enclave’s role in the protocol. The destination must publish its public key before knowing who

the source will be, and would otherwise need to persist an ephemeral private key across ecalls; using its stable, IK-derived key avoids this entirely, since it can be re-derived on demand from a sealing key. The source, in contrast, performs the entire export inside a single ecall, so a per-call ephemeral keypair fits naturally and provides forward secrecy as a bonus.

This binding closes what would otherwise be a credential-injection vulnerability. Without it, any party in possession of a single valid source quote with the expected MRENCLAVE could substitute their own ephemeral public key and inject arbitrary attacker-chosen credentials into a destination's database, since the destination would accept any envelope that decrypts cleanly under its own IK. With the binding in place, the destination verifies that the e_{pub} in the transfer envelope is the one attested in the source's quote, and rejects the transfer otherwise. An attacker who substitutes their own e_{pub} would need to produce a fresh quote attesting that key under the expected MRENCLAVE, which they cannot do without compromising SGX itself.

Although the DCAP quote signature is non-deterministic (the underlying ECDSA signing draws a fresh nonce per call, per [51]), the protocol does not bind quotes to a per-request challenge. This is acceptable because freshness is guaranteed by the ephemeral keypair: a replayed source quote attests the same e_{pub} as before, but the corresponding e_{priv} existed only inside the original source enclave at the moment of export and was zeroed before the response left. A replay therefore yields a transfer envelope the replayer cannot decrypt, since they hold neither e_{priv} nor IK_{dst} .

3.5.4 Migration of imported credentials

The destination enclave is not a passive backup. After import, it is fully functional: it can register new credentials, serve assertions, and itself act as the source in a future migration to a third enclave. Two kinds of credential therefore coexist in any given database: those derived natively from the enclave's own IK (the *derived* type), and those imported through migration with their private key embedded in the record (the *migrated* type). The export logic handles both uniformly: derived credentials are upgraded to migrated form on the fly, while migrated credentials are forwarded as stored. The destination, in turn, always stores incoming credentials in migrated form, since it cannot regenerate them from its own IK.

3.5.5 Trust root for quote verification

Both quote validations rely on verifying the DCAP certificate chain, which terminates in the Intel SGX Provisioning Certification Root CA. The Authenticator Emulator can fetch this root at runtime from Intel's servers,¹ but the enclave cannot: outbound network requests from inside an enclave are not possible, and obtaining the certificate from the surrounding host would mean trusting an untrusted component for a critical trust anchor. We therefore embed the root CA directly in the enclave code at build time.

The certificate is valid until 2049, after which the enclave code must be updated, changing its MRENCLAVE measurement. Any party that pins trust to the old measurement will need to be made aware of the change, and credentials stored under the old MRENCLAVE cannot be opened

¹https://certificates.trustedservices.intel.com/Intel_SGX_Provisioning_Certification_RootCA.pem

by the new enclave; users would need to re-register their credentials with each relying party. This is a known maintenance cost of binding the trust root to the enclave code, and we accept it as the price of avoiding a runtime dependency on an untrusted host.

3.6 Security properties

This section summarizes the security guarantees of the design and identifies, for each, the mechanisms in earlier sections that deliver it and the adversaries from Section 3.2 that it resists. The aim is not to re-describe the protocols but to make the guarantees explicit and traceable.

3.6.1 Confidentiality of credential material at rest

FIDO2 private keys are never written to persistent storage in plaintext, and in the common case — credentials derived locally — they are never written to storage at all. Locally-derived keys are reconstructed on demand from the Identity Key (IK) and the Credential_ID, both inside the enclave (Section 3.3). Migrated credentials, which cannot be re-derived locally, are stored with their private key embedded in the AEAD-encrypted record, under a per-row key K_{row} that itself depends on IK (Section 3.3.2).

Because IK is reconstructed on each enclave launch from a SGX sealing key, bound to MRENCLAVE, neither the host operating system, nor the HTTP service, nor any other enclave — including a different version of this enclave — can derive IK or K_{row} . This property defeats the *honest-but-curious or fully malicious service provider* and the *opportunistic database attacker*: a full database breach yields only AEAD ciphertexts under keys the attacker cannot reconstruct.

3.6.2 Unlinkability of stored records

The credential database contains, for each registered credential, a row of the form (mac_cred, mac_uid, blob), where both indexes are HMACs computed under IK and the blob is an AEAD ciphertext (Section 3.3.2). Neither the UID nor the Credential_ID appears in the database in plaintext.

This means an attacker with full read access to the database — the *opportunistic database attacker* — cannot:

- enumerate the users registered in the system, since user identities exist nowhere on disk;
- determine whether a given user has any credentials, since reproducing mac_uid requires the IK;
- determine whether two rows correspond to credentials of the same user, for the same reason;
- correlate any row with a specific relying party, since the RP identifier is inside the encrypted blob.

The property extends across migration boundaries: as described in Section 3.5, the destination enclave re-indexes each imported credential under its own IK, so the same logical

credential is stored under entirely different `mac_cred` and `mac_uid` values at the source and at the destination. An attacker who compromises both databases cannot tell that any pair of rows refers to the same credential.

3.6.3 Integrity of stored records

The MAC values `mac_cred` and `mac_uid` are bound into each row’s AEAD as additional authenticated data (Section 3.3.2). Consequently, an attacker who manipulates the database — swapping a credential row to a different index pair, splicing rows across users, or tampering with the ciphertext — causes the AEAD authentication tag to fail on the next access, and the operation aborts inside the enclave.

This defeats integrity-targeted variants of the *opportunistic database attacker* and the *honest-but-curious or fully malicious service provider*: while such an attacker can delete or corrupt rows (causing availability loss), they cannot silently substitute or relocate credential material. The authentication flow further re-checks the `Credential_ID` and the RP embedded in the decrypted record (Section 3.4), providing defense in depth against a misbehaving database layer.

3.6.4 End-to-end channel confidentiality

All traffic between the Authenticator Emulator and the enclave is wrapped in a per-request channel established by ephemeral-ECDH against the enclave’s stable, attested public key. The Emulator generates a fresh ephemeral keypair per request and derives a request key that only the enclave can compute, because deriving it requires the enclave’s IK-derived private key (Section 3.4).

This defeats the *network adversary*: messages on the wire are AEAD-encrypted under a key the attacker cannot derive without compromising SGX, and the enclave’s identity is bound into its quote, so a man-in-the-middle cannot substitute its own public key without producing a fresh quote attesting that key under the expected `MRENCLAVE`. The system therefore does not depend on TLS or any other transport-layer protection for its security claims.

3.6.5 Mutual attestation in migration

Credential migration requires both enclaves to verify each other’s SGX quote (Section 3.5.3). Each side checks two properties: (i) the DCAP certificate chain validates, proving the quote was produced by a genuine SGX platform; and (ii) the peer’s `MRENCLAVE` matches the verifier’s own, proving the peer is running an identical and trusted codebase.

Crucially, the public key used by each side in the ECDH exchange is bound into its own quote’s report data field. The destination’s quote binds its stable identity key P_{dst} ; the source’s quote, generated per migration, binds the ephemeral key e_{pub} used for the ECIES exchange. The destination explicitly checks that the e_{pub} supplied in the transfer envelope matches the one attested in the source’s quote.

Together, these checks defeat the *dishonest peer enclave during migration* and close what would otherwise be a credential-injection vulnerability against the destination: without the e_{pub} binding, an attacker who obtained any valid quote with the expected `MRENCLAVE` could

submit attacker-chosen credentials encrypted under an attacker-chosen ephemeral key, since the destination would accept any envelope that decrypts cleanly under its own IK.

3.6.6 Forward secrecy of migration transfers

The source’s contribution to the ECDH key agreement in migration is an ephemeral keypair, generated immediately before the transfer and zeroed inside the source enclave before the response leaves (Section 3.5.3). The wrapping key K_{wrap} that protects the payload key K_{pay} therefore depends on a private value that exists only momentarily and never persists.

An attacker who later compromises IK_{dst} already holds the destination’s database decryption capability, so confidentiality of stored credentials is lost regardless. Forward secrecy adds a narrower guarantee: the attacker cannot decrypt past migration *network* payloads observed on the wire, because reconstructing K_{wrap} also requires the ephemeral e_{priv} , which is gone. The value of this is that intercepted network traffic is generally easier to obtain at scale than the destination’s database file, so the forward-secrecy property closes the cheaper of the two attack paths.

A separate concern applies if an attacker later recovers an ephemeral e_{priv} . The attacker cannot recover the original message without IK_{dst} , but they hold a valid source quote whose `report_data` binds the corresponding e_{pub} , which is exactly what the destination’s binding check (Section 3.5.3) accepts. They can therefore craft a new transfer envelope under the captured e_{priv} , fill it with attacker-chosen credentials, and present it together with the original source quote to a destination of their choice, re-opening the credential-injection vulnerability for any future import. The destination could close this by maintaining a list of ephemeral keys that have already been consumed by a successful import and rejecting any envelope whose e_{pub} appears on that list. The list assumes e_{priv} stays secret during the short window between export and import: if compromise happens inside that window, the attacker’s forged envelope reaches the destination before the legitimate one and registers itself on the list, so the legitimate import is rejected instead.

3.6.7 Summary

Table 3.1 maps each security property to the threats from Section 3.2 that it counters.

Three threats from Section 3.2 are not addressed by the cryptographic design and are acknowledged as out of scope: SGX micro-architectural side channels, weak user attributes (online guessing against the UID), and denial of service. The first relies on hardware-level

Table 3.1: Mapping of security properties to threats from the threat model.

Property	Threats addressed
Confidentiality at rest	Malicious service provider; opportunistic database attacker
Unlinkability of records	Opportunistic database attacker
Integrity of records	Opportunistic database attacker; malicious service provider
Channel confidentiality	Network adversary
Mutual attestation in migration	Dishonest peer enclave; credential-injection variant of network adversary

mitigations outside this work; the second on the user choosing sufficient entropy; the third on host-level accounting in an untrusted layer.

Implementation

This chapter describes the implementation of the proposed solution. Section 4.1 covers the Authentication Emulator and how it presents itself as a FIDO2 token to client applications. Section 4.2 covers the CTAP interactions between the two entities and the engineering choices made in the SGX SDK. Section 4.3 covers the SGX Host: the enclave code, the host-side HTTP service, and the credential storage.

4.1 Authentication Emulator

4.1.1 UHID device

The Emulator targets Linux and uses the kernel’s UHID interface to present itself as a USB HID device entirely from userspace. UHID (User-space HID I/O driver) is a kernel subsystem that lets a userspace process implement HID transport drivers without writing any kernel code: the process opens the character device `/dev/uhid`, issues a `UHID_CREATE2` event describing the device, and then reads and writes HID reports through the same file descriptor. This is the natural transport for the Emulator, since CTAP over HID is the protocol that browsers and other Clients already speak to roaming authenticators.

4.1.1.1 Device creation

At startup, the Emulator opens `/dev/uhid` for read/write and sends a `UHID_CREATE2` event containing the device’s HID report descriptor and identifying information¹. The report descriptor is the most security- and compatibility-relevant part of the device declaration: it tells the kernel’s HID core — and any userspace stack walking it through `/dev/hidraw` — what kind of device this is and what reports it accepts.

A HID report descriptor is a sequence of typed items, each declaring one aspect of the device’s interface (its usage class, the size and direction of its reports, the value ranges of its data, and so on). The Emulator’s descriptor declares the minimum needed for a CTAP-HID authenticator, and is reproduced in Listing 1.

The descriptor breaks down as follows. The opening `Usage Page (0xF1D0)` item places the device under the FIDO Alliance usage page, and the following `Usage (0x01)` declares it

¹The UHID event types, payload structures are defined in the kernel header `/usr/include/linux/uhid.h`, which we treat as the authoritative specification.

```

0x06, 0xD0, 0xF1,      # Usage Page (FIDO Alliance, 0xF1D0)
0x09, 0x01,            # Usage (U2F Authenticator Device)
0xA1, 0x01,            # Collection (Application)

0x09, 0x20,            # Usage (Input Report Data)
0x15, 0x00,            # Logical Minimum (0)
0x26, 0xFF, 0x00,      # Logical Maximum (255)
0x75, 0x08,            # Report Size (8 bits)
0x95, 0x40,            # Report Count (64)
0x81, 0x02,            # Input (Data, Variable, Absolute)

0x09, 0x21,            # Usage (Output Report Data)
0x15, 0x00,            # Logical Minimum (0)
0x26, 0xFF, 0x00,      # Logical Maximum (255)
0x75, 0x08,            # Report Size (8 bits)
0x95, 0x40,            # Report Count (64)
0x91, 0x02,            # Output (Data, Variable, Absolute)

0xC0                   # End Collection

```

Code 1: HID report descriptor used by the Emulator. Each line is one HID descriptor item.

specifically as a U2F Authenticator Device. This is the marker that browsers and OS-level HID enumerators look when discovering CTAP authenticators on the system; without this exact pair, the device would still be a valid HID device but no FIDO-aware client would route CTAP traffic to it.

The `Collection (Application)` bracket groups the rest of the descriptor as the device's primary functional surface. Inside the collection, two reports are declared:

- An **Input Report** (host receives data *from* the device), with `Usage (0x20)` marking it as CTAP-HID input data, a logical value range of 0–255, and dimensions of 64 reports of 8 bits each — in other words, a 64-byte payload. The `Input (Data, Variable, Absolute)` item commits these properties as a single Input report of that shape.
- An **Output Report** (host sends data *to* the device), with `Usage (0x21)`, the same value range, and the same 64-byte payload shape.

The 64-byte report size is mandated by the CTAP-HID transport, which fragments larger CBOR-encoded CTAP messages across multiple 64-byte packets [46]. The closing `End Collection` item terminates the application collection and the descriptor.

Together, these items tell the kernel everything it needs to know to expose the virtual device as a CTAP-HID authenticator: clients enumerating HID devices will see the FIDO usage page, recognize the U2F authenticator usage, and begin exchanging 64-byte HID reports through the Input and Output channels.

Beyond the descriptor, the `UHID_CREATE2` payload also carries device-identifying strings (name, physical path, serial number) and integer fields for bus type, vendor ID, product ID, and version. The Emulator announces itself with the vendor and product identifiers of a real Yubico YubiKey (`0x1050:0x0407`). Although the Emulator could use arbitrary identifiers, using known-good ones avoids edge cases in client stacks that maintain allowlists or apply heuristics keyed on device IDs.

Once the kernel accepts the `UHID_CREATE2` event, the device appears in `/dev/hidrawN`.

4.1.1.2 Event loop

After device creation, the Emulator enters a single-threaded read loop on the UHID file descriptor. Each iteration waits up to 100 ms for the kernel to deliver an event, using the POSIX `select` system call to avoid busy-waiting; if the timeout elapses with no event, the loop checks whether a shutdown has been requested and otherwise loops again. This keeps the Emulator responsive to both incoming HID traffic and external termination signals without spawning additional threads.

Each event read from the kernel begins with a 4-byte event type. The two events relevant to CTAP-HID traffic are:

- `UHID_OUTPUT` (type 6): the kernel delivers an HID Output report from the host. For CTAP-HID, this is one packet of a (possibly fragmented) CTAP-HID message destined for the authenticator. The payload follows a fixed layout: 4096 bytes of data, a 2-byte size field, and a 1-byte report type. The Emulator extracts the first *size* bytes as the actual HID data and passes it to the CTAP-HID handler described in Section 4.1.2.
- `UHID_INPUT2` (type 12): the Emulator writes this event back to the kernel to deliver an HID Input report to the host. Each response from the CTAP handler may yield one or more 64-byte HID packets (CTAP-HID fragments a single CBOR response across multiple packets when needed).

Other UHID events (`UHID_OPEN`, `UHID_CLOSE`, etc.) are received but not acted upon, since the Emulator's behavior does not depend on host-side open/close transitions.

On shutdown, the Emulator writes a `UHID_DESTROY` event and closes the file descriptor, removing the virtual device from the system.

4.1.1.3 Implications for the design

UHID gives the Emulator a few useful properties at no cost. First, no kernel module or root-level driver code is required — the Emulator runs as a normal userspace process and depends only on access to `/dev/uhid`. That access is, however, not granted by default: on a typical Linux installation, the device node is owned by root with restrictive permissions, as illustrated below.

```
$ ls -la /dev/uhid
crw----- 1 root root 10, 239 May 21 21:09 /dev/uhid
```

In practice, this means the Emulator must either be launched with elevated privileges, granted access via a udev rule, or have its user added to a group that owns the device node. This is a one-off administrative step but worth acknowledging: the privilege to create a virtual HID device is non-trivial, since the same mechanism could in principle be used to impersonate any HID class device on the system.

Second, the Emulator is indistinguishable from a hardware authenticator at the transport level: WebAuthn implementations in Chromium, Firefox, and other host applications all discover and exercise it without modification, which made client-side validation straightforward.

Third, the transport itself is reused unmodified — all of the protocol logic above the HID layer (CTAP-HID framing, CBOR encoding, command dispatch) lives in userspace and is testable in isolation.

4.1.2 CTAP Interface

Above the UHID transport, the Emulator implements the CTAP protocol stack in two layers: a transport layer (CTAP-HID) that fragments and reassembles messages across 64-byte HID packets, and a protocol layer (CTAP2) that interprets the messages themselves.

4.1.2.1 CTAP-HID framing

Every CTAP-HID packet is exactly 64 bytes, the size mandated by the report descriptor declared in Section 4.1.1. Each packet begins with a 4-byte channel identifier (CID) that distinguishes concurrent CTAP-HID sessions on the same device. The fifth byte determines the packet's role: if its high bit is set, the packet is an *initialization* packet and the remaining seven bits encode the command; otherwise the packet is a *continuation* packet and the remaining seven bits encode a sequence number.

This bit-level discriminator lets a single virtual device interleave multiple commands on different channels without ambiguity. The Emulator tracks per-CID state in a dictionary indexed by channel: when an initialization packet declares a payload length larger than 57 bytes (the largest payload that fits alongside the 7-byte CTAP-HID header in a single 64-byte packet), the partial payload is stored and the Emulator returns control to the read loop to wait for continuation packets. Each subsequent continuation packet contributes up to 59 additional bytes (the 5-byte continuation header leaves 59 bytes of payload). The Emulator validates the sequence number on every continuation and discards the in-flight message if a packet arrives out of order, as required by the CTAP-HID specification.

Outgoing responses use the same fragmentation rules in reverse: the first response packet repeats the request's command byte and declares the total payload length, and continuation packets are emitted with monotonically increasing sequence numbers until the payload is exhausted.

4.1.2.2 Command dispatch

CTAP-HID defines a small set of transport-level commands. The Emulator implements four of them:

- **INIT** (0×86): session initialization and channel allocation, described separately in Section 4.1.2.3 because it is the hook for the attestation handshake with the SGX host.
- **CBOR** (0×90): the entry point for CTAP2 commands. The first byte of the payload identifies the CTAP2 command; the remainder is a CBOR-encoded parameter map. The

Emulator routes by command byte to dedicated handlers for `authenticatorGetInfo`, `authenticatorMakeCredential`, `authenticatorGetAssertion`, and `authenticatorSelection`.

- `PING (0x81)`: echo, used by clients to probe device responsiveness.

Other CTAP-HID commands (`MSG`, `LOCK`, `WINK`, `CANCEL`) are not implemented, since none of them is required by the operations the Emulator supports.

The CTAP2 command set if the Emulator is also intentionally narrow. `authenticatorClientPIN` and the PIN-related sub-commands are absent because the design declares built-in user verification (`uv = true`) and performs verification cryptographically via the user-attributes-derived UID, as discussed in Section 3.1.1.1. Removing PIN support simplifies the implementation considerably: there is no PIN/UV token to provision, no PIN protocol state machine, and no need to bind authentications to a per-session PIN authentication.

4.1.2.3 INIT and the attestation handshake

The CTAP-HID `INIT` command is sent by the client at the start of every session to negotiate a channel identifier and exchange protocol capabilities. The standard response carries the client's echoed nonce, an assigned CID, a protocol version, build numbers, and a capability bitmask.

Binding the attestation to `INIT` was a convenience choice rather than a protocol requirement. The CTAP-HID specification reserves `INIT` as the first command of any session, so it offers a natural moment for the Emulator to perform any one-time setup before credential operations begin. Coupling the SGX attestation handshake to this same moment kept the Emulator's flow simple: when an `INIT` arrives, the Emulator obtains a quote, validates it, and stores the resulting enclave public key alongside the CID, so every subsequent CTAP2 command on that channel can be encrypted against an already-verified target.

Nothing in the security argument depends on this binding. The attestation could equally happen at Emulator startup, on the first credential operation, or on demand from a separate management interface, and the per-request channel construction (Section 4.2) would behave identically. Quotes also do not need to be re-fetched on every session: a successfully validated quote can be cached, keyed by the SGX host's address, and re-used. The current implementation re-fetches on each `INIT` because the cost is modest at the request volumes considered here and a fetch is the simplest way to detect a host that has been redeployed under a different `MRENCLAVE`. A production deployment with higher request volume would benefit from a TTL-bound cache.

In the Emulator, `INIT` additionally serves as the trigger for the per-host attestation handshake with the SGX host. On the `INIT` of a session the Emulator issues an out-of-band request to the SGX host's quote endpoint, retrieves the enclave's SGX quote, and validates it in four stages. The first three correspond to the cryptographic and structural integrity checks defined by the DCAP ECDSA quote format, revision 1.3 [52], against which this implementation is built; the fourth is the higher-level platform freshness check against Intel's TCB service.

1. **Quote integrity.** The Emulator verifies the three signatures that bind the quote together internally:

- The *ISV Enclave Report Signature*, an ECDSA-P256 signature over the quote header and the ISV (Independent software vendor) enclave report, is verified against the *ECDSA Attestation Key* (AK) carried in the quote.
- The AK's identity is then bound to the Quoting Enclave by recomputing SHA-256(AK || QE_Auth_Data) and checking that the result matches the first 32 bytes of the QE Report's `report_data` field. This step prevents an adversary from re-signing the enclave report under a substituted attestation key.
- The *QE Report Signature*, an ECDSA-P256 signature over the QE Report, is verified against the *Provisioning Certification Key* (PCK) recovered from the quote's certificate data.

Together, these checks prove that the quote is a coherent artifact whose signature chain leads back to a single PCK, rather than a stitched-together combination of fragments.

2. **Platform genuineness.** The PCK certificate chain embedded in the quote's QE Certification Data is walked up to the Intel SGX Provisioning Certification Root CA. The Root CA is fetched at runtime from Intel's distribution endpoint² when the Emulator starts and used as the trust anchor for the chain walk. For each certificate in the chain, the Emulator checks that the current time falls within the certificate's `notBefore` and `notAfter` validity period. It also fetches the platform CRL from Intel's PCK CRL endpoint³ and verifies that none of the certificates in the chain appears in the revocation list. Together these checks prove that the PCK belongs to a genuine Intel SGX platform whose attestation credentials are still in force.
3. **Platform freshness.** The Emulator extracts the platform's FMSPC value from the PCK certificate and queries Intel's TCB Info service⁴ to obtain a signed TCB Info document. The TCB level reported by the platform in its PCK certificate extensions is then looked up in this document; the quote is accepted only if the level is reported as `UpToDate`. This guards against the case where a quote is cryptographically valid but issued by hardware running outdated microcode or SGX firmware with known vulnerabilities.
4. **Enclave identity.** The `MRENCLAVE` measurement read from the verified ISV enclave report is compared against the value the Emulator expects for the deployment. A mismatch indicates the SGX host is running a different (possibly modified) enclave codebase and the session is rejected.

The enclave's identity public key (COSE algorithm ES256, ECDSA over P-256) is then extracted from the ISV enclave report's `report_data` field, and is loaded into memory on the handler instance as the anchor for all subsequent communication with that SGX host.

If any step of the attestation fails, the Emulator returns a CTAP-HID error response to the client and refuses to proceed with the session. From the client's perspective, this manifests as

²https://certificates.trustedservices.intel.com/Intel_SGX_Provisioning_Certification_RootCA.pem

³<https://api.trustedservices.intel.com/sgx/certification/v4/pckcrl?ca=platform&encoding=pem>

⁴<https://api.trustedservices.intel.com/sgx/certification/v4/tcb?fmssp=<FMSPC>>

a non-functional authenticator; from the security perspective, it is the implementation-level realization of the channel-confidentiality property of Section 3.6: the Emulator never engages in CTAP2 traffic with an enclave whose code has not been remotely attested.

After successful attestation, the Emulator builds the standard CTAP-HID `INIT` response: the eight-byte nonce is echoed, a freshly-allocated CID is assigned (the previous channel identifier is replaced if the request arrived on the broadcast CID `0xFFFFFFFF`), the protocol version is set to 2 (CTAP2), and the capability byte advertises support for CBOR commands. The response is fragmented across CTAP-HID packets in the usual way and written back to the kernel as `UHID_INPUT2` events.

4.1.2.4 CTAP2 command surface

The CTAP2 commands routed through the CBOR transport are:

authenticatorGetInfo (0x04). Returns the authenticator’s capabilities (supported algorithms, extensions, AAGUID, transport, options including `uv`). The contents are obtained from the SGX host through the encrypted request channel rather than being hardcoded in the Emulator, so that an upgrade of the enclave’s policy (e.g. enabling a new algorithm) is reflected in the response without an Emulator-side change.

authenticatorMakeCredential (0x01). Registers a new credential. The Emulator extracts the relying party identifier, the user identifier within the relying party, and the client data hash from the CBOR parameter map; collects the user’s attributes through the local UI; derives the UID (Section 4.1.3); and forwards the request to the SGX host’s `makecredential` endpoint. On success the SGX host returns a `Credential_ID` and the credential’s public key, which the Emulator assembles into the WebAuthn `authData` and attestation response described by the CTAP2 specification.

authenticatorGetAssertion (0x02). Produces an assertion for a registered credential. The Emulator extracts the relying party, client data hash, and optional allow-list, collects user attributes, and forwards the request to the SGX host. The response carries the ECDSA signature and the `Credential_ID` used; for resident credentials, where the allow-list is absent, the response also contains the user identifier within the relying party, as described in Section 3.4.

authenticatorSelection (0x0B). Used by some clients to disambiguate between multiple connected authenticators. The Emulator responds with success, since selection is straightforward when only one virtual device is present.

The dispatch logic itself is mechanical: a single switch on the CTAP2 command byte routes to the appropriate handler, with unknown commands answered by `CTAP2_ERR_INVALID_COMMAND`. The handlers for `MakeCredential` and `GetAssertion` are the substantive ones, since both bridge between the CTAP2 protocol on the client side and the SGX host’s encrypted endpoints on the network side; their behaviour is described together with the corresponding SGX-host operations in Section 4.3.

4.1.3 User Attribute Collection and UID derivation

The Authenticator Emulator does not perform user verification through a PIN or biometric. Instead, as discussed in Section 3.1, verification is realized by binding cryptographic operations to a user identifier (UID) derived from attributes that the user supplies at the moment of each registration or authentication. This subsection describes how those attributes are collected and how the UID is computed.

4.1.3.1 Attribute set

The Emulator currently collects five attributes from the user, chosen to provide a mix of stable identifying information and a user-controlled secret:

- `User` — a name or handle.
- `Domain` — a logical scope (for instance, an organization or an arbitrary user-chosen label that distinguishes one credential set from another).
- `Index` — a small disambiguator chosen by the user, allowing them to maintain multiple parallel credential sets on the same host.
- `ID` — a personal identifier (e.g. an identity-document number or a similar long-lived identifier).
- `Password` — a user-chosen secret, treated as the main source of entropy in the attribute set.

The exact choice of fields is not security-critical at the protocol level: only their combined entropy and the user’s ability to reproduce them later are. The five fields above were chosen as a representative set for evaluation purposes; a deployment could substitute or extend them without changing any of the cryptographic constructions in this work. What matters is that the user can reliably re-enter the same values on every operation, since any change yields a different UID and therefore a disjoint credential set.

4.1.3.2 User interface

Attribute entry happens through a local GUI implemented with Tkinter, running on the same host as the Emulator process. When the Emulator receives a CTAP `authenticatorMakeCredential` or `authenticatorGetAssertion` command, it suspends the CTAP handler and emits an event to the GUI, which renders a request panel showing the relying party identifier and (where available) the user account name. The GUI invites the user to approve or reject the request; on approval, the attribute-entry fields become editable and the user supplies the five values. A pair of preset buttons populates common sets for testing.

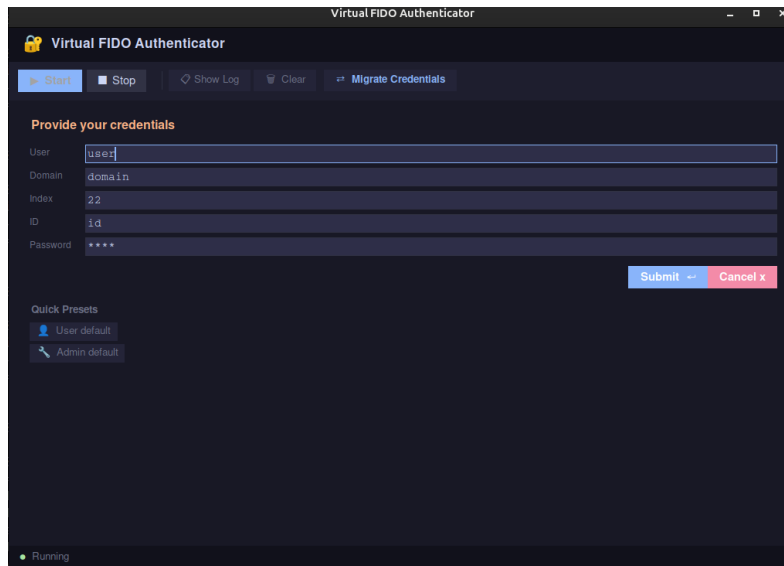


Figure 4.1: The Emulator’s attribute-collection panel during a registration request. The header shows the relying party and the user account; the five entry fields collect the user attributes; preset buttons populate common values for testing.

Only the concatenated attribute string crosses from the GUI back into the Emulator process: the GUI itself performs no cryptographic operation and holds no long-term state. Once the user submits, the fields are joined with a delimiter (‘|’) into a single byte string and handed to the CTAP handler, which proceeds with the UID derivation described below. Rejection causes the CTAP handler to return `CTAP2_ERR_OPERATION_DENIED` to the client, mirroring the behaviour of a hardware authenticator whose user-presence test was declined.

Resident-credential disambiguation. The GUI carries a second responsibility specific to discoverable-credential assertions. When the relying party does not supply a `Credential_ID`, the SGX host’s enclave returns *all* of the user’s credentials registered with that relying party in a single response (Section 4.3.2.5, Flow B). The Emulator acts as the disambiguator. After the enclave returns its array of matching credentials, the GUI presents the user with a chooser listing each candidate by its `RP_username` field. The user selects exactly one, and the Emulator returns only that assertion to the client. The other assertions are discarded without leaving the Emulator process.

The enclave job is to produce every assertion that the request authorizes, and to leave the choice of which one to use to other trusted component. The Emulator, trusted by the user, runs

on their own machine, under their own session, is the natural place to ask the question “which account do you want to sign in with?”.

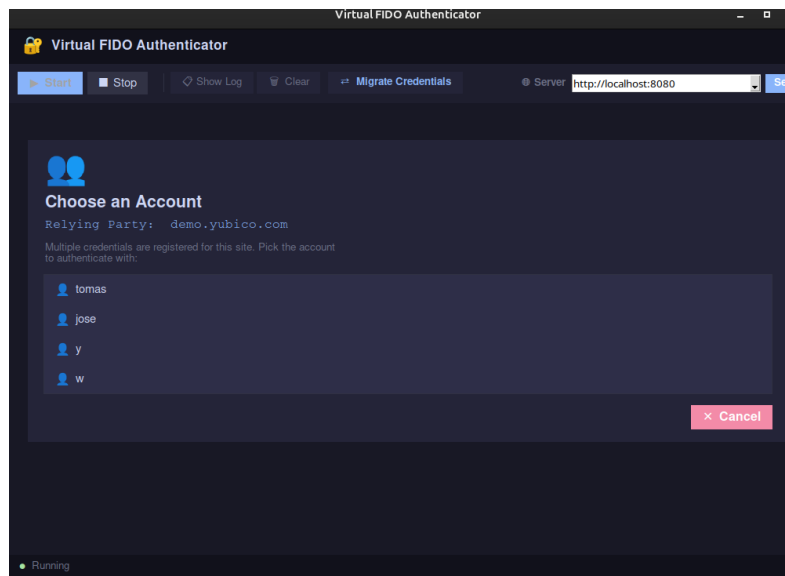


Figure 4.2: In an authentication with resident credentials, after the attribute-collection requests the user’s choice on which user to authenticate.

Migration control. The GUI is also the entry point for credential migration. Unlike registration and assertion, which are initiated by a WebAuthn client through the CTAP transport, migration has no CTAP origin: it is a user-driven operation. The user opens a migration panel in the GUI, provides the addresses of the source and destination SGX hosts, and supplies the same attribute values used at registration so that the Emulator can derive the appropriate UID. The Emulator then drives the three-message protocol of Section 3.5.2 on the user’s behalf, talking to each enclave over the per-request channel described in Section 4.2.

The user can select which destination host receives the imported credentials, allowing a single source to fan out credentials to multiple verifiable destinations across separate migration sessions.

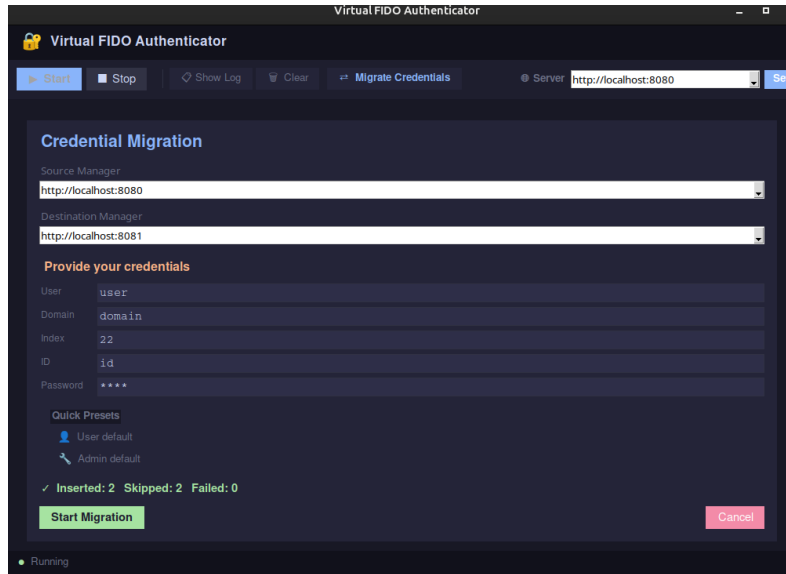


Figure 4.3: The migration panel from two services and enclaves located on the same host but in different ports. The two skipped credentials were credentials already migrated and the two inserted, credentials that were added after that previous migration.

4.1.3.3 UID derivation

The UID is computed as the SHA-256 hash of the concatenated attribute string:

$$\text{UID} = \text{SHA-256}(\text{User} \parallel \text{"|"} \parallel \text{Domain} \parallel \text{"|"} \parallel \text{Index} \parallel \text{"|"} \parallel \text{ID} \parallel \text{"|"} \parallel \text{Password})$$

The resulting 32-byte value is what the Emulator passes to the SGX host alongside each registration or authentication request, and is the input the enclave uses for all subsequent MAC and key-derivation operations described in Sections 3.3 and 3.4.

This construction is deliberately simple. It does not apply a slow KDF or per-user salt, which means that an attacker who obtains both (i) the database (yielding MAC values under IK) and (ii) the ability to invoke the enclave can mount an online guessing attack against weak attribute combinations, as discussed in Section 3.2. The choice to keep the derivation simple — rather than, for example, running PBKDF2 or Argon2 inside the enclave — reflects the design’s reliance on the enclave-bound nature of verification: brute-forcing the UID requires interaction with the enclave on every guess, which in a production environment will be rate-limited. Strengthening this against high-throughput online attacks would be a natural extension, and is discussed under future work.

4.1.3.4 Volatility and design implications

The attributes are volatile in two senses. First, they exist on the user’s host only for the duration of a single CTAP operation: they are zeroed from the GUI fields after submission, never written to persistent storage, and never leave the user’s machine in plaintext beyond what the enclave receives over the encrypted request channel. Second, the choice of which attributes to collect is itself not fixed: nothing in the design or in the enclave’s processing depends on the names of the fields or on having exactly five of them. The current set is a working choice for

evaluation; a future iteration could replace the entire input scheme, without altering any of the cryptography described elsewhere in the chapter.

In particular, the same UID-aggregation property described in Section 3.1 — a user can hold multiple disjoint credential sets by varying the attributes — carries through unchanged: each distinct attribute combination yields a different UID and therefore an independent set of credentials.

4.2 Communication channel

All traffic between the Authenticator Emulator and the enclave is wrapped in an encrypted channel established by the Emulator on every request. This section describes how each request bootstraps its channel, how the resulting wire format is structured, and what the channel does and does not protect.

4.2.1 Per-request handshake

The channel is not a long-lived session. Each call from the Emulator to the SGX host — whether it is fetching a quote at `INIT`, getting the authenticator information, registering a credential, requesting an assertion, or driving the migration protocol — carries its own freshly negotiated session key. The handshake is an ECIES-style construction [53] that uses the enclave’s attested identity public key as the static recipient key and a per-request ephemeral keypair on the Emulator side. The exact steps performed by the Emulator before each request are:

1. Generate a fresh ephemeral P-256 keypair, $(e_{\text{priv}}, e_{\text{pub}})$.
2. Compute the shared secret $s = \text{ECDH}(e_{\text{priv}}, P_{\text{enclave}})$, where P_{enclave} is the enclave’s identity public key obtained and verified during the `INIT` attestation (Section 4.1.2.3).
3. Derive a 256-bit transport key from the shared secret via HKDF-SHA256 [54]: HKDF-Extract with an all-zero salt, followed by HKDF-Expand with the domain-separation label “create_session”. The first 128 bits of the output are used as the AES key for the handshake encryption; the remaining bits are unused in the current scheme.
4. Generate a fresh 16-byte AES symmetric session key K_{sess} and a 12-byte IV. K_{sess} is the key under which both the request body and the corresponding response body are encrypted; IV_{body} is used for the request body, and the response uses an IV derived from it (Section 4.2.2).
5. Build a small *handshake payload* that wraps K_{sess} and the IV; encrypt this payload under the HKDF-derived transport key with AES-GCM, using a separate IV. The result is sent in an HTTP header, along with the e_{pub} and the IV.
6. Encrypt the actual request body — a JSON map of the operation’s parameters — under K_{sess} with AES-GCM, using the body IV from step 4. Place the ciphertext in the HTTP request body.

The enclave reverses the construction on receipt: it derives the same transport key from its own static private key and the received e_{pub} , unwraps the handshake payload to recover K_{sess} and the body IV, and decrypts the request body. Its response is encrypted under the same K_{sess} , with a derived IV, and returned alongside no additional handshake material, since the Emulator already holds the session key.

4.2.1.1 Why per-request rather than per-session

A single long-lived session key would be cheaper, but the per-request handshake has two design benefits worth highlighting. First, there is no session state to maintain across requests, either at the Emulator or at the enclave: the enclave's only persistent secret on the channel side is its IK-derived private key, which never changes, and the Emulator's only persistent material is the attested enclave public key. Second, every request has independent forward secrecy on the Emulator side: the ephemeral e_{priv} exists only for the duration of one HTTP exchange and is dropped immediately. A future compromise of the Emulator's process memory does not retroactively decrypt past requests.

The cost is one extra ECDH per request and a slightly larger HTTP header, both of which are negligible compared to the operations the enclave performs (database lookup, AEAD decryption, ECDSA signing).

4.2.2 Wire format

Each request from the Emulator to the SGX host is a single HTTP POST, in most operations, or a GET, for attestation and authenticator information, with two pieces of cryptographic content: the handshake material in an HTTP header, and the encrypted body in the JSON payload.

4.2.2.1 Handshake header

The handshake material is carried in a custom HTTP header, `X-Auth-Payload`, whose value is the base64 encoding of the following byte sequence:

$$e_{\text{pub}} \parallel \text{IV}_{\text{hs}} \parallel \text{ct}_{\text{hs}}$$

where e_{pub} is the Emulator's ephemeral public key on P-256 in raw uncompressed form (64 bytes: g_x followed by g_y in big-endian, with the `0x04` indicator byte of X9.62 encoding stripped), IV_{hs} is the AES-GCM IV for the handshake encryption, and ct_{hs} is the AES-GCM ciphertext (with appended authentication tag) of a small JSON object containing the base64-encoded K_{sess} and body IV:

```
{ "iv": "<base64 12 bytes>", "symmetric_key": "<base64 16 bytes>" }
```

The construction therefore involves two distinct encryptions, each with its own key and IV: the *handshake*, keyed by the HKDF-derived transport key and using IV_{hs} , protects the JSON object above; the *body*, keyed by the random K_{sess} recovered from that JSON and using the body IV, protects the request body described next.

4.2.2.2 Request body

The request body is JSON of the form:

```
{ "encrypted": "<base64 ciphertext + tag>" }
```

where the ciphertext is the AES-GCM encryption of the operation's parameters (themselves a JSON map) under K_{sess} , using the body IV carried in the handshake header. No additional authenticated data is bound into the body AEAD, since the integrity of the handshake header is independently authenticated by its own AEAD tag.

4.2.2.3 Response body

Responses use the same body format, a JSON object with a single encrypted field, but no handshake header, since the Emulator already holds K_{sess} . The enclave encrypts the response under K_{sess} , but with a distinct IV derived from the request IV as $IV_{\text{resp}} = \text{SHA-256}(IV_{\text{body}})$ truncated to its first 12 bytes. Deriving a fresh response IV rather than reusing the request IV is essential: AES-GCM loses both confidentiality and integrity if a single (K_{sess}, IV) pair encrypts two distinct plaintexts, and the request and response bodies are distinct plaintexts. Since IV_{resp} differs from IV_{body} , the nonce-uniqueness requirement [55] holds for both encryptions. The derivation carries no bytes on the wire, as the Emulator recomputes IV_{resp} locally from the IV it generated.

4.2.2.4 What the channel protects, and what it does not

The construction defeats a network attacker positioned between the Emulator and the SGX host: the handshake header is unintelligible without the enclave's static private key, and the body is unintelligible without K_{sess} . A man-in-the-middle attempting to substitute their own public key in the handshake would have to produce a quote attesting that key under the expected `MRENCLAVE`, which is exactly what the `INIT` attestation step rules out (Section 4.1.2.3).

The channel does not, on its own, provide authentication of the Emulator to the enclave: any caller who knows the enclave's attested public key can construct a valid handshake. This is intentional in the present design, user-presence is established by the user-attribute mechanism of Section 4.1.3, and request authorisation rests on possession of the correct UID rather than on any property of the channel.

4.3 SGX Host

4.3.1 HTTP Service

The HTTP service is the untrusted host process that wraps the SGX enclave and exposes its operations as a network API.

The service is implemented in C++17 using Boost.Beast for HTTP and Boost.Asio for asynchronous I/O [56], with `nlohmann/json` [57] for JSON handling. These choices keep the host code self-contained: there is no external web framework or runtime to trust beyond standard libraries and the SGX SDK itself.

4.3.1.1 Architecture

The service follows the standard Boost.Asio acceptor pattern. A `Listener` object binds to a TCP endpoint (port 8080 in the current deployment, on `0.0.0.0`) and asynchronously accepts incoming connections. Each accepted connection is handed off to a `Session` object, which reads one HTTP request, dispatches it to a single `handle_request` function for routing, and writes back the response.

At startup, the service performs three steps in order: it loads the signed enclave from `enclave.signed.so` into an SGX EID, opens the credential database, and starts accepting connections. The enclave's lifetime is tied to the service's lifetime; on shutdown the database is closed and the enclave destroyed.

4.3.1.2 Endpoint surface

The service exposes six application endpoints, one for each operation defined in Chapter 3. With the exception of `/atteststate`, each endpoint is a thin wrapper around a corresponding ecall: it extracts the handshake material from the `X-Auth-Payload` HTTP header, parses the request body's `encrypted` JSON field, calls into the enclave with the two opaque byte arrays, and returns the enclave's encrypted response wrapped in a JSON envelope. The six endpoints, with their HTTP verbs and the enclave calls they invoke, are:

- **GET /atteststate** — returns a DCAP quote attesting the enclave's identity public key. Used by the Emulator at `INIT` (Section 4.1.2.3). This is the only endpoint that does not forward to an ecall directly; its host-side handling is described below.
- **GET /getauthinfo** — invokes `ecall_auth_info` and returns the encrypted `authenticatorGetInfo` response (AAGUID, supported algorithms, and capability options).
- **POST /makecredential** — invokes `ecall_make_credential` to register a new FIDO2 credential under a UID, returning the new public key and `Credential_ID`.
- **POST /getassertion** — invokes `ecall_get_assertion` to produce an assertion for an existing credential, or for all matching credentials in the resident-credential flow.
- **POST /export** — invokes `ecall_export`, the source-side step of the migration protocol: enumerates the user's credentials and builds the transfer envelope (Section 3.5.2).
- **POST /import** — invokes `ecall_import`, the destination-side step of the migration protocol: verifies the source's quote, re-indexes the imported credentials under the local IK, and inserts them into the database.

The response, when successful, is symmetrically a JSON object `{"encrypted": "<base64>"}`. On error, the service returns a JSON object with an `error` field and an HTTP 4xx or 5xx status code, without revealing internal state beyond a numeric ecall return code.

4.3.1.3 Host-side report generation

The `/atteststate` endpoint differs from the others in that it does not just forward an opaque payload to the enclave.

The host runs the two-step report-then-quote production described in Section 4.3.2.2: it calls `ecall_get_report` to obtain a report bound to the enclave’s identity public key, then `sgx_qe_get_quote` to turn that report into a DCAP quote.

The `ecall_get_report` ecall is the only path through which the enclave’s stable identity public key becomes visible outside the enclave. It computes the enclave’s identity keypair by calling the helper `get_enclave_keypair` described below, copies the public key’s 64 bytes into the report’s `report_data` field and calls `sgx_create_report` to produce the local-attestation report targeted at the QE. The private key is zeroed before the ecall returns; only the public half ends up in the report. The report itself carries the enclave’s identity (`MRENCLAVE`, `MRSIGNER`, attributes, ISV product ID, ISV SVN), the 64-byte `report_data` field that the enclave just populated, and a MAC under the platform’s report key that the QE will verify in the next step.

The `get_enclave_keypair` requests the SGX sealing key via `sgx_get_key` with policy `SGX_KEYPOLICY_MRENCLAVE`, hashes it with SHA-256 to obtain a 32-byte seed, copies the seed into a P-256 private-key scalar, and calls `sgx_ecc256_calculate_pub_from_priv` to derive the corresponding public key. Because the sealing key is bound to the platform’s CPU and to the enclave’s `MRENCLAVE` measurement, the resulting keypair is unique to this enclave on this physical processor. Any other enclave, including a rebuilt version of this one with even one byte of code or configuration changed, would derive a different keypair.

4.3.1.4 What the host can and cannot do

Because the host is outside the TCB, the design assumes it may be compromised. The endpoint surface above is structured so that a malicious or buggy host cannot break confidentiality or unlinkability:

- Every credential operation crosses the enclave boundary as opaque ciphertext. The host has no decryption key, since each session key is established by ECDH against the enclave’s identity private key (which exists only inside the enclave).
- Database operations are issued as ocalls from inside the enclave, and the host implements them mechanically (SQLite read/write through the `ocall_db_*` family). All records crossing back to the host are AEAD-encrypted under per-row keys (Section 3.3.2); the host’s view of the database is purely opaque.
- Routing errors, replayed requests, or dropped responses can deny service but cannot extract credential material, since the security properties of the channel and storage layers do not depend on the host’s behavior beyond delivery (Section 3.2).

The host does carry one privilege the design has to accept: it can refuse to serve requests, return stale ciphertext from the database, or drop ocall responses. These attacks are availability problems, treated as out of scope by the threat model.

4.3.2 Enclave

4.3.2.1 Build and Trusted Computing Base

This subsection describes how the enclave is built, what code is measured by its identity (MRENCLAVE), and where the line between trusted and untrusted code falls in the implementation. Together these define the concrete Trusted Computing Base that the security argument of Section 3.6 relies on.

Toolchain. The enclave is written in C and C++ (C++11), built with GCC against the Intel SGX SDK version 2.28.100.1 and linked against the SDK's in-enclave runtime libraries (`sgx_trts`, `sgx_tstdc`, `sgx_tcxx`, `sgx_tcrypto`, `sgx_tservice`). The DCAP runtime libraries (`libsgx_dcap_ql` and `libsgx_dcap_quoteverify`, both version 1.25.100.1) are linked against the untrusted host process, not the enclave. Build orchestration is driven by CMake, with the enclave compiled as a position-independent shared object and then signed in a separate build step. Development and testing targeted Ubuntu 22.04 on an Intel SGX-capable processor.

The enclave interface (EDL). The boundary between the enclave and the untrusted host is defined by a single Enclave Definition Language file, processed at build time by the SGX `edger8r` tool to generate marshalling code for both sides. The EDL declares each function that can cross the boundary together with the direction (`[in]` / `[out]`) and the size of every buffer parameter. The SDK's generated marshalling layer copies parameter buffers across the enclave boundary at the declared sizes, so the host cannot tamper with in-flight data without the enclave noticing.

A consequence of this marshalling model is that each `ecall` must declare a compile-time-constant maximum size for any `[out]` buffer the enclave writes back to the host: these maxima are baked into the EDL and are part of the enclave's measurement. They are operation-specific and cap how large each response can be, discussed alongside each operation in the subsections that follow.

The boundary is deliberately narrow:

- **Ecalls** (host calls into the enclave): `ecall_get_report`, `ecall_auth_info`, `ecall_make_credential`, `ecall_get_assertion`, `ecall_export`, `ecall_import`. Each corresponds to one HTTP endpoint in Section 4.3.1.
- **Ocalls** (enclave calls out to the host): the database family (`ocall_db_open`, `ocall_db_insert`, `ocall_db_lookup`, `ocall_db_list_count`, `ocall_db_list_at`, `ocall_db_delete`, `ocall_db_close`); the quote-production helpers (`ocall_get_qe_target_info`, `ocall_get_quote`); and a logging hook (`ocall_print`) used during development.

No data ever crosses these boundaries in cleartext beyond what the enclave has chosen to expose: encrypted blobs and opaque MAC values for the database `ocalls`, an SGX report for the quote `ocalls`, and human-readable strings for the development log.

Runtime configuration. The enclave’s runtime parameters are specified in an XML configuration file consumed by the SGX signer. The relevant settings for this implementation are:

- **Concurrency.** Ten Thread Control Structures (`TCSNum = 10`) are provisioned, allowing up to ten concurrent ecalls. This matches the per-request model of the HTTP service, where each incoming HTTP request results in one ecall, and is consistent with the thread-safe configuration of the SQLite layer in the host (`SQLITE_THREADSafe=1`).
- **Memory layout.** The enclave is provisioned with a 256 KiB stack per thread and a 1 MiB heap. These limits are sized for the largest operation the enclave handles, which is migration export: a temporary serialization buffer must hold all of a user’s credentials before they are encrypted under the payload key.
- **Debug flag.** The enclave is currently built with `DisableDebug = 0`, meaning it is loadable in SGX debug mode and its memory can be inspected by an attached debugger. This is appropriate for development but is incompatible with any security claim: a production deployment must set `DisableDebug = 1`, which changes the enclave’s measurement and prevents debug-mode loading.
- **Product identifiers.** The product ID (`ProdID`) and security version number (`ISVSVN`) are both set to zero. These are development placeholders; a production deployment would set meaningful values, so that the SDK’s sealing-key derivation can distinguish versions of the same enclave product when applying upgrade policies.

What is measured. The enclave’s `MRENCLAVE` value is the SHA-256 measurement of every page that the SGX loader brings into the enclave at creation time: the enclave’s compiled code, its read-only data, its initial heap and stack layout, and the values from the configuration file above. Concretely, the measured content includes:

- The enclave’s own source — the registration, assertion, migration, and storage-mediation logic, the secure-channel handshake, and the quote-production helpers, organized across three translation units (`enclave_auth.cpp`, `enclave_migration.cpp`, `enclave_utils.cpp`).
- The trusted SDK runtime libraries listed above, which provide the in-enclave C/C++ standard library, the TCB-internal crypto primitives, and the SGX-specific service interface.
- The hard-coded Intel SGX Provisioning Certification Root CA, embedded as a constant in the enclave’s code at build time (Section 3.5.5).

Anything outside this measurement — the host process, its dependencies (Boost.Beast, nlohmann/json, SQLite), the operating system, and the DCAP runtime libraries — is not part of the TCB.

Signing and identity. The compiled enclave is signed in a separate build step using the SDK's `sgx_sign` tool and an RSA-3072 key pair. The signing key determines the `MRSIGNER` value carried in every quote. In the current implementation the signing key is generated for development purposes only and held alongside the source tree; a production deployment would use a key held under access control (for instance, in a hardware security module) and would publish the corresponding public key so that clients can pin trust to `MRSIGNER` as well as to specific `MRENCLAVE` measurements.

The mutual attestation protocol in Section 3.5.3 compares `MRENCLAVE` rather than `MRSIGNER`, requiring that the source and destination enclaves run *identical* code, not merely code from the same signer. This is the stronger of the two checks and is the appropriate choice for a credential-handling enclave, since it prevents a maliciously rebuilt enclave from accepting credentials migrated from a legitimate one.

External trust dependencies. Beyond the local TCB, the system depends on Intel's attestation infrastructure: the Provisioning Certification Service that signs PCK certificates, the TCB Info service (Section 4.1.2.3), and the DCAP runtime that produces quotes. These are outside our control and are trusted transitively, as detailed in Section 3.2. A compromise of Intel's signing infrastructure would invalidate the attestation chain for every SGX deployment, including this one.

4.3.2.2 Quote production

Whenever the enclave needs to prove its identity to a remote party — to the Authenticator Emulator at `INIT`, or to a peer enclave during migration — it produces an SGX quote. Quote production is a two-step process that crosses the enclave boundary, since the data path requires both intra-platform enclave attestation primitives (which only the enclave can invoke) and Intel's Quoting Enclave (which only the host can reach).

Reports and quotes. The SGX architecture distinguishes *reports* from *quotes* [52]:

- A **report** is a 432-byte structure produced by an enclave through the `sgx_create_report` primitive. It contains the enclave's identity (`MRENCLAVE`, `MRSIGNER`, `attributes`, ISV product ID and version) and a 64-byte `report_data` field set by the enclave itself. The report is signed using a key derived inside the processor and is only verifiable by another enclave on the same platform: it is the SGX local-attestation primitive.
- A **quote** is a remotely verifiable artifact produced by Intel's *Quoting Enclave* (QE), a special-purpose SGX enclave shipped as part of the DCAP runtime. The QE takes a report whose target is the QE itself, verifies it locally (since both run on the same platform), and re-signs it under the platform's ECDSA Attestation Key. The result is a quote: the same identity information bound to a signature whose chain of trust leads back, via the PCK certificate, to the Intel SGX Root CA.

Production flow. The implementation runs quote production as a sequence of four steps that span the host, the enclave, and Intel’s Quoting Enclave. The cryptographic structure of the resulting quote is described in detail in revision 1.3 of the DCAP ECDSA Quote Library specification [52]; we focus here on the operations relevant to the validation flow described in Section 4.1.2.3.

1. **Host obtains the QE’s target descriptor.** The host calls `sgx_qe_get_target_info` to retrieve an `sgx_target_info_t` structure identifying the QE on the local platform. This descriptor is needed in the next step because SGX local attestation is point-to-point: a report produced by one enclave can only be verified by the specific enclave named as its target.
2. **Enclave produces a local report.** As mentioned in Section 4.3.1.3, the host invokes `ecall_get_report` on our enclave, passing the QE target descriptor. Inside the `ecall`, the enclave fills the report’s 64-byte `report_data` field with the public key it wishes to attest — either its stable, IK-derived identity key (Section 3.1) or, in the source-side migration case, a per-migration ephemeral key (Section 3.5.3) — and calls `sgx_create_report` with the QE as the target. The result is a 432-byte `sgx_report_t` containing the enclave’s identity (`MRENCLAVE`, `MRSIGNER`, attributes, ISV product ID and version), the chosen `report_data`, and a MAC computed under a symmetric report key derived inside the processor. By construction, only the target enclave (the QE) on the same physical platform can verify this MAC.
3. **QE converts the report into a quote.** The host hands the report to the DCAP runtime via `sgx_qe_get_quote`, which delivers it into Intel’s Quoting Enclave. The QE performs three operations:
 - It verifies the local report’s MAC under its own copy of the report key, confirming that the report originated from a co-resident enclave on the same platform.
 - It reads the report’s identity fields and `report_data`, copies them into the user-enclave portion of the quote, and signs that portion with the QE’s long-lived ECDSA P-256 *Attestation Key* (AK). This signature is the *ISV Enclave Report Signature* of the quote.
 - It attaches its pre-prepared certification material: the AK’s public key, a *QE Report* that binds the AK to the QE’s identity via the formula $\text{report_data}_{\text{QE}} = \text{SHA-256}(\text{AK} \parallel \text{QE_Auth_Data})$, a signature over that QE Report produced under the platform’s Provisioning Certification Key (PCK), and the PCK certificate chain rooted at the Intel SGX Provisioning Certification Root CA.

The QE Report and its PCK signature are generated once per platform provisioning and cached, since the AK is long-lived; they are reused across all quotes produced by the same QE. Only the AK signature over the user-enclave portion is recomputed per quote.

4. **Host returns the quote.** The host either returns the quote to the caller (over the `/attest` endpoint) or includes it in a migration response. No further cryptographic processing occurs at this stage; the host is a passive conveyor.

A point worth noting: quote production is entirely local. Once the platform has been provisioned with a PCK certificate signed by Intel (a one-time step requiring HTTPS contact with Intel’s Provisioning Service, typically mediated by a local caching service), no further network interaction with Intel is required to produce quotes. The PCK signature on the QE Report and the AK signature on the user enclave’s report are both produced inside the platform’s enclaves, using keys derived within the processor. Intel’s online infrastructure is contacted only by the verifier, to check the current revocation status of the PCK certificate and the current TCB Info (Section 4.1.2.3).

In our deployment, the DCAP runtime is configured to contact Intel’s Provisioning Certification Service directly, rather than through an intermediate local caching service. The configuration file `/etc/sgx_default_qcnl.conf` sets `pccs_url` to Intel’s PCS endpoint⁵ and instructs the runtime to cache both PCK certificates and verification collateral on disk for 168 hours (seven days) before re-fetching them. A production deployment would typically interpose a local PCCS (Provisioning Certification Caching Service) between the SGX host and Intel, to insulate the host from transient unavailability of Intel’s infrastructure and to centralize cert management across a fleet; for the development setup used in this work, the direct configuration is simpler and equivalent in effect.

The DCAP runtime, the QE, and the host are all untrusted from this work’s perspective, the QE itself is trusted by Intel, not by us, and we rely on its signature only because the Emulator independently verifies the certificate chain that establishes the QE’s legitimacy. Misbehavior at any layers would result in a quote whose verification fails on the recipient’s side, rather than in a successfully-forged attestation.

Quote format and version. The implementation parses DCAP ECDSA quotes against the format version 3 layout defined in revision 1.3 of the Intel DCAP ECDSA Quote Library specification [52]. All offset constants in the enclave-side and Emulator-side parsing code (e.g., `report_data` at offset $48 + 320$ from the start of the quote) come from this version. The DCAP runtime in use (version 1.25, see Section 4.3.2.1) continues to emit format-version-3 quotes by default for SGX, so the parser’s assumptions remain valid against the current runtime. Newer quote format versions (4 and 5) have been introduced to additionally carry Intel TDX (Trust Domain Extensions) attestation; migrating to them would require updating the parsing offsets throughout the codebase but would not change the cryptographic or trust properties of the construction.

4.3.2.3 Authenticator Information

The CTAP `authenticatorGetInfo` command, which clients use to discover an authenticator’s capabilities, is implemented inside the enclave and exposed to the Emulator through `ecall_auth_info`. The enclave returns a fixed CBOR-encoded structure describing the authenticator’s supported versions, options, and parameters; the Emulator forwards this structure unchanged to the calling CTAP client as the body of its CBOR response.

⁵<https://api.trustedservices.intel.com/sgx/certification/v4/>

Although the contents of this response are static and could in principle be hardcoded into the Emulator instead, locating them inside the enclave keeps a single source of truth for what the authenticator claims to be. Changes to the advertised capabilities — adding a new supported algorithm, for example, or toggling a CTAP option — happen at the enclave level, alongside the cryptographic code that actually realizes those capabilities. The cost is that any such change alters the enclave's MRENCLAVE, requiring clients that pin trust to a specific measurement (including the Emulator itself, Section 4.1.2.3) to be re-pointed at the new value.

The structure currently returned is:

```
{
  1:  ["FIDO_2_2", "FIDO_2_1", "FIDO_2_0"],
  2:  ["credProtect", "hmac-secret", "largeBlob"],
  3:  "00000000-0000-0000-0000-000000000000",
  4:  { "rk": true, "up": true, "uv": true, "plat": false },
  5:  1200,
  8:  256,
  9:  ["usb"],
  10: [ { "alg": -7, "type": "public-key" } ]
}
```

The field meanings, with the rationale for each value, are as follows:

- **(1) Supported versions** — the authenticator declares support for CTAP 2.0, 2.1, and 2.2. The Emulator is built against the CTAP 2.2 specification but advertises the older versions for compatibility with clients that have not yet adopted 2.2.
- **(2) Supported extensions** — `credProtect`, `hmac-secret`, and `largeBlob` are declared but not exercised by the credential operations described in this chapter. They are advertised to satisfy clients that probe for them; the credential storage and authentication paths use only the core CTAP commands.
- **(3) AAGUID** — the all-zero value is a placeholder. In a deployed authenticator the AAGUID identifies the device model, allowing relying parties to track which makes and models registered which credentials. For this work, leaving it zero has no functional consequence: the security argument does not depend on AAGUID-based identification.
- **(4) Options** — the most consequential field for this work:
 - `rk = true` advertises support for resident (discoverable) credentials, which is required for the resident-credential authentication flow described in Section 4.3.2.5.
 - `up = true` advertises user presence: the authenticator can prove a user was physically present at the time of an operation. In our design, user presence is established by the user actively supplying their attributes through the Emulator's GUI (Section 4.1.3).
 - `uv = true` advertises built-in user verification. As discussed in Section 3.1, this is the design's core substitute for PIN or biometric verification: the enclave's UID-based key derivation verifies the user cryptographically.

- `plat = false` marks the authenticator as roaming rather than platform-bound. The UHID transport (Section 4.1.1) presents the device as a USB-attached token rather than as an integrated platform component, which is what FIDO clients expect for the USB transport.

The absence of `clientPin` from the options map signals that the authenticator does not use a PIN, consistent with the UID-based verification scheme.

- **(5) `maxMsgSize`** — 1200 bytes, the maximum CTAP message the authenticator will accept or emit. This is the standard size for hardware tokens and is comfortably above the size of any operation in our design.
- **(8) `maxCredentialIdLength`** — 256 bytes, the upper bound on the length of any `Credential_ID` the authenticator will emit. The `Credential_ID`s actually produced by the enclave are 32 bytes (Section 3.3), well within this limit.
- **(9) `Transports`** — the authenticator advertises USB, matching the UHID-based transport.
- **(10) `Supported algorithms`** — `alg = -7` is COSE ES256 (ECDSA with P-256 and SHA-256), matching the signature algorithm used by the enclave for all credential operations (Section 4.3.2.5).

The enclave-side handler does no further work: it decrypts the per-session handshake header to recover the session key (Section 4.2.1), encrypts the structure above under that key with AES-GCM, and returns the ciphertext to the host for delivery to the Emulator.

4.3.2.4 Registration

Moving to the Registration — the implementation of CTAP `authenticatorMakeCredential`, is realized by `ecall_make_credential`. The `ecall` receives two encrypted inputs (the per-session handshake header described in Section 4.2.1, and the request body containing the operation parameters) and returns an encrypted response containing the new credential’s public key and `Credential_ID`.

The body, once decrypted under the session key, is a small JSON object with three fields:

- `rp_id` — the relying party identifier (a string).
- `user_id` — the UID as a hex-encoded 32-byte hash (the SHA-256 of the user’s attributes, computed by the Emulator as described in Section 4.1.3).
- `rp_user_id` — the user’s identifier within the relying party (a string, supplied by the calling WebAuthn client).
- `rp_username` — the human-readable display label that the Emulator will show during resident-credential disambiguation (Section 4.3.2.5). Unlike `rp_user_id`, this field is consumed only inside the trust boundary; the relying party never sees it.

The enclave validates that all four fields are present and non-empty, then proceeds through six logical phases. Apart from the I/O around the encrypted envelopes, every phase realizes a construction already specified in Section 3.3; the description below names the SDK primitives used to implement each one.

Step 1 — recover IK. The enclave requests its sealing key from the processor via `sgx_get_key` with policy `SGX_KEYPOLICY_MRENCLAVE`, yielding the 128-bit Identity Key. The `MRENCLAVE` binding ensures that any other enclave on the platform — including any variant of this enclave with even one bit of code or configuration changed — would derive a different IK and therefore be unable to read this enclave’s stored credentials. The retrieved IK is held in a stack-allocated buffer that is explicitly zeroed before the `ecall` returns.

Step 2 — derive the Credential_ID. The enclave concatenates the 32-byte UID with the RP identifier and the in-RP user identifier into a contiguous buffer and computes `Credential_ID` as defined in Section 3.3.1 using `sgx_hmac256_init / update / final`. The 32-byte result is the credential’s identifier, which will be returned to the relying party at the end of the operation.

Step 3 — derive the storage MACs. The enclave computes the two unlinkable storage indexes specified in Section 3.3.2: `mac_cred` and `mac_uid`, also using the HMAC and SHA256.

Step 4 — derive the credential key pair. The enclave derives the credential’s private key from IK and the `Credential_ID` via a two-step HMAC construction equivalent to HKDF [54] with a single output block:

$$\begin{aligned} \text{PRK} &= \text{HMAC-SHA256}_{\text{IK}}(\text{"key_derive"} \parallel \text{Credential_ID}) \\ \text{OKM} &= \text{HMAC-SHA256}_{\text{PRK}}(\text{"cred_keypair"} \parallel 0x01) \end{aligned}$$

The 32-byte OKM is interpreted as a P-256 private scalar, copied into an `sgx_ec256_private_t` structure. The corresponding public key is computed by the SDK’s `sgx_ecc256_calculate_pub_from_priv`. The private key is held in the enclave only long enough to derive the public key from it and is then zeroed; for credentials stored in derived form, the private key is never persisted and will be re-derived on demand at each assertion (Section 4.3.2.5)

Step 5 — build the persistent record. The enclave constructs the plaintext record P defined in Section 3.3.2:

$$P = 0x01 \parallel \text{Credential_ID} \parallel \text{rp_len} \parallel \text{RP} \parallel \text{rpu_len} \parallel \text{RP_user} \parallel \text{rpuname_len} \parallel \text{RP_username}$$

where `0x01` is the type marker for a derived credential (the migrated variant uses `0x02` and additionally carries the private key;). The two length prefixes are little-endian 16-bit integers, allowing the record to be parsed back unambiguously at assertion or migration time. The

Credential_ID is included in the plaintext, despite already being implicit in mac_cred, so that a future migration to a peer enclave can re-derive mac_cred under a different IK (Section 3.5.1).

The enclave then derives the per-row encryption key and IV through two further HMAC invocations under IK, with the distinct domain-separation labels "cred_enc" and "cred_iv" as described in Section 3.3.2, and encrypts *P* under those parameters with AES-128-GCM (`sgx_rijndael128GCM_encrypt`). The MAC values mac_cred and mac_uid are passed as additional authenticated data, binding the ciphertext to the index pair under which it will be stored. The resulting blob (16-byte tag followed by the ciphertext) is the value that will be persisted.

Step 6 — insert. The enclave issues an ocall to `ocall_db_insert` with the triple (mac_cred, mac_uid, blob). Because mac_cred is the database's primary key, a collision is detected and reported back to the enclave as a distinct status code. The enclave treats a collision as an error and aborts the operation, returning failure to the caller: a repeated registration with the same (UID, RP, RP_user) triple would deterministically produce the same Credential_ID and therefore the same mac_cred, so a collision means either a duplicate request or a deliberate replay.

Note on duplicate-detection ordering. The collision check happens only at Step 6 (insertion), after Steps 1–5 have already executed the full HMAC and key-derivation pipeline. A repeated registration with the same (UID, RP, RP_user) performs all the cryptographic work before the duplicate is detected and the operation is rejected. In normal use this is harmless, since a legitimate client does not re-register an existing credential. A misbehaving or compromised client that issues many such repeated registrations would consume enclave CPU on every attempt while producing no database writes. An early-exit optimisation that consults `ocall_db_lookup` on mac_cred after Step 3 would close this gap by skipping Steps 4 and 5 when the credential already exists, at the cost of an extra ocall on every registration.

Response. On success, the enclave constructs a JSON response containing the credential's public key (as a 128-character hex string of the concatenated $g_x \parallel g_y$ coordinates in big-endian), the Credential_ID (64 hex characters), the UID, and the RP identifier. This response is encrypted under the session key recovered from the handshake header in Step 1, and the resulting ciphertext-plus-tag is returned to the host. The Emulator receives this response and assembles it into the CTAP `authenticatorMakeCredential` response that the calling WebAuthn client expects — including the credential public key in COSE_Key format, the attested credential data, and the authData structure (Section 4.1.2).

Cleanup. The enclave's locally-allocated buffers — IK, the derived private key, the per-row encryption key and IV, the plaintext record, the response JSON — are explicitly zeroed before the ecall returns. The implementation uses a single `goto cleanup` chain rather than scattered cleanup blocks, which requires all locals to be declared at the top of the function: a small C++ ergonomic cost in exchange for guaranteed key zeroization on every error path.

4.3.2.5 Assertion

Assertion — the implementation of CTAP `authenticatorGetAssertion` — is realized by `ecall_get_assertion`. As described in Section 3.4, the operation has two distinct sub-flows depending on whether the relying party supplies a `Credential_ID`: a direct flow that retrieves and signs with a specific credential, and a discoverable-credential flow that enumerates all of a user’s credentials for a given RP and returns one assertion per match. Both sub-flows are implemented in the same `ecall` and share their cryptographic preliminaries.

Inputs. The `ecall` receives the per-request handshake header and an encrypted body whose plaintext, once decrypted under the session key, is a JSON object with five fields:

- `rp_id` — the relying party identifier.
- `user_id` — the UID as a hex-encoded 32-byte hash.
- `auth_data` — the WebAuthn `authData` structure assembled by the Emulator, hex-encoded. The implementation expects the minimal 37-byte `authData` (32-byte RP ID hash, 1-byte flags, 4-byte signature counter); attested-credential extensions and WebAuthn extensions are not exercised.
- `client_data_hash` — the SHA-256 hash of the client data JSON, hex-encoded.
- `cred_id` — a hex-encoded `Credential_ID`, present in the direct flow and an empty string in the discoverable-credential flow.

The data to be signed in either flow is the 69-byte concatenation `authData || clientDataHash`, per [47].

Preliminaries. On entry, the enclave recovers IK from the SGX sealing key (Section 4.3.2.4, Step 1) and computes `mac_uid` with HMAC-SHA256 from IK and UID. If a `Credential_ID` is present in the request, it additionally computes `mac_cred`, with HMAC-SHA256 from it and IK. These values are the same MACs used at insertion time, so the database can be queried directly under whichever indexes are needed. The `ecall` then branches on the presence of `cred_id`.

Flow A — assertion with a `Credential_ID`. The enclave issues a single `ocall_db_lookup` with `mac_cred` as the key. The lookup returns the row’s stored `mac_uid` alongside the AEAD blob. Before attempting decryption, the enclave compares the returned `mac_uid` against the one it locally computed from the request’s UID: a mismatch indicates that the host returned a row belonging to a different user, which would be caught by the AAD-bound AEAD anyway but is rejected early to surface a clearer error.

The enclave then derives the per-row key and IV from IK and `mac_cred`, using the same “`cred_enc`” and “`cred_iv`” labelled HMACs as at registration (Section 3.3.2), and decrypts the blob with `mac_cred || mac_uid` as the additional authenticated data.

The plaintext layout is, in order: a 1-byte type marker; the 32-byte `Credential_ID`; a 2-byte little-endian RP length followed by the RP identifier bytes; a 2-byte little-endian `RP_user` length

followed by the `RP_user` bytes; a 2-byte little-endian `RP_username` length followed by the `RP_username` bytes; and, only for migrated credentials, a 32-byte private key. The enclave inspects the type marker to decide how to obtain the private key:

- For a *derived* credential (0x01), the enclave re-derives the key pair from IK and the `Credential_ID` using the same HKDF construction as in registration (Section 4.3.2.4, Step 4). The private key was never persisted; it is regenerated on demand and discarded again at the end of the call.
- For a *migrated* credential (0x02), the enclave reads the private key directly from the plaintext.

With the private key in hand, the enclave signs `authData || clientDataHash` using ECDSA over P-256 via `sgx_ecdsa_sign`. The SDK returns the signature with its *r* and *s* components in the processor's native little-endian byte order; the enclave reverses each into the big-endian *r || s* layout expected by WebAuthn, hex-encodes them, and assembles the response.

The response JSON in this flow has the form:

```
{ "status": "ok", "sig_r": "...", "sig_s": "...", "cred_id": "..." }
```

Flow B — discoverable-credential assertion. When `cred_id` is empty, the enclave enters the discoverable-credential flow. It queries `ocall_db_list_count` for the number of records indexed under `mac_uid`. If the count is zero, the enclave returns a non-distinguishing error to the host without revealing whether the UID itself was unknown or whether the user simply has no credentials at any relying party (Section 3.4).

If the count is non-zero, the enclave iterates over the rows by repeatedly calling `ocall_db_list_at` with successive indices. For each returned row, it derives the per-row key and IV from IK and the row's `mac_cred` (returned alongside the blob), attempts AEAD decryption with the row's `mac_cred || mac_uid` as the AAD, and on success checks whether the embedded RP matches the request's `rp_id`. Rows whose decryption or RP-check fails are skipped silently and the iteration continues with the next index. This silent skipping ensures that a single corrupted or unrelated row does not abort the assertion, since the iteration may legitimately encounter rows for the same UID at other relying parties.

For each matching row, the enclave reconstructs the private key (re-derived or read from the blob, exactly as in Flow A), signs `authData || clientDataHash`, and appends a tuple of (signature, `Credential_ID`, `RP_user`, `RP_username`) to the response array. The two trailing fields serve different consumers: `RP_user` is the WebAuthn-defined user handle that the relying party expects in the assertion, since it did not specify a credential up front; `RP_username` is the human-readable display label that the Emulator uses to populate its disambiguation chooser (Section 4.1.3). Both travel together because both are needed before the assertion can be delivered: the Emulator picks one entry based on `RP_username`, then forwards the corresponding `RP_user` and signature to the relying party. If no rows match after the full iteration, the enclave returns the same non-distinguishing error as if the user had no credentials at all.

The response JSON in this flow has the form:

```

{
  "status": "ok",
  "assertions": [
    { "sig_r": "...",
      "sig_s": "...",
      "cred_id": "...",
      "user_rp": "...",
      "username": "..." },
    ...
  ]
}

```

Response size and credential cap. The output buffer is statically sized at 4 KiB, which sets an implicit cap on how many resident credentials for a single RP can be returned in one call. A typical per-credential entry in Flow B consumes on the order of 250–400 bytes after encryption (signature plus hex-encoded Credential_ID plus hex-encoded RP_user plus the RP_username display string plus JSON overhead), so the buffer accommodates around a dozen matches comfortably, well above the typical case of one or a few credentials per RP per user. The iteration explicitly checks the buffer fill level after each appended entry and breaks early if continuing would exceed the capacity, so a user with an unusually large number of credentials at the same RP would receive a truncated but still well-formed response. Pagination would be a straightforward extension and is not implemented in this work.

Cleanup. As in registration, the enclave’s locally-allocated buffers are explicitly zeroed before the `ecall` returns, through the same goto-based cleanup chain.

4.3.2.6 Export Credentials

The export operation is the source-side half of the migration protocol (Section 3.5.2). It receives the destination enclave’s quote, validates it, extracts the destination’s attested public key, and uses that key to produce an end-to-end encrypted transfer envelope containing all of the user’s credentials. The implementation is realized by `ecall_export`, which spans several discrete phases described below.

Quote validation. The first task on the source side is to validate the quote received from the destination. The same validation logic is reused by the destination during import (Section 4.3.2.7), so it is described once here.

The validation walks the chain of signatures introduced in Section 4.3.2.2, in reverse: starting from the trust anchor (the Intel SGX Root CA, hard-coded in the enclave’s text section as discussed in Section 3.5) and verifying each successive signature down to the quote’s ISV Enclave Report Signature. Concretely, the enclave performs six checks, all using the SDK’s `sgx_ecdsa_verify` primitive over ECDSA P-256.

1. **Root CA pin.** The quote arrives with its full PCK certificate chain attached as PEM-encoded data (PCK, intermediate CA, root CA). The enclave splits the chain into its three constituent certificates, converts each from PEM to DER in-enclave (no host-side parsing is trusted), and extracts the public key from each certificate's `SubjectPublicKeyInfo`. The Root CA's extracted public key is then compared, byte-for-byte, against the public key extracted from the hard-coded Intel Root CA PEM. A mismatch aborts validation immediately.
2. **Intermediate CA signature.** The intermediate CA's `TBSCertificate` (the canonically-signed portion of the certificate) is verified under the Root CA's public key. This establishes that Intel signed the intermediate.
3. **PCK certificate signature.** The PCK certificate's `TBSCertificate` is verified under the intermediate CA's public key. This establishes that the PCK was issued by Intel through its intermediate CA, and therefore that the platform was provisioned by Intel.
4. **PCK $\rightarrow$ QE Report signature.** The QE Report carried in the quote (a 384-byte `sgx_report_t`) is verified under the PCK's public key, using the QE Report Signature also carried in the quote. This binds the QE's report to the platform's PCK, establishing that the QE on this specific platform produced the certification material.
5. **AK $\leftrightarrow$ QE Report binding.** The enclave computes $h = \text{SHA-256}(\text{AK} \parallel \text{QE_Auth_Data})$ using the AK public key and authentication data extracted from the quote, and compares the result against the first 32 bytes of the QE Report's `report_data` field. A match confirms that the AK public key in the quote is the one that was certified by the QE Report just verified in step 4; without this check, an attacker could substitute their own AK alongside a stolen QE Report and pass step 4 trivially.
6. **AK $\rightarrow$ ISV Report signature.** Finally, the quote's ISV Enclave Report Signature is verified under the AK's public key. The signed payload is the first 432 bytes of the quote — the ISV Enclave Report, including its `report_data` field — which is the actual content the peer enclave is attesting.

A failure at any step causes the entire ecall to abort, with the credentials never decrypted and the response not produced. A successful pass through all six steps establishes that:

- The quote was produced by a genuine SGX platform provisioned by Intel;
- The QE on that platform certified the AK that signed the ISV report; and
- The ISV report's contents — crucially, including the public key in its `report_data` — have not been tampered with since the QE produced them.

The MRENCLAVE comparison that establishes the peer is running an identical codebase is performed separately, by a helper that produces a fresh local report and reads its MRENCLAVE field.

What this validation does not cover. The validation just described is a full chain-of-trust verification: it establishes that the quote is structurally and cryptographically valid, that it chains to Intel’s trust anchor and has a valid code base. Three further checks that a complete DCAP verifier normally performs are deliberately omitted at this stage, because none of them is feasible from inside the enclave:

- **Wall-clock validity periods.** The PCK certificate’s `notBefore` and `notAfter` are not checked, because the enclave has no reliable source of trusted time.
- **TCB level.** The TCB Info document from Intel encodes whether the platform’s security version is up to date. The host could fetch it and the enclave verify its signature against the pinned Intel root, but the enclave cannot check the document’s freshness without trusted time, so a stale TCB Info passing a downgraded platform could go unnoticed.
- **Certificate revocation.** Intel’s CRL lists revoked PCK certificates and intermediate CAs. The same host-fetch-plus-signature-verification path works, but a stale CRL is again undetectable without trusted time, so a recent revocation could be missed.

These checks are not skipped silently — they are performed by the Authenticator Emulator during the `INIT` attestation (Section 4.1.2.3), where network access to Intel’s services is available and the verifying party (the Emulator) is independent of the platform being verified. The Emulator therefore acts as the policy layer of the attestation pipeline, while the enclave handles the chain-of-trust layer. For migration specifically, this asymmetry is acceptable because both enclaves are validating peer quotes in a context where the Emulator has already attested both platforms at session start.

Credential gathering. With the destination’s quote validated and P_{dst} extracted from its `report_data`, the source enclave assembles the export response. The cryptographic structure of this response is described abstractly in Section 3.5.2; this paragraph fills in the implementation-level concrete details: the serialization format of the payload, the row-iteration pattern, and the buffer sizing constraints.

ECIES envelope. The envelope construction follows the design of Section 3.5.2 step by step. The enclave produces fresh randomness for the payload key K_{pay} (16 bytes), the payload IV and the wrapping IV (12 bytes) using `sgx_read_rand`; generates a fresh P-256 ephemeral keypair (e_{priv}, e_{pub}) ; computes s via `sgx_ecc256_compute_shared_dhkey` against P_{dst} ; derives $K_{wrap} = \text{HMAC-SHA256}(s, \text{"wrap_key"})$ truncated to 128 bits; and encrypts K_{pay} under K_{wrap} with AES-GCM. The result is serialized into 108 bytes in the order $e_{pub}^x \parallel e_{pub}^y \parallel \text{wrap_iv} \parallel \text{enc}(K_{pay}) \parallel \text{tag}$ (32+32+12+16+16), with the public-key bytes in big-endian to match the convention used elsewhere in the protocol, and then hex-encoded for transport. e_{priv} and K_{wrap} are zeroed immediately after the envelope is sealed.

Payload plaintext layout. The credential payload — the buffer that will be encrypted under K_{pay} — has the following structure, assembled in a single pass over the user’s rows:

$$\underbrace{\text{UID}}_{32 \text{ bytes}} \parallel \underbrace{\text{count}}_{2 \text{ bytes LE}} \parallel \left(\underbrace{\text{pt_len}}_{2 \text{ bytes LE}} \parallel \underbrace{\text{record plaintext}}_{\text{pt_len bytes}} \right)^{\text{count}}$$

Each *record plaintext* is exactly the on-disk plaintext from the source’s database, augmented for derived credentials by the upgrade step described below. The 2-byte length prefixes allow the destination to parse records back unambiguously even though they vary in size.

Row iteration. The enclave derives the local `mac_uid` for the UID under its own IK, then calls `ocall_db_list_count` to obtain the number of rows indexed there. It allocates a serialization buffer on the enclave heap, sized for the worst case of every row carrying the maximum credential plaintext (1 KiB), and iterates over the rows via `ocall_db_list_at`.

For each row, the enclave derives K_{row} and IV (the “`cred_enc`” / “`cred_iv`” HMACs from Section 3.3.2), decrypts the blob with `mac_cred` $\parallel$ `mac_uid` as the AAD, and inspects the type marker:

- For a *derived* credential, the enclave re-derives the private key from IK and the Credential_ID embedded in the plaintext (the same HKDF construction as in registration), flips the type marker from 0x01 to 0x02, and appends the 32-byte private key after the existing length-prefixed RP and `RP_user` fields. The credential is thereby converted into migrated form, ready for the destination to store directly.
- For a credential already in *migrated* form (0x02), the plaintext already contains the private key in the same layout, and is forwarded as-is.
- Rows whose decryption fails, or whose type marker is unrecognized, are logged and skipped. The iteration continues with the next row, and the count field at the start of the payload is patched at the end of iteration with the actual number of successfully serialized records.

When iteration completes, the serialized buffer is encrypted under K_{pay} with AES-GCM and the payload IV, producing the triple (IV, ciphertext, tag), which is hex-encoded for transport.

The encrypted payload is then paired with the envelope and the source’s own quote to form the response JSON, which is encrypted under the session key before returning to the host:

```
{ "status": "ok",
  "quote":   "<source's DCAP quote, hex>",
  "auth":    "<108-byte envelope, hex>",
  "payload": "<IV || ciphertext || tag, hex>" }
```

The `ecall`’s response buffer is statically sized at 64 KiB (Section 4.3.2.1), and after the envelope and source’s quote are factored in — the latter dominates at roughly 4 KiB of DCAP-quote bytes, expanded to 8 KiB by the hex encoding required by the JSON transport — the practical cap on the encrypted credential payload is on the order of 50 KiB, accommodating several hundred credentials in a single migration call. This is well above any plausible per-user load; users with extreme numbers of credentials at a single source would need either a more compact serialization format (e.g., binary transport in place of hex) or a paginated export protocol.

4.3.2.7 Import Credentials

The destination side of the migration protocol is implemented by `ecall_import`. The `ecall` receives, inside its session-encrypted body, three hex-encoded fields produced by the source: the source's quote, the 108-byte auth envelope (Section 4.3.2.6), and the payload ciphertext. The destination's job is to validate the source, unwrap the payload, and re-index every received credential under its own IK before inserting it into the local database.

The quote-validation logic is identical to the source side (Section 4.3.2.6): the same chain-of-trust verifier is invoked, and the same scope limitations apply. On success, the destination extracts the source's attested public key from the quote's `report_data` field and holds it for the binding check below.

Binding check. After parsing the auth envelope into its four components ($e_{\text{pub}}^x \parallel e_{\text{pub}}^y$, the wrap IV, the encrypted payload key, and the wrap tag), the destination compares the e_{pub} from the envelope against the public key it just extracted from the source quote's `report_data`. The two must match exactly. A mismatch aborts the import and the destination returns an error to the host without touching the payload.

This check is the defense against the credential-injection attack described in Section 3.5.3: an attacker who possessed a valid quote with the expected `MRENCLAVE` but who had not produced this specific transfer envelope would be unable to make their substitute e_{pub} appear in the quote's `report_data`, since that would require either producing a fresh quote (impossible without compromising SGX) or finding a hash collision in the AK signature chain over the report. Without the binding check, the destination would accept any envelope whose ECDH product happened to be computable under its own private key, which an attacker controlling the ephemeral keypair can trivially arrange.

Payload unwrap. Once the binding is confirmed, the destination performs the ECIES-style unwrap. It retrieves its stable IK-derived keypair via the same helper used at `INIT` attestation, and computes the shared secret $s = \text{ECDH}(\text{IK}_{\text{dst-priv}}, e_{\text{pub}})$ via `sgx_ecc256_compute_shared_dhkey`. The wrapping key $K_{\text{wrap}} = \text{HMAC-SHA256}(s, \text{"wrap_key"})$ is then derived (truncated to 128 bits), and the encrypted payload key is decrypted under K_{wrap} with AES-GCM, using the wrap IV and tag from the envelope. The shared secret and K_{wrap} are zeroed immediately after this single use.

The recovered K_{pay} is then used to decrypt the payload ciphertext: the payload buffer is laid out as `IV || ciphertext || tag` (Section 4.3.2.6), so the destination separates these three fields by their fixed positions, allocates a buffer for the plaintext on the enclave heap, and decrypts with `sgx_rijndael128GCM_decrypt`. A decryption failure at this stage aborts the entire import; partial recovery is not attempted, since a single AEAD failure indicates either a tampered payload or a corrupted envelope, both of which are integrity violations.

Re-indexing and insertion. The decrypted payload has the format defined in Section 4.3.2.6: a 32-byte UID, a 2-byte little-endian record count, and the count many length-prefixed credential records. The destination derives its own `mac_uiddst` from the UID under its local IK, with the `"uid_index"` domain-separation label, and then iterates over the records.

For each record, the destination performs the operations that were the source's at registration time (Section 4.3.2.4), now under its own IK:

- it extracts the `Credential_ID` from offset 1 within the record's plaintext;
- it computes $\text{mac_cred}^{\text{dst}} = \text{HMAC}_{\text{IK}_{\text{dst}}}(\text{"storage_index"} \parallel \text{Credential_ID})$;
- it derives a fresh per-row key $K_{\text{row}}^{\text{dst}}$ and IV from IK_{dst} and the new $\text{mac_cred}^{\text{dst}}$, using the `"cred_enc"` and `"cred_iv"` labelled HMACs from Section 3.3.2;
- it encrypts the record plaintext (without modification, including the trailing 32-byte private key) with $K_{\text{row}}^{\text{dst}}$ and the new IV under AES-128-GCM, with $\text{mac_cred}^{\text{dst}} \parallel \text{mac_uid}^{\text{dst}}$ as the additional authenticated data;
- it issues `ocall_db_insert` with $\text{mac_cred}^{\text{dst}}$ as the primary key, the new blob as the value, and $\text{mac_uid}^{\text{dst}}$ as the secondary index.

The primary-key constraint on $\text{mac_cred}^{\text{dst}}$ provides the duplicate-detection mechanism described in Section 3.5.2: if the same credential has already been imported to this destination, the new $\text{mac_cred}^{\text{dst}}$ collides with the existing row's key, the database returns a distinct status code, and the destination skips that record rather than overwriting. Three counters track the outcome across the iteration: `inserted`, `skipped` (duplicates), and `failed` (records whose decryption, encryption, or insertion failed for any other reason). A single failed record does not abort the import; the iteration continues to the next record, matching the silent-skip pattern from the assertion and export flows.

After iteration completes, IK and $\text{mac_uid}^{\text{dst}}$ are zeroed, the decrypted payload buffer is zeroed and freed, and the destination assembles its response:

```
{ "status": "ok", "inserted": N, "skipped": M, "failed": K }
```

The response is then encrypted under the session key and returned to the host. The three counters are visible to the Emulator and ultimately to the user, who is told how many of the requested credentials were successfully migrated, how many were already present at the destination, and how many failed to transfer.

4.3.3 Storage

Persistent storage for credential records lives entirely in the untrusted host process. The enclave never touches the filesystem directly: it issues ocalls to a small SQLite layer that the host implements, and treats every byte returned from those ocalls as adversarial input until validated cryptographically (Sections 4.3.2.4 and 4.3.2.5).

SQLite was chosen for three reasons. It is embedded, so the host has no external database process to coordinate with. It is well understood and conservatively engineered, so its surface area is predictable. And its single-file format makes backup, deletion straightforward. The cryptographic design accommodates this: every value the enclave hands to the database is an AEAD ciphertext or an opaque MAC, so a fully compromised SQLite layer can drop, corrupt, or replay rows, but it cannot read credential material or forge an authentic record (Section 3.6).

4.3.3.1 Schema and indexing

The credential store uses a single table with three columns:

```
CREATE TABLE creds (  
    cred_id_mac BLOB PRIMARY KEY,  
    uid_mac     BLOB NOT NULL,  
    blob        BLOB NOT NULL  
) WITHOUT ROWID;  
  
CREATE INDEX idx_uid ON creds(uid_mac);
```

The three columns hold the values defined in Section 3.3.2: `cred_id_mac` is `mac_cred` (the storage-indexed MAC of the credential identifier), `uid_mac` is `mac_uid` (the storage-indexed MAC of the UID), and `blob` is the AEAD-encrypted record. The `WITHOUT ROWID` option removes SQLite’s hidden integer row identifier, so each row’s physical layout is keyed directly on `cred_id_mac`. Lookup by credential identifier is then a single B-tree traversal on the primary key.

The secondary index on `uid_mac` supports the resident-credential and migration-export flows, which enumerate all rows for a given UID without knowing any of their credential identifiers. Without this index, those flows would degrade to a full table scan.

The `PRIMARY KEY` constraint on `cred_id_mac` is load-bearing for correctness, not just performance. An insert that collides with an existing row returns `SQLITE_CONSTRAINT`, which the host translates into status code 2 and the enclave interprets as a duplicate. Registration treats this as an error (Section 4.3.2.4); migration import treats it as a successful no-op (Section 4.3.2.7). In both cases the existing record is preserved untouched.

4.3.3.2 Concurrency

The enclave’s runtime configuration allows up to ten concurrent `ocalls` (Section 4.3.2.1), so the database layer must be safe under parallel access. The host opens a single SQLite connection at startup and serializes all access to it with a process-wide `pthread` mutex. Each `ocall` acquires the mutex on entry and releases it before returning.

A more aggressive design would open one connection per worker thread and rely on SQLite’s own `SQLITE_OPEN_FULLMUTEX` mode for finer-grained locking. The single-connection-plus-mutex approach is simpler to reason about and adequate for the request rate this prototype is designed to handle, where the bottleneck is the enclave’s cryptographic work rather than database contention.

The connection itself is opened with the `SQLITE_OPEN_FULLMUTEX` flag, so its internal state is also mutex-protected against any misuse from outside the `ocall` layer. It is configured with the WAL journal mode and normal synchronization, which give a useful durability and throughput balance for write-heavy workloads.

4.3.3.3 Prepared statements

Each of the five SQL queries the host issues is prepared once at database-open time and cached for the lifetime of the connection. The five statements are:

- `INSERT INTO creds(cred_id_mac, uid_mac, blob) VALUES(?,?,?)`, used by registration and import.
- `SELECT uid_mac, blob FROM creds WHERE cred_id_mac=?`, used by assertion with a credential identifier.
- `SELECT COUNT(*) FROM creds WHERE uid_mac=?`, used at the start of the resident-credential and export flows.
- `SELECT cred_id_mac, blob FROM creds WHERE uid_mac=? ORDER BY cred_id_mac LIMIT 1 OFFSET ?`, used to iterate rows for a given UID during resident-credential assertion and export.
- `DELETE FROM creds WHERE cred_id_mac=?`, exposed for completeness. The present enclave code does not invoke it, but the ocall is in place for future credential-revocation logic.

Caching the prepared statements avoids repeated parse-and-plan overhead on every ocall. The effect is most visible in the export and resident-assertion flows, where the per-row `list_at` statement is invoked many times in a tight loop.

4.3.3.4 Status codes

The ocall API uses a small set of integer status codes that the host returns alongside each operation's data. The enclave is responsible for interpreting them and converting them into ecall return values. Five distinct codes are in use:

- **0** for success.
- **1** for not found (lookup returned no row, or iterator offset is out of range).
- **2** for duplicate (insert violated the primary-key constraint).
- **-1** for generic SQLite step failure (unexpected return code from `sqlite3_step`).
- **-2** for capacity exceeded (a column value from the database is larger than the buffer the enclave provided).

The capacity-exceeded path matters because of the SDK marshalling constraint discussed in Section 4.3.2.1: ocall output buffers have fixed maximum sizes declared in the EDL, so the host has to bound-check every column before copying it back across the enclave boundary. A blob that would not fit is reported as a status, never truncated silently.

4.3.3.5 What the storage layer does not protect

The host can mount availability attacks: it can drop the database, replace it with an empty file, replay an old snapshot, or fail to commit a write. The first three are detectable to the user, because credentials simply stop working, with no signature of authenticity to fall back on. The fourth is a rollback attack: the host accepts a write and reports success, but never persists the row. The enclave's interaction with the database is write-once, so a rollback against any individual credential reduces to the credential being absent on the next assertion, and the user notices on their next attempt.

Introducing credential revocation or update would reopen the rollback attack, because the host could then replay a snapshot in which a revoked credential is still present. Defending against that variant requires anchoring the database state to something the host cannot rewrite, through trusted monotonic counters or external state pinning, neither of which is available in our deployment. We acknowledge this as a known limitation, noted alongside denial of service in the threat model (Section 3.2).

Evaluation

This chapter presents a temporal evaluation of the proposed solution against a hardware FIDO2 authenticator. The goal is to quantify the client-observed cost of authentication operations on both devices.

5.1 Test Bench

The reference hardware device is a FEITIAN ePASS FIDO Security Key¹, certified in 2018 and accessed over its USB interface. Detailed information about the internal storage and processing components is not publicly disclosed and would require physical inspection of the device to determine.

A hardware token is not a straightforward subject for temporal evaluation, as the standard FIDO2 flow requires physical user presence (UP) for credential creation and assertion. The duration of this human-driven step is large relative to the protocol cost and varies between executions. The methodology described in Section 5.3 addresses this constraint.

A more complete evaluation would include other virtual FIDO2 implementations backed by a TPM for credential storage (Section 2.5.1), platform authenticators such as Windows Hello and Apple Touch ID, and a more recent hardware device than the FEITIAN ePASS. Owing to time constraints, the broader comparison is deferred to future work (Chapter 6.2).

5.2 Communication with the SGX Host

For security reasons, the HTTP service running on the SGX host is not exposed beyond the internal network of the Universidade de Aveiro. Access requires either a VPN connection and an SSH session from an authorised host. For the purposes of this evaluation, the HTTP requests issued by the virtual authenticator were tunnelled through an SSH local-port forward:

```
ssh -L 8080:localhost:8080 username@IP
```

¹Manual and specifications available in the Federal Communications Commission database: https://apps.fcc.gov/oetcf/eas/reports/ViewExhibitReport.cfm?mode=Exhibits&RequestTimeout=500&calledFromFrame=N&application_id=UXAjLT9NJKVpssjIDqjCUg%3D%3D&fcc_id=ZD3FTMULTIPASSK13

The SSH tunnel adds an additional layer of encryption and one extra network hop to every request. Although this overhead is included in the reported measurements, the results in Section 5.4 show that its impact on per-operation latency is limited.

5.3 Test Client

The measurements were collected with a custom Python client that communicates directly with both authenticators over the CTAPHID transport. This avoids the variability introduced by browser-mediated WebAuthn flows and gives precise control over the timing instrumentation. The procedure for each run was:

1. Insert the hardware device; the kernel exposes it as `/dev/hidrawA`.
2. Start the virtual authenticator; the kernel exposes it as `/dev/hidrawB`.
3. Issue the CTAP messages `init`, `authenticatorInfo`, `makeCredential` and `getAssertion` by writing the corresponding CBOR-encoded frames to each character device.
4. Record the timing of each operation as described below.

5.3.1 Timing Methodology

For the virtual authenticator no user-presence step is involved, so the measurement is the interval between the dispatch of the request frame and the receipt of the complete response.

The hardware device is more delicate. During the user-presence wait, the authenticator periodically emits a CTAPHID keepalive frame (`0xBB`) to inform the host that the operation is still in progress. These frames bound the user-presence interval from both sides and allow us to subtract it from the total elapsed time. Two timestamps are recorded:

- *pre-touch*: the interval between dispatching the request and receiving the first keepalive frame. This captures the cost of frame parsing on the device before it begins waiting for user input.
- *post-touch*: the interval between the device emitting its final keepalive or first response packet and the arrival of the full response.

The final measurement is the sum of pre-touch and post-touch intervals, excluding the human-driven user-presence wait.

The post-touch interval admits two reasonable definitions, each measuring a slightly different quantity:

- (a) **last keepalive to end-of-response**. The interval from the timestamp at which the host reads the final `0xBB` frame to the timestamp at which the last response packet is received. This interval includes the device's post-touch work but overestimates it substantially. The recorded "last keepalive" timestamp marks the device's most recent keepalive emission,

which falls at some point before the user touch, not at the touch itself. The interval therefore includes an unknown fraction of the user-presence wait, bounded above by the device’s keepalive emission period. In our measurements this inflation dominated the reported value, making the interval unsuitable as a measurement of device-side computation.

- (b) **first response packet to end-of-response.** The interval from the arrival of the first non-keepalive packet to the arrival of the last. This captures only the response transfer over the HID transport. Device-side cryptographic work occurs during the user-presence window and is therefore not included.

Both definitions are reported separately in Section 5.4. Definition (b) yields a stable lower bound on the protocol cost, while definition (a) gives a noisier upper bound that incorporates some fraction of the device’s internal computation.

5.4 Results

Tables 5.1 and 5.2 report the measured timings in milliseconds for both devices under both post-touch definitions. The two definitions were measured in separate runs of the test client; the differences between the virtual-device columns of Tables 5.1 and 5.2 reflect some run-to-run variance and differences related to the measurement methodology.

Table 5.1: Per-operation timing (ms) for the hardware FEITIAN device and the SGX-backed virtual authenticator, with post-touch defined as first response packet to end of response (definition (b)). Real-device measurements exclude the user-presence wait. Excludes device-side cryptographic work, which is not observable from the host. Single execution per cell.

	Hardware	Virtual	Δ
<i>Setup</i>			
init	4.01	2057.83	+2053.82
get_info	91.94	59.26	−32.68
setup total	95.95	2117.09	+2021.14
<i>Per-operation</i>			
make_credential	107.92	31.77	−76.15
get_assertion	61.63	31.08	−30.55
operation total	169.56	62.86	−106.70
total	265.51	2179.95	+1914.44

Table 5.2: Per-operation timing (ms) for the hardware FEITIAN device and the SGX-backed virtual authenticator, with post-touch defined as last keepalive to end of response (definition (a)). Real-device measurements exclude the user-presence wait. Includes part of the user-presence window, as discussed in Section 5.3.1. Single execution per cell.

	Hardware	Virtual	Δ
<i>Setup</i>			
init	2.50	2046.45	+2043.94
get_info	93.90	51.11	−42.79
setup total	96.40	2097.55	+2001.15
<i>Per-operation</i>			
make_credential	1635.86	35.41	−1600.45
get_assertion	1277.74	31.58	−1246.16
operation total	2913.60	66.99	−2846.61
total	3010.00	2164.54	−845.46

5.4.1 First-Packet Measurement

In the setup phase, `init` costs 2–4 ms on the hardware device and approximately two seconds on the virtual authenticator; this gap reflects SGX remote attestation. For `get_info`, the hardware device takes 91.94 ms against 59.26 ms on the virtual authenticator, a difference of roughly 33 ms. Both devices return their response over the same CTAPHID transport, framed into 64-byte HID packets; the difference lies in where the response is assembled. The hardware device executes the operation on a constrained microcontroller, whereas the virtual authenticator runs the same logic on the host CPU. The 33 ms gap is consistent with the cost of microcontroller execution that the virtual authenticator avoids.

For `make_credential`, the hardware device takes 107.92 ms against 31.77 ms on the virtual authenticator. The hardware figure decomposes into 59.88 ms of pre-touch command parsing and 48.04 ms of post-touch response transfer; reflecting the size of the response payload which carries the newly generated public key and the attestation statement. The virtual authenticator uses the same CTAPHID transport to communicate with the test client, but the cryptographic work runs on the host CPU rather than on a constrained microcontroller, and the response payload is assembled in memory before being framed into HID packets. The dominant cost on the hardware side—microcontroller execution and per-frame USB transfer—does not apply to the virtual device, which accounts for the difference.

For `get_assertion`, the hardware device takes 61.63 ms against 31.08 ms on the virtual authenticator. The hardware figure is dominated by pre-touch parsing (53.71 ms) with a small post-touch transfer (7.93 ms), since the response carries only a signature and is therefore much smaller than the `make_credential` response. On the virtual authenticator, the cost is essentially identical to its `make_credential` cost, because both operations follow the same transport path on the host side and the difference in response size is small.

5.4.2 Last-Keepalive Measurement

Table 5.2 reports a separate run using the last-keepalive definition of post-touch. The setup and virtual-device figures are consistent with those of Table 5.1, as neither involves the user-

presence window.

The hardware-device operations, in contrast, increase a lot: `make_credential` rises from 107.92 ms to 1635.86 ms, and `get_assertion` from 61.63 ms to 1277.74 ms. As discussed in Section 5.3.1, this inflation is not a measurement of device-side computation but a consequence of the device’s keepalive cadence. The recorded “last keepalive” timestamp falls at the device’s most recent keepalive emission, which can precede the user touch by up to one full keepalive interval. The 1530 ms and 1216 ms inflations observed here therefore approximate the time between the device’s last keepalive and the user touch in this run; this measurement includes not only the user-presence interval but also some portion of the wait that preceded the device’s final keepalive emission.

This definition was retained as an intermediate step in the development of the measurement methodology rather than as a final result. Although it discards the interval between the device’s first and last keepalive—which covers most of the user-presence wait—the gap between the last keepalive and the actual touch remains in the measurement and inflates the reported time. The measurements presented in Section 5.4.1 are those we rely on for the comparison between the two devices.

Conclusion

This chapter concludes the thesis. Section 6.1 restates the authenticator that was built and the security properties it provides, and Section 6.2 sets out the directions the work leaves open.

6.1 Conclusion

This thesis built a virtual FIDO2/WebAuthn authenticator backed by an Intel SGX enclave. The system has three components: a Linux Authenticator Emulator that presents itself to the operating system as a USB HID token through the UHID interface and speaks CTAP2 to whatever WebAuthn client requests it; an SGX host that exposes the enclave's operations over HTTP; and the enclave itself, which holds the Identity Key from which every credential's material is derived and inside which every cryptographic operation takes place.

Credential private keys for locally-derived credentials are never written to persistent storage at all, since they are reconstructed on demand from the Identity Key. The credential database, even under full read access by an attacker, reveals no information about which users have credentials at which relying parties: every row is an opaque MAC index pointing to an AEAD ciphertext, both derived under a key the attacker cannot recompute. Credentials can be migrated between SGX hosts through a mutually-attested protocol that re-indexes them under the destination's Identity Key, preserving the same confidentiality and unlinkability properties at the destination as at the source.

The approach occupies a position distinct from both existing alternatives. Hardware tokens are physical objects, and lose, replace, or break the way physical objects do, with no portability mechanism short of registering a second token alongside the first. Authenticators have evolved into passkeys, which do support portability through cloud sync providers (Apple's iCloud Keychain, Google Password Manager, Microsoft account, and third-party providers like 1Password), so the credential survives device loss. That portability, however, is mediated by a sync provider that holds the user's credentials in its own infrastructure, protected by whatever the user's account with that provider permits. The SGX-backed authenticator is, to our knowledge, the only design that supports cross-device portability without an intermediate custodian: credentials move directly between two SGX enclaves, both verified through DCAP mutual attestation to be running the same code. The trade-off is the operational cost of running an SGX host; whether that cost is the right price for removing the cloud provider from the trust

chain is a question this work does not try to settle. What it does settle is that the combination is technically achievable with the SGX primitives.

6.2 Future Work

This section describes the directions in which the proposed solution could be extended, and the gaps the present implementation leaves open.

FIDO2 Certification with SGX keys. The registration flow currently produces credentials without an attestation statement, so relying parties that require one reject them. The fix equips the enclave with a group attestation key, provisioned once: the enclave generates a DCAP quote, a certification authority in the FIDO2 quality PKI (not necessarily Intel) verifies it against Intel's Provisioning Certification chain, confirms the expected MRENCLAVE, and only then issues a certificate and the group key bound to that MRENCLAVE. The key is sealed to survive restarts. At registration the enclave signs the attestation data with this key and emits a packed statement whose `x5c` chain carries the MRENCLAVE-bound certificate. The relying party validates that chain against the FIDO2 quality root and reads the certification level, both obtained from the metadata service via the AAGUID, and never handles a quote; the AAGUID indexes the trust root and level for the certified build, while the MRENCLAVE binding lives in the leaf certificate, since the service describes a model and has no field for an enclave measurement. Verifying the quote at the relying party was set aside, as it would force every party to implement DCAP and break the WebAuthn abstraction.

Authentication with allow lists. The CTAP `authenticatorGetAssertion` command can carry an *allow list*, a relying-party-supplied set of `Credential_ID`s indicating which specific credentials the RP is willing to accept. In the current implementation this list is not honoured: the Emulator forwards only the first `Credential_ID` from the list to the SGX host, and the resulting flow behaves identically to the single-`Credential_ID` case (Section 3.4, "Authentication with a Credential ID"). A user who has registered more than one credential with the same RP and whose authentication request arrives with an allow list naming several of them will always authenticate with whichever credential the RP happened to list first.

The fix is to handle the allow list properly on the Emulator side: each `Credential_ID` should be resolved against the SGX host, the user should be presented with the resulting candidates through the same disambiguation chooser used for resident credentials (Section 4.1.3), and the user's selection should determine which credential is forwarded to the relying party. The enclave already supports the underlying operation; this is purely an Emulator-side change. It was deferred for time reasons, but is the most user-visible gap in the current implementation.

Resident-credential management. Every credential in the present design is a resident credential, since the enclave stores a record for each one and exposes it through the discoverable-credential flow. The only operation currently supported on these records is creation; CTAP defines two further management operations, `authenticatorCredentialManagement's update` (to change a credential's user information, including the `RP_username` field that drives

the disambiguation chooser) and *delete* (to remove a credential the user no longer wants). Both can be added without changing the cryptographic design: the database layer already exposes a `ocall_db_delete` primitive, and *update* is a delete-then-reinsert under a fresh row encryption. Adding these completes the lifecycle on the resident-credential side and gives the user a way to clean up old or unwanted entries.

Wider evaluation. The evaluation in Chapter 5.1 compares the Emulator against a single hardware authenticator. A more meaningful evaluation would cover several physical authenticators across different vendors and generations, and would extend to platform authenticators, in particular Windows Hello and Apple’s Touch ID-backed implementations. A comparison against platform authenticators would also bring out a useful contrast on portability. Platform authenticators keep their credential keys inside the secure element of the device they were created on, with no supported way to move them to a different device; losing or replacing the device means losing the credentials. The SGX-backed authenticator’s credential keys are similarly bound to one platform (through the enclave’s hardware-derived Identity Key).

Rate limiting at the HTTP layer. Section 4.1.3.3 notes that an attacker with database access and the ability to invoke the enclave can mount an online guessing attack against weak user attributes by submitting candidate UIDs to the SGX host and observing which produce successful assertions. Although the HTTP service is outside the TCB and not relied upon for any confidentiality property, it is the right place to bound the rate of such requests, since it is the only component in the stack that can observe network-layer properties (source address, authenticated session identifier, request rate) that the enclave cannot. A production deployment should implement per-source rate limits and exponential back-off on repeated UID failures, with quotas tracked outside the enclave. This does not change the enclave’s security argument, but it raises the cost of online attribute-guessing attacks from negligible to practically prohibitive.

References

- [1] D. Temoshok, J. L. Fenton, Y.-Y. Choong, *et al.*, «Digital Identity Guidelines: Authentication and Lifecycle Management», National Institute of Standards and Technology, Tech. Rep. NIST SP 800-63B-4, Jun. 2025. DOI: 10.6028/NIST.SP.800-63B-4.
- [2] S. Das, A. Dingman, and L. J. Camp, «Why Johnny Doesn't Use Two Factor: A Two-Phase Usability Study of the FIDO U2F Security Key», in *Financial Cryptography and Data Security (FC 2018)*, ser. Lecture Notes in Computer Science, vol. 10957, Berlin, Heidelberg: Springer, 2018. DOI: 10.1007/978-3-662-58387-6_9.
- [3] K. Reese, T. Smith, J. Dutson, J. Armknecht, J. Cameron, and K. Seamons, «A Usability Study of Five Two-Factor Authentication Methods», in *Fifteenth Symposium on Usable Privacy and Security (SOUPS 2019)*, Santa Clara, CA, USA: USENIX Association, 2019. DOI: 10.5555/3361476.3361502.
- [4] J. Colnago, S. Devlin, M. Oates, *et al.*, «It's Not Actually That Horrible: Exploring Adoption of Two-Factor Authentication at a University», in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, ACM, 2018. DOI: 10.1145/3173574.3174030.
- [5] V. Costan and S. Devadas, *Intel SGX Explained*, Cryptology ePrint Archive, Paper 2016/086, 2016. [Online]. Available: <https://eprint.iacr.org/2016/086>.
- [6] M. Barbosa, A. Boldyreva, S. Chen, and B. Warinschi, «Provable Security Analysis of FIDO2», in *Advances in Cryptology – CRYPTO 2021*, ser. Lecture Notes in Computer Science, vol. 12827, Cham: Springer, 2021. DOI: 10.1007/978-3-030-84252-9_5.
- [7] J. Kunke, S. Wiefeling, M. Ullmann, and L. Lo Iacono, «Evaluation of Account Recovery Strategies with FIDO2-based Passwordless Authentication», in *Open Identity Summit 2021 (OID '21)*, H. Roßnagel, C. H. Schunck, and S. Mödersheim, Eds., Bonn: Gesellschaft für Informatik e.V., 2021, ISBN: 978-3-88579-706-7. DOI: 10.48550/arXiv.2105.12477. [Online]. Available: <https://nbn-resolving.org/urn:nbn:de:hbz:1044-opus-54904>.
- [8] S. Ghorbani Lyastani, M. Schilling, M. Neumayr, M. Backes, and S. Bugiel, «Is FIDO2 the Kingslayer of User Authentication? A Comparative Usability Study of FIDO2 Passwordless Authentication», in *2020 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2020. DOI: 10.1109/SP40000.2020.00047.
- [9] M. Barbosa, A. Cirne, and L. Esquivel, «Rogue Key and Impersonation Attacks on FIDO2: From Theory to Practice», in *Proceedings of the 18th International Conference on Availability, Reliability and Security (ARES '23)*, ACM, 2023. DOI: 10.1145/3600160.3600174.
- [10] R. Hussein, «Exploration of the Security and Usability of the FIDO2 Authentication Protocol», M.S. thesis, University of Maryland, 2022. [Online]. Available: <https://api.drum.lib.umd.edu/server/api/core/bitstreams/63e5eace-630f-4a78-8186-5516e94e6d68/content>.
- [11] M. Casagrande and D. Antonioli, «CTRAPS: CTAP Client Impersonation and API Confusion on FIDO2», in *2025 IEEE 10th European Symposium on Security and Privacy (EuroS&P)*, Venice, Italy, Jun. 2025. DOI: 10.1109/EuroSP63326.2025.00063.
- [12] M. Sabt, M. Achemlal, and A. Bouabdallah, «Trusted Execution Environment: What It Is, and What It Is Not», in *2015 IEEE Trustcom/BigDataSE/ISPA*, vol. 1, IEEE, 2015. DOI: 10.1109/Trustcom.2015.357.
- [13] S. Fei, Z. Yan, W. Ding, and H. Xie, «Security Vulnerabilities of SGX and Countermeasures: A Survey», *ACM Computing Surveys*, vol. 54, no. 6, 2021, ISSN: 0360-0300. DOI: 10.1145/3456631.

- [14] Intel Corporation, *Foreshadow (L1 Terminal Fault): SGX Enclave Memory Disclosure*, Intel Security Advisory INTEL-SA-00161, 2018. [Online]. Available: <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00161.html>.
- [15] Intel Corporation, *Microarchitectural Data Sampling (MDS)*, Intel Security Advisory INTEL-SA-00233, 2019. [Online]. Available: <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00233.html>.
- [16] Intel Corporation, *Plundervolt: Voltage Fault Injection Vulnerability*, Intel Security Advisory INTEL-SA-00289, 2019. [Online]. Available: <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00289.html>.
- [17] J. Chuang, A. Seto, N. Berrios, S. van Schaik, C. Garman, and D. Genkin, «TEE.fail: Breaking Trusted Execution Environments via DDR5 Memory Bus Interposition», in *47th IEEE Symposium on Security and Privacy (IEEE S&P '26)*, IEEE Computer Society, 2026. DOI: 10.1109/SP63933.2026.00101. [Online]. Available: <https://tee.fail>.
- [18] Intel Corporation, *Intel SGX SDK Improper Initialization*, Intel Security Advisory INTEL-SA-00336, 2020. [Online]. Available: <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00336.html>.
- [19] Intel Corporation, *Intel SGX SDK for SGX2 Privilege Escalation*, Intel Security Advisory INTEL-SA-00548, 2021. [Online]. Available: <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00548.html>.
- [20] Intel Corporation, *Intel SGX SDK Information Disclosure*, Intel Security Advisory INTEL-SA-00677, 2022. [Online]. Available: <https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-00677.html>.
- [21] W. Zheng, Y. Wu, and X. Wu, «A Survey of Intel SGX and Its Applications», *Frontiers of Computer Science*, 2020. DOI: 10.1007/s11704-019-9096-y.
- [22] B. Fisch, D. Vinayagamurthy, D. Boneh, and S. Gorbunov, «IRON: Functional Encryption Using Intel SGX», in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*, New York, NY, USA: Association for Computing Machinery, 2017, ISBN: 9781450349468. DOI: 10.1145/3133956.3134106.
- [23] C. Priebe, K. Vaswani, and M. Costa, «EnclaveDB: A Secure Database Using SGX», in *2018 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2018. DOI: 10.1109/SP.2018.00025.
- [24] T. Peters, R. Lal, S. Varadarajan, P. Pappachan, and D. Kotz, «BASTION-SGX: Bluetooth and Architectural Support for Trusted I/O on SGX», in *Proceedings of the International Workshop on Hardware and Architectural Support for Security and Privacy (HASP)*, ACM, Jun. 2018. DOI: 10.1145/3214292.3214295.
- [25] F. Kelbert, F. Gregor, R. Pires, *et al.*, «SecureCloud: Secure Big Data Processing in Untrusted Clouds», in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2017, IEEE, 2017. DOI: 10.23919/DATE.2017.7926999.
- [26] S. Arnaudov, B. Trach, F. Gregor, *et al.*, «SCONE: Secure Linux Containers with Intel SGX», in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI '16)*, Savannah, GA, USA: USENIX Association, 2016, ISBN: 9781931971331. DOI: 10.5555/3026877.3026930.
- [27] S. Kim, J. Han, J. Ha, T. Kim, and D. Han, «SGX-Tor: A Secure and Practical Tor Anonymity Network with SGX Enclaves», 5, vol. 26, 2018. DOI: 10.1109/TNET.2018.2868054.
- [28] H. Duan, X. Yuan, and C. Wang, «LightBox: SGX-assisted Secure Network Functions at Near-native Speed», 2017. DOI: 10.48550/arXiv.1706.06261.
- [29] M.-W. Shih, M. Kumar, T. Kim, and A. Gavrilovska, «S-NFV: Securing NFV States by Using SGX», in *Proceedings of the 2016 ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization*, ACM, 2016. DOI: 10.1145/2876019.2876032.
- [30] F. Schuster, M. Costa, C. Fournet, *et al.*, «VC3: Trustworthy Data Analytics in the Cloud Using SGX», in *2015 IEEE Symposium on Security and Privacy*, IEEE, 2015. DOI: 10.1109/SP.2015.10.
- [31] A. Baumann, M. Peinado, and G. Hunt, «Shielding Applications from an Untrusted Cloud with Haven», in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI '14)*, Broomfield, CO, USA: USENIX Association, Oct. 2014, ISBN: 978-1-931971-16-4. DOI: 10.1145/2799647. [Online]. Avail-

able: <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/baumann>.

- [32] C.-C. Tsai, D. E. Porter, and M. Vij, «Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX», in *2017 USENIX Annual Technical Conference (USENIX ATC '17)*, Santa Clara, CA, USA: USENIX Association, Jul. 2017, ISBN: 978-1-931971-38-6. [Online]. Available: <https://www.usenix.org/conference/atc17/technical-sessions/presentation/tsai>.
- [33] J. Lind, C. Priebe, D. Muthukumaran, *et al.*, «Glamdring: Automatic Application Partitioning for Intel SGX», in *2017 USENIX Annual Technical Conference (USENIX ATC '17)*, Santa Clara, CA, USA: USENIX Association, Jul. 2017, ISBN: 978-1-931971-38-6. [Online]. Available: <https://www.usenix.org/conference/atc17/technical-sessions/presentation/lind>.
- [34] N. Weichbrodt, A. Kurmus, P. Pietzuch, and R. Kapitza, «AsyncShock: Exploiting Synchronisation Bugs in Intel SGX Enclaves», in *Computer Security – ESORICS 2016*, ser. Lecture Notes in Computer Science, vol. 9878, Cham: Springer, 2016. DOI: 10.1007/978-3-319-45744-4_22.
- [35] R. Sinha, S. K. Rajamani, S. A. Seshia, and K. Vaswani, «Moat: Verifying Confidentiality of Enclave Programs», in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15)*, Denver, CO, USA: ACM, 2015. DOI: 10.1145/2810103.2813608.
- [36] D. Tychalas, N. G. Tsoutsos, and M. Maniatakos, «SGXCrypter: IP Protection for Portable Executables Using Intel's SGX Technology», in *2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC)*, IEEE, 2017. DOI: 10.1109/ASPAC.2017.7858348.
- [37] E. Bauman and Z. Lin, «A Case for Protecting Computer Games with SGX», in *Proceedings of the 1st Workshop on System Software for Trusted Execution (SysTEX '16)*, New York, NY, USA: Association for Computing Machinery, 2016, ISBN: 9781450346702. DOI: 10.1145/3007788.3007792.
- [38] F. Chen, C. Wang, W. Dai, *et al.*, «PRESAGE: Privacy-preserving Genetic Testing via Software Guard Extension», in *BMC Medical Genomics*, vol. 10, 2017. DOI: 10.1186/s12920-017-0281-2.
- [39] W. Wang, S. Deng, J. Niu, M. K. Reiter, and Y. Zhang, «ENGRAFT: Enclave-guarded Raft on Byzantine Faulty Nodes», in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, ACM, 2022. DOI: 10.1145/3548606.3560639.
- [40] H. Qi, M. Xu, D. Yu, and X. Cheng, «SoK: Privacy-preserving Smart Contract», *High-Confidence Computing*, vol. 4, no. 1, 2024, ISSN: 2667-2952. DOI: 10.1016/j.hcc.2023.100183.
- [41] S. Van Schaik, A. Seto, T. Yurek, *et al.*, «Sok: Sgx.fail: How stuff gets exposed», in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024. DOI: 10.1109/SP54263.2024.00260.
- [42] M. McReynolds, «Azure announces next generation intel sgx confidential computing vms», URL: <https://techcommunity.microsoft.com/t5/azure-confidential-computing/azure-announces-next-generation-intel-sgx-confidential-computing/ba-p/2839934> (accessed on 01/29/2021), 2021.
- [43] Intel Corporation, *Architecture for Privacy Preserving Machine Learning with Intel SGX and TensorFlow*, <https://www.intel.com/>, 2020.
- [44] Microsoft and Intel, *Microsoft Deploys Confidential Computing to Protect \$25B per Year in Customer Payments*, <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/2024-02/microsoft-azure-confidential-computing-solution-brief.pdf>, 2023.
- [45] Microsoft Learn, *Azure Confidential Computing Products*, <https://learn.microsoft.com/en-us/azure/confidential-computing/overview-azure-products>, 2024.
- [46] FIDO Alliance, *FIDO Client-to-Authenticator Protocol (CTAP) v2.2*, <https://fidoalliance.org/specs/fido-v2.2-ps-20250714/fido-client-to-authenticator-protocol-v2.2-ps-20250714.pdf>, Proposed Standard, 2025.
- [47] W3C, *Web Authentication: An API for Accessing Public Key Credentials Level 2*, <https://www.w3.org/TR/webauthn-2/>, W3C Recommendation, 2021.
- [48] C. Shepherd, R. N. Akram, and K. Markantonakis, «Remote Credential Management with Mutual Attestation for Trusted Execution Environments», in *Information Security Theory and Practice (WISTP 2018)*, O. Blazy and C. Y. Yeun, Eds., Cham: Springer International Publishing, 2019, ISBN: 978-3-030-20074-9. DOI: 10.1007/978-3-030-20074-9_12.

- [49] T. Frassetto, D. Gens, C. Liebchen, A.-R. Sadeghi, and G. Tsudik, «FeIDo: Recoverable FIDO2 Tokens Using Electronic IDs», in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, ACM, 2022. DOI: 10.1145/3548606.3560584.
- [50] L. Wu, H. J. Cai, and H. Li, «SGX-UAM: A Secure Unified Access Management Scheme with One Time Passwords via Intel SGX», *IEEE Access*, vol. 9, 2021. DOI: 10.1109/ACCESS.2021.3063770.
- [51] National Institute of Standards and Technology, «Digital Signature Standard (DSS)», U.S. Department of Commerce, Tech. Rep. FIPS PUB 186-4, 2013. DOI: 10.6028/NIST.FIPS.186-4.
- [52] Intel Corporation, «Intel SGX Data Center Attestation Primitives: ECDSA Quote Library API», Intel Corporation, Tech. Rep. Revision 1.3, 2018. [Online]. Available: https://download.01.org/intel-sgx/sgx-dcap/1.3/linux/docs/Intel_SGX_ECDSA_QuoteLibReference_DCAP_API.pdf.
- [53] Standards for Efficient Cryptography Group, «SEC 1: Elliptic Curve Cryptography», Certicom Research, Tech. Rep. Version 2.0, 2009. [Online]. Available: <https://www.secg.org/sec1-v2.pdf>.
- [54] H. Krawczyk and P. Eronen, «HMAC-based Extract-and-Expand Key Derivation Function (HKDF)», Internet Engineering Task Force, Tech. Rep. RFC 5869, 2010. DOI: 10.17487/RFC5869.
- [55] M. Dworkin, «Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC», National Institute of Standards and Technology, Tech. Rep. NIST SP 800-38D, 2007. DOI: 10.6028/NIST.SP.800-38D.
- [56] V. Falco, *Boost.Beast: HTTP and WebSocket Built on Boost.Asio in C++11*, <https://github.com/boostorg/beast>.
- [57] N. Lohmann, *JSON for Modern C++*, <https://github.com/nlohmann/json>.